

Insertion Sort Ricorsivo. La soluzione può comporsi di due funzioni:

```
InsSort(A, j)      # ordina A[1..j]
  if j > 1
    InsSort(A,j-1)  # ordina A[1..j-1]
    Insert(A,j)     # inserisce A[j] in A[1,j-1] ordinato

Insert(A, j)        # inserisce A[j] in A[1,j-1] ordinato

# se j=1 oppure A[j-1] <= A[j],      In poche parole:
# A[j] e' gia' nel posto giusto      ragiona sull'indice "j" e ordina tutti gli elementi
# altrimenti ...                     sequenzialmente (ordinando tutto fino all'indice
                                     precedente a quello attuale).

if (j > 1) and (A[j] < A[j-1])      Fatto questo, se l'indice è valido, sappiamo che il
  A[j] <-> A[j-1]                  massimo è su A[j] (ordinato rispetto ad A[j-1]).
  Insert(A, j-1)
```

La correttezza di `InsSort`, ovvero il fatto che `InsSort(A, j)` ordina il sottoarray `A[1..j]` si dimostra per induzione su j

- ($j = 1$) In questo caso `A[1..j]` contiene un solo elemento e quindi è già ordinato e correttamente la funzione non fa niente.
- ($j > 1$) In questo caso, per ipotesi induttiva, la chiamata `InsSort(A, j-1)` ordina `A[1..j-1]`. Quindi la preconditione per `Insert(A, j)` è soddisfatta, e, dato che `Insert` è corretto (vedi sotto), si conclude che dopo la chiamata di `Insert` l'intero sottoarray `A[1..j]` è ordinato, come desiderato.

Anche la correttezza di `Insert` si dimostra per induzione. Più precisamente dimostriamo che, se `A[1..j-1]` è ordinato, allora `Insert(A, j)` inserisce `A[j]` in `A[1..j-1]`, ovvero restituisce `A[1..j]` ordinato.

- ($j = 1$) In questo caso `A[1..j-1]` è vuoto, quindi `A[j]=A[1]` è già nel posto giusto, `A[1..1]` è ordinato, come desiderato.
- ($j > 1$) Distinguiamo due sottocasi
 - Se `A[j-1] ≤ A[j]`, allora `A[1..j]` è già ordinato, ovvero, anche in questo caso `A[j]` è già al posto giusto.
 - Se `A[j-1] > A[j]`, ricordando che `A[1..j-1]` è ordinato e quindi `A[j-1]` massimo in `A[1..j-1]`, si ha che `A[j-1]` è il massimo di `A[1..j]`. Lo scambio con `A[j-1]` lo porta quindi correttamente in posizione j . A questo punto la chiamata `Insert(A, j-1)`, per ipotesi induttiva inserisce l'elemento `A[j-1]` in `A[1..j-2]` ordinato, quindi ordina `A[1..j-1]`, per cui alla fine `A[1..j]` è ordinato.

Ricerca di duplicati in un array

Verifica la presenza di duplicati

La prima versione dell'esercizio chiede di verificare la presenza di duplicati in un array. Più precisamente, dato un array $A[1..n]$ si vuole verificare se ci sono due indici diversi i e j tali che $A[i] = A[j]$, restituendo conseguentemente un booleano.

L'idea è quella di realizzare una funzione di tipo divide et impera $\text{CheckDup}(A, p, r)$ che verifica la presenza di duplicati in $A[p, r]$ e, qualora non ci siano, ordina l'array $A[p, r]$ (in stile mergesort). Le osservazioni fondamentali sono che

- un array con meno di due elementi non contiene duplicati (caso base)
- un array $A[p, r]$ che contenga almeno due elementi, può essere diviso in due parti $A[p, q]$ e $A[q+1, r]$, e la presenza di duplicati nell'array originale può essere ridotta alla presenza di duplicati in $A[p, q]$ oppure in $A[q+1, r]$ oppure di duplicati "a cavallo", ovvero $i \in [p, q]$ e $j \in [q+1, r]$ tali che $A[i] = A[j]$. Per verificare la presenza di duplicati in $A[p, q]$ e $A[q+1, r]$ si possono effettuare delle chiamate ricorsive, mentre i duplicati "a cavallo" sono ricercati nella fase di merge.

$\text{CheckDup}(A, p, r)$ Se l'esercizio usa due indici ("p" ed "r") --> si chiama l'algoritmo su due pezzi e si chiama Merge

```
if r-p > 1                # se l'array contiene almeno due elementi
    q = (r+p)/2
    return CheckDup(A, p, q) or    # duplicati in A[p,q]?
           CheckDup(A, q, r) or    # duplicati in A[q+1,r]?
           Merge(A, p, q, r)      # duplicati tra A[p,q] e A[q+1,r]?
else
    return False
```

$\text{Merge}(A, p, q, r)$ # assumendo $A[p, q]$ e $A[q+1, r]$ ordinati, verifica la
presenza di duplicati "a cavallo"
se assenti, ordina l'intero sottoarray $A[p, r]$

```
n1 = q-p+1                Inizializzazione degli indici delle due metà di array
n2 = r-q                  e dei due pezzi da considerare per ordinare.
L[1..n1] = A[p..q]
R[1..n2] = A[q+1..r]
L[n1+1] = R[n1+1] = infity
```

```
i = j = 1
k = p                     Quello che cambia nell'algoritmo di Merge è questa condizione
while (k <= r) and (L[i] <> R[j])    # i duplicati "a cavallo" saranno
                                     # consecutivi
    if L[i] < R[j]
        A[k] = L[i]
        i += 1
    else
        A[k] = R[j]
        j += 1
```

In questo modo, ordino la parte destra e la parte sinistra, avanzando gli indici delle due metà, solo se non ci sono dei duplicati (ecco cosa vuol dire duplicati "a cavallo").

```
return (k <= r)            Ritorno poi l'indice fino a dove è stato ordinato.
                           # l'assenza di duplicati corrisponde
                           # al completamento del merge (k <= r)
                           # in questo caso, l'intero sottoarray
                           # è ordinato
```

La complessità nel caso peggiore è la stessa del mergesort, quindi $\Theta(n \log n)$. Tuttavia, si osservi che se pensiamo che l'or della funzione CheckSup sia di tipo "lazy", ovvero appena trova un valore

true, si ferma e restituisce true senza valutare i rimanenti argomenti, il costo può essere anche inferiore.

Conta i duplicati

La seconda versione dell'esercizio, richiede di contare duplicati, ovvero le coppie di indici (i, j) con $i < j$ e $A[i] = A[j]$.

Si può raffinare la soluzione precedente osservando che se indichiamo con $dup(A, p, r)$ il numero di duplicati in $A[p..r]$ ovvero

$$dup(A, p, r) = |\{(i, j) \mid p \leq i < j \leq r \wedge A[i] = A[j]\}|$$

allora

- un sottoarray con meno di due elementi, ovvero con $p \leq r$, non contiene nessun duplicato, $dup(A, p, r) = 0$;
- un array $A[p, r]$ che contenga almeno due elementi, può essere diviso in due parti $A[p, q]$ e $A[q+1, r]$, e il numero di duplicati nell'array originale può essere ottenuto sommando il numero di duplicati in $A[p, q]$, quelli in $A[q+1, r]$ e i duplicati "a cavallo", ovvero $i \in [p, q]$ e $j \in [q+1, r]$ tali che $A[i] = A[j]$. Indicato con

$$crossDup(A, p, r) = |\{(i, j) \mid p \leq i < j \leq r \wedge A[i] = A[j]\}|$$

si ha dunque

$$dup(A, p, r) = dup(A, p, q) + dup(A, q+1, r) + crossDup(A, p, q, r)$$

Le considerazioni fatte conducono all'algoritmo che segue:

```
CountDup (A, p, r)
  if r-p > 1
    q = (r+p)/2
    return CountDup (A, p, q) +      # duplicati in A[p,q]
           CountDup (A, q, r) +      # duplicati in A[q+1,r]
           CrossDup (A, p, q, r)      # duplicati tra A[p,q] e A[q+1,r]
  else
    return 0
```

Il ragionamento è questo: prendo le due metà di array e conto i duplicati delle due metà moltiplicandoli (perché possono esserci dei duplicati da entrambe le metà dell'array e quindi si rende necessario moltiplicare).

Nella fase di merge, l'idea è di tener traccia di quanti elementi identici sono inseriti consecutivamente da L e R : un blocco di $countL$ elementi da L e di $countR$ elementi da R , determina $countL * countR$ duplicati a cavallo.

```
# funzione che dati il numero di elementi consecutivi inseriti nel
# merge da L e R rispettivamente, ritorna il corrispondente numero di
# duplicati "a cavallo", ovvero countL * countR
```

```
Check (countL, countR)
  return countL*countR
```

```
CrossDup (A, p, q, r)
  # assumendo A[p,q] e A[q+1,r] ordinati, ritorna
  # il numero di duplicati "a cavallo"
  # e ordina l'intero sottoarray A[p,r]
```

```
n1 = q-p+1
n2 = r-q
L[1..n1] = A[p..q]
```

Come sempre, inizializziamo gli indici per l'analisi delle due metà di array.

```

R[1..n2] = A[q+1..r]
L[n1+1] = R[n1+1] = infity

i = j = 1

# crossDup contiene i duplicati a cavallo tra A[p,i-1] e
# A[q+1,j-1], inizialmente 0
crossDup = 0

# current contiene l'ultimo elemento inserito da L nel merge,
# mentre countL e countR indicano quanti elementi consecutivi uguali a
# current sono stati inseriti da L e da R rispettivamente

# inizialmente, dato che il processo di merge deve ancora iniziare,
# current e' inizializzato ad un valore inesistente
# countL e countR sono inizializzate a zero
current= infity
countL=countR=0

for k = p to r

    if (L[i] <= R[j])
        if L[i] == current
            countL++
        else
            # altrimenti aggiorna crossDup usando countL e countR
            crossDup = crossDup + check(countL, countR)

            current = L[i]
            countL = 1
            countR = 0

        A[k] = L[i]

        i++

    else:
        if R[j] == current:
            countR++

        A[k] = R[j]
        j++

# aggiorna con i valori countL e countR calcolati nell'ultima parte del ciclo
crossDup = crossDup + check(countL, countR)

return crossDup

```

Conta gli indici e valori duplicati

Se volessimo contare gli indici duplicati ovvero

$$dupIdx(A, p, r) = |\{i \mid \exists j. j \neq i \wedge A[i] = A[j]\}|$$

è sufficiente osservare che se ho un blocco di *countL* elementi da *L* e di *countR* elementi da *R* identici

Ultima variante dell'esercizio;
capire quali sono gli indici dei valori duplicati (primo algoritmo)
e
contare i valori duplicati (secondo algoritmo)

- se $countL = 0$ o $countR = 0$ non si identificano nuovi duplicati
- se $countL = 1$ e $countR = 1$, i due elementi sono due nuovi indici duplicati; (cioè, ci sono due duplicati, uno a sx ed uno a dx)
- se $countL > 1$ e $countR = 1$, gli indici in L erano già stati identificati come duplicati, mentre l'unico indice in R rappresenta un nuovo duplicato; (cioè, avendo $countL > 1$, significa che è un indice già apparso più volte a sx, ma a dx è un duplicato una volta sola)
- se $countL = 1$ e $countR > 1$, la situazione è duale. (cioè, avendo $countR > 1$, significa che è un indice già apparso più volte a dx, ma a sx è un duplicato una volta sola)

Pertanto, sarà sufficiente modificare la funzione `Check(countL, countR)` come segue:

```
Check(countL, countR)
if (countL==1) and (countR==1)
    res = 2
elseif ((countL==1) and (countR>1)) or ((countL>1) and (countR==1))
    res = 1
else
    res = 0
return res
```

Analogamente, se volessimo contare i valori ripetuti, ovvero

$$dupVal(A, p, r) = |\{A[i] \mid \exists j. j \neq i \wedge A[i] = A[j]\}|$$

è sufficiente osservare che se ho un blocco di $countL$ elementi da L e di $countR$ elementi da R identici, verifico la presenza di un nuovo valore duplicato solo se entrambi i blocchi hanno dimensione 1. Pertanto, sarà sufficiente modificare la funzione `Check(countL, countR)` come segue:

```
Check(countL, countR)
if (countL==1) and (countR==1)
    res = 1
else
    res = 0
return res
```

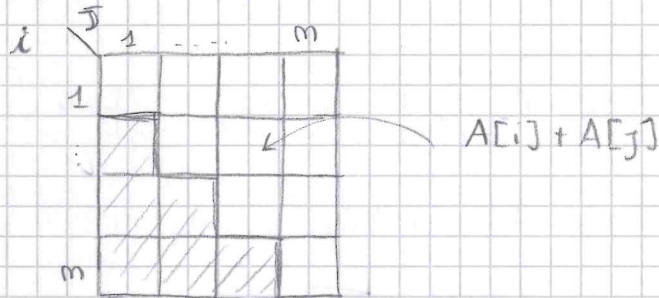
Se $countL$ oppure $countR$ sono pari ad 1, significa che esiste un nuovo valore duplicato, altrimenti (si guardi l'algoritmo precedente), significa che un duplicato appare più volte o a sx oppure a dx e quindi, non è da contare. A noi interessa, essendo merge sort, un duplicato su entrambe le metà degli array.

In tutti i casi la complessità resta sempre $\Theta(n \log n)$ nel caso peggiore.

ESERCIZIO: Realizzare una funzione $\text{Sum}(A, \text{key})$ che dato un array A ed un intero key verifica se key è esprimibile come somma di elementi di A (14/10)

Soluzione brutale:

considero tutte le somme



$\text{Sum}(A, \text{key})$

$i = 1$

$j = 1$

while $(i \leq m) \text{ and } (A[i] + A[j] \neq \text{key})$

if $j = m$

$i = i + 1$

$j = i$

else

$j = j + 1$

return $(i \leq m)$

costo $\mathcal{O}(m^2)$

Devo verificare se la somma degli elementi è pari a "key" (somma di elementi corrisponde ad avere due indici)
Vediamo che:

1) se non abbiamo ancora raggiunto l'ultima riga e la somma degli elementi non è pari a "key":
- se non abbiamo ancora raggiunto l'ultima colonna, incrementiamo "j" (dato che aumenta anche "i", ci stiamo spostando in diagonale)
- se abbiamo raggiunto l'ultima colonna, incrementiamo anche "i" e abbiamo percorso tutto l'array in diagonale.

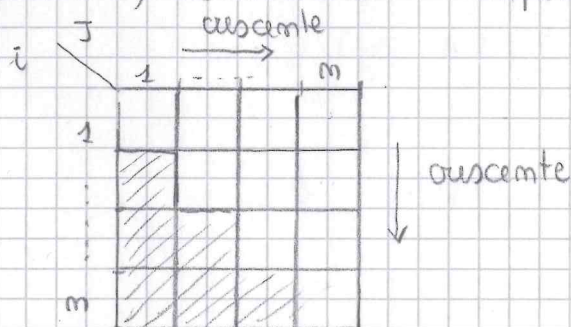
2) se invece abbiamo raggiunto l'ultima riga oppure abbiamo trovato che i due indici, diagonalmente, corrispondono per somma a "key", restituiamo l'indice di match.

L'algoritmo ha complessità n^2 , dato che usiamo due indici; tuttavia è inefficiente, perché scandiamo tutto l'array inutilmente.

Se l'array fosse ordinato, allora computazionalmente si risparmia abbastanza.

* Si può fare di meglio? implicitamente, le somme.

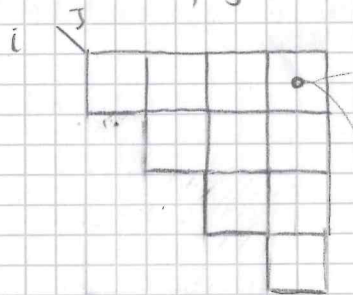
Sì, ordinando l'array e quindi, Infatti, se A è ordinato



anche le somme $A[i] + A[j]$ spostandosi a dx e in basso ciascuno

posso sfruttare utilmente questo fatto

parto con $i=1, j=m$



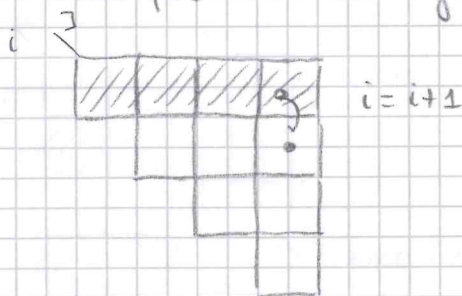
Ecco che, avendolo ordinato, quando $j=n$, abbiamo l'elemento maggiore di tutta la riga i , per cui:

- se la somma degli elementi è minore di key , è minore tutta la riga (dato che letteralmente, ogni singolo elemento è minore di key)
- se la somma degli elementi è pari a key , abbiamo finito (abbiamo trovato il match maggiore, essendo la riga ordinata).

se $A[i] + A[j] = key$ fine!

se $A[i] + A[j] < key$

è vale per l'intera riga



se $> key \rightarrow$ elimino la colonna

$j = j - 1$

in questo modo restringo lo spazio di ricerca concludendo in tempo lineare

Sum (A, key)

Sort (A)

Immaginiamo di usare un generico Sort (a meno di richieste particolari dell'esercizio, si può sempre assumere). Inizializziamo $i=1$ e j pari alla fine

$i = 1$

$j = A.length$

while ($i \leq j$) and ($A[i] + A[j] \neq key$)

if $A[i] + A[j] < key$

$i = i + 1$

else

$j = j - 1$

return $i \leq j$

Ecco che l'algoritmo migliora.

Cambia la condizione (j è pari ad n e si mette $i \leq j$), ma si guarda sempre la somma di una coppia di elementi.

Ecco che:

1) se la somma è minore di key , come detto, è minore tutta la riga e avanziamo in riga (i).

1) se la somma è maggiore di key , come detto, è maggiore tutta la riga e torniamo indietro in colonna (j).

Trovato l'indice, viene restituito.

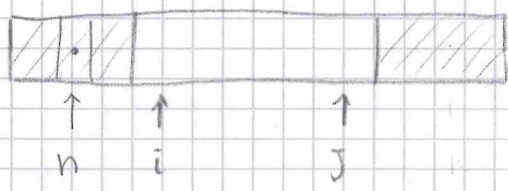
La correttezza ragiona sull'indice:

se la somma è diversa da key rispetto ad i , ogni elemento a sx non produce key

se la somma è diversa da key rispetto a j , ogni elemento a dx non produce key

Una volta trovato, viene restituito.

Qual è l'invarianza?

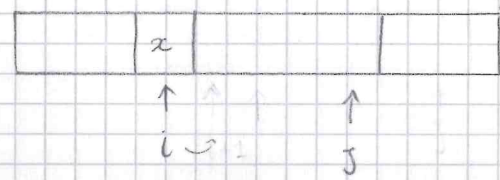


- (a) $\forall h \in [1, i-1] \quad \forall k \in [h, m] \quad A[h] + A[k] \neq \text{key}$
 (ogni elemento della parte scorciata o sx
 sommato a qualunque elemento successivo non produce key)
- (b) $\forall k \in [j+1, m] \quad \forall h \in [1, k] \quad A[h] + A[k] \neq \text{key}$
 (duale)

→ inizializzazione: banalmente vero all'inizio

→ mantenimento:

- se $A[i] + A[j] < \text{key}$ $\leadsto$ quindi spostato $i \leadsto i+1$



(a) resta vera, infatti
 per $k \in [i, j]$

$$A[k] \leq A[j]$$

$$\Rightarrow A[i] + A[k] \leq A[i] + A[j] < \text{key}$$

per $k \in [j+1, m]$

$i \in [1, k-1]$ quindi per (b)
 $A[i] + A[k] \neq \text{key}$

(b) non influenzato

- se $A[i] + A[j] > \text{key}$
 duale



Conclusione =

- se $A[i] + A[j] = \text{key}$ ok!

- se $i > j \leadsto i = j + 1$

(a) e (b) implicano che la somma non si può ottenere

precisamente per $h, k \in [1..m] \quad h \leq k$

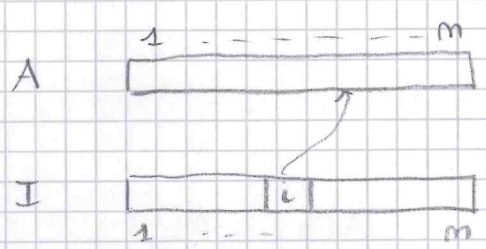
• $h \leq i$ per (a) $A[h] + A[k] \neq \text{key}$

• altrimenti $h > i \Rightarrow k \geq j \Rightarrow$ per (b) $A[h] + A[k] \neq \text{key}$

* Complessità $O(m)$ (incluso l'ordinamento $O(m \log m)$)

→ se volessimo gli indici i, j dell'array originale?

idea = costruiamo un array indice $I[1..m]$ e lavoriamo su quello!



Banalmente, se volessimo gli indici in diagonale, si costruisce un array I per cui, ogni volta che ci spostiamo in diagonale, usiamo A per salvare l'indice attuale (quindi, piuttosto che usare

$I[i] = A[i]$
oppure
 $I[j] = A[j]$

si fa subito
 $A[I[i]]$
oppure
 $A[I[j]]$.

Usiamo l'array I solo per salvare la posizione dell'indice somma (e basta)

tali che $i \leq j \Rightarrow A[I[i]] \leq A[I[j]]$

Array delle occorrenze

Se ho un limite inferiore per i valori in A (ad es. sono tutti ≥ 0) posso mantenere un array delle occorrenze "utili" che ricordi la presenza in A di elementi che possono concorrere a formare una somma = Key (quindi nello specifico $\leq \text{Key}$)

(Quindi, se esistono elementi da contare (questo significa "limite inferiore", quindi un numero > 0 di elementi), allora posso contare gli elementi che per somma danno "key" e quante volte appaiono).



com $\forall [n] = \text{true}$ sse $\exists i \in [1, m]$ t.c
 $A[i] = n$

la funzione può quindi essere la seguente:

sum(A, key)

$$V[0..key] \leftarrow \text{false}$$
$$i = 1$$

found = false

while ($i \leq m$) and not found

Se non abbiamo ancora trovato l'elemento

if ($A[i] \leq \text{key}$) Se l'elemento i-esimo è minore di key

$$\forall [A[i]] = \text{true}$$
$$\text{found} = V[\text{key} - A[i]]$$

U++

return found

Dato che vogliamo contare la somma fino a "key", inizializziamo V fino a "key" per dimensione; sarà inizializzato a "false" per capire quali sono gli elementi che sono minori a "key" (che saranno poiisti a true e quindi, per somma, contribuiranno ad ottenere "key").

Similmente, inizializziamo l'indice "i" e "found" a false, per valori di default.

Usiamo A "in-place" in V (quindi $V[i] = A[i]$ diventa $V[A[i]]$)
e diciamo, mettendolo uguale a "true" la seguente cosa
"l'elemento i-esimo contribuisce alla somma fino a key".
Dato che l'elemento contribuisce, togliamo a V da "key" (sarebbe l'indice finale "n") l'indice attuale, fino a trovare iterativamente l'elemento.

Complessità lineare.

Correttezza:

1) se $i = 0$, allora la somma per key è $=0$
ed è corretto

2) se $i > 0$, si verifica se l'elemento i -esimo contribuisce alla somma

- se $i < n$, significa che l'elemento è stato trovato e:

- se $A[i] \leq \text{key}$, allora l'elemento contribuisce alla somma

- se $A[i] > \text{key}$, vado avanti. Anche se non lo trovassi, comunque siamo sicuri che $i < \text{key}$ e l'elemento somma verrà trovato

3) trovo l'elemento somma e restituisco

Monete

(Questo esercizio è semplice discorsivo di ragionamento che usa Merge Sort: io inserisco anche una versione sotto forma di algoritmo. Esercizio di ragionamento, ma non per altro).

Siano date n monete delle quali una falsa. Si supponga che tutte le monete abbiano lo stesso peso, tranne quella falsa che pesa di meno. Si chiede di fornire un algoritmo di complessità $O(\log n)$ che determini la moneta falsa usando una bilancia a due piatti che può confrontare il peso di sottoinsiemi di monete. Mostrare che il problema ha complessità $\Omega(\log n)$ e quindi l'algoritmo è ottimo. Si può generalizzare al caso in cui si sa solo che la moneta falsa ha un peso diverso da quelle vere, ma non è noto se pesi di più o di meno?

Soluzione Lavoriamo nell'ipotesi che la moneta falsa pesi di meno e iniziamo rispondendo alla domanda sulla complessità del problema, ovvero mostrando che il problema ha complessità $\Omega(\log n)$.

Detto S l'insieme di monete, una *pesata* è un'operazione che prende due sottoinsiemi disgiunti di monete S_1 e S_2 e li confronta tramite la bilancia. La pesata sarà maggiormente informativa se i due insiemi scelti contengono lo stesso numero di monete. Più precisamente abbiamo

$$S = S_1 \uplus S_2 \uplus S_3$$

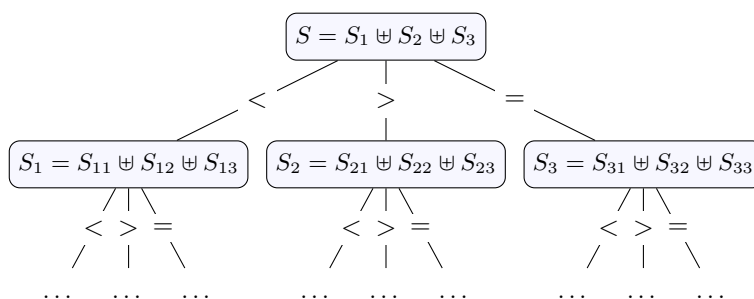
con $|S_1| = |S_2|$ e $S_i \cap S_j = \emptyset$ per $i \neq j$. In questa situazione, se:

- peso $S_1 <$ peso S_2 : la moneta falsa è in S_1 ;
- peso $S_1 =$ peso S_2 : la moneta falsa è in S_3 ;
- peso $S_1 >$ peso S_2 : la moneta falsa è in S_2 .

Quindi ci si riduce a cercare la moneta in uno dei tre sottoinsiemi.

A margine, si ribadisce che considerare sottoinsiemi S_1 e S_2 che contengano un numero diverso di monete non è conveniente. Infatti, supponendo ad esempio che $|S_1| < |S_2|$, se peso $S_1 <$ peso S_2 non si ha alcuna informazione, la pesata è stata inutile!

L'esecuzione di un qualunque algoritmo che evidenzi le operazioni "pesata" può essere rappresentato mediante un albero di decisione del tipo:



Si osserva che, qualunque sia l'algoritmo seguito

- in ciascuno nodo, almeno uno dei sottoinsiemi ha cardinalità maggiore o uguale a

$$(\text{numero totale di monete nel nodo})/3;$$

- le foglie, per portare ad una decisione, devono avere cardinalità 1.

quindi l'albero ha altezza $\geq \log_3 n$.

La complessità (numero di pesate) dell'algoritmo nel caso peggiore è data dal cammino più lungo dalla radice ad una foglia (altezza dell'albero). Infatti, al variare dell'input, essendo le monete indistinguibili, potrò ottenere un'esecuzione corrispondente ad uno qualunque dei cammini.

Dato che questo vale per ogni algoritmo che risolve il problema, $\log_3 n$ è un limite inferiore per la complessità, e dunque il problema ha complessità $\Omega(\log n)$.

Algoritmo: Un algoritmo ottimo si può ottenere dividendo l'insieme corrente di n monete in tre sottoinsiemi $S = S_1 \uplus S_2 \uplus S_3$ con $|S_1| = |S_2| = \lfloor n/3 \rfloor$. Questo porta ad una complessità $O(\log_3 n)$, quindi ottima. Anche una suddivisione con $|S_1| = |S_2| = \lfloor n/2 \rfloor$ porta ad un algoritmo asintoticamente ottimo, dato che la complessità sarà $O(\log_2 n)$ e la base del logaritmo è irrilevante dal punto di vista del limite asintotico.

Se non è noto se la moneta falsa pesa di più o di meno: L'algoritmo precedente può essere facilmente adattato purché $n \geq 3$. Alla prima pesata, si considerano $|S_1| = |S_2| = \lfloor n/3 \rfloor$. Se S_1 pesa di più, significa che S_3 è interamente costituito da monete vere. Dunque, preso un sottoinsieme S'_3 di S_3 con cardinalità $\lfloor n/3 \rfloor$, e confrontatolo con S_1 :

- se S'_3 pesa di meno di S_1 , allora S_1 contiene la moneta falsa e la moneta falsa pesa di più di quelle vere, si continua dunque come nel già trattato (dualizzando);
- se S'_3 e S_1 hanno lo stesso peso, allora S_2 contiene la moneta falsa, che pesa di meno di quelle vere e si continua dunque come nel caso precedente;
- si noti che non può essere che S'_3 pesi di più di S_1 , perché S_1 a sua volta pesa più di S_2 , mentre il peso di S'_3 deve coincidere con quello di S_1 o di S_2 .

Se invece alla prima pesata, S_1 e S_2 hanno lo stesso peso, sono entrambi costituiti da monete vere, posso dunque estrarre da $S_1 \cup S_2$ un sottoinsieme di monete vere S' con la stessa cardinalità di S_3 . Confrontando S' con S_3 capirò se la moneta falsa, necessariamente contenuta in S_3 , pesa di più o di meno di quelle vere. A questo punto si continua come prima.

In ogni caso, aggiungo un'unica pesata all'algoritmo, la cui complessità asintotica non cambia.

1. Prendiamo come input il numero di monete, n , e il loro peso, con quella falsa che ha un peso diverso da quella vera.
 2. Se è presente una sola moneta, questa sarà la moneta falsa.
 3. Se ci sono più monete, allora dividete le monete in mucchietti di $A = \text{floor}(n/3)$, $B = \text{floor}(n/3)$ e $C = n - 2 * \text{floor}(n/3)$.
 4. Pesare A e B, se la bilancia è in equilibrio allora ripetere dal primo passo con il numero totale di monete pari al numero di monete in C.
 5. Se la bilancia si sbilancia, allora ripetere dal punto 1, con l'alleggerimento di A e B.
- Se $n \bmod 3 = 0$, allora i tre mucchietti saranno di dimensioni k, k, k .
 Se $n \bmod 3 = 1$, allora tre pile avranno dimensioni $k, k, k + 1$.
 Se $n \bmod 3 = 2$, le tre pile avranno dimensioni $k + 1, k + 1, k$.

Es: - Supponiamo che ci siano 12 monete di cui 11 sono vere e una è più leggera, quindi, trovare quella falsa con la minor pesatura possibile.

Poiché il numero di 12 monete è divisibile per 3, dividiamo per 3 e formiamo pile di dimensioni 4, cioè A, B e C.

Ora, sono possibili solo 3 situazioni:

1. Confronto tra A e B, peso di $A > B$.
2. Peso di $A < B$.
3. Peso di $A = B$.

Se $A > B$, allora è chiaro che la moneta falsa si trova nella pila B, dato che la pila B ha 4 monete e sappiamo che $4 \bmod 3 = 1$, quindi creiamo pile di dimensioni 1, 1 e 2.

Allo stesso modo, se $A < B$, allora è chiaro che la moneta falsa si trova nella pila A e si procederà come nel caso di $A > B$.

Se invece $A = B$, allora è chiaro che la moneta falsa si trova nel mucchio C e anche in questo caso divideremo il mucchio C in gruppi come indicato nel caso in cui $A > B$.

Confrontiamo le pile di dimensioni 1 e 1. Anche in questo caso, sono possibili solo 3 situazioni, ossia se una delle due è più leggera allora è la moneta falsa o se entrambe hanno lo stesso peso allora la moneta falsa deve trovarsi nella pila di dimensioni 2 e, quindi, dividiamo di nuovo le 2 monete in 3 gruppi, ossia di dimensioni 1, 1 e 0, e confrontiamo le pile di dimensioni 1, quella che è più leggera è la moneta falsa.

Implementazione

```
import random
real_coin = 1
fake_coin = 0

def randomize_coins(n):
    global coins
    coins = [real_coin] * (n - 1) + [fake_coin] * (1)
    random.shuffle(coins)
    print(coins)
    return coins

def testing_fake_one(x,y,n):
    if n==1:
        print(1,"st coin is the fake one")
    else:
        if n % 3==0 or n%3==1:
            A=[coins[i] for i in range(x,x+(y-x)//3 )]
            B=[coins[i] for i in range(x+(y-x)//3,x+2*(y-x)//3)]
            C=[coins[i] for i in range(x+2*(y-x)//3,y)]
            coins_index_A=[i+1 for i in range(x,x+(y-x)//3)]
```

```

coins_index_B=[i+1 for i in range(x+(y-x)//3,x+2*(y-x)//3)]
coins_index_C=[i+1 for i in range(x+2*(y-x)//3,y)]
if sum(A)<sum(B):
    if len(A)>1:
        testing_fake_one(x,x+(y-x)//3,(y-x)//3)
    if len(A)==1:
        return print(coins_index_A[0],"th coin is the fake coin")
elif sum(A)>sum(B):
    if len(B)>1:
        testing_fake_one(x+(y-x)//3,x+2*(y-x)//3,(y-x)//3)
    if len(B)==1:
        return print(coins_index_B[0],"th coin is the fake coin")
else:
    if len(C)>1:
        testing_fake_one(x+2*(y-x)//3,y,y-2*(y-x)//3-x)
    if len(C)==1:
        return print(coins_index_C[0],"th coin is the fake coin")
else:
    A=[coins[i] for i in range(x,x+(y-x)//3+1)]
    B=[coins[i] for i in range(x+(y-x)//3+1,x+2*(y-x)//3+1)]
    C=[coins[i] for i in range(x+2*(y-x)//3+1,y)]
    coins_index_A=[i+1 for i in range(x,x+(y-x)//3+1)]
    coins_index_B=[i+1 for i in range(x+(y-x)//3+1,x+2*(y-x)//3+1)]
    coins_index_C=[i+1 for i in range(x+2*(y-x)//3+1,y)]
    if sum(A)<sum(B):
        if len(A)>1:
            testing_fake_one(x,x+(y-x)//3+1,(y-x)//3+1)
        if len(A)==1:
            return print(coins_index_A[0],"th coin is the fake coin")
    elif sum(A)>sum(B):
        if len(A)>1:
            testing_fake_one(x+(y-x)//3+1,x+2*(y-x)//3+1,(y-x)//3)
        if len(A)==1:
            return print(coins_index_B[0],"th coin is the fake coin")
    else:
        if len(A)>1:
            testing_fake_one(x+2*(y-x)//3+1,y,y-2*(y-x)//3-1-x)
        if len(A)==1:
            return print(coins_index_C[0],"th coin is the fake coin")

```

```

n=int(input("Enter the number of coins:"))
randomize_coins(n)
testing_fake_one(0,n,n)

```

Esercizio

Dimostrare le seguenti uguaglianze:

1. $f(n) = O(g(n))$ sse $g(n) = \Omega(f(n))$
2. $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
3. $f(n) = \Theta(f(n))$
4. $f(n) = O(g(n))$ e $g(n) = O(h(n))$ implica $f(n) = O(h(n))$
(e dualmente $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ implica $f(n) = \Omega(h(n))$),
e quindi $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implica $f(n) = \Theta(h(n))$
5. $f(n) = \Theta(g(n))$ sse $\Theta(f(n)) = \Theta(g(n))$ sse $g(n) = \Theta(f(n))$

Fondalmente si applicano sempre le definizioni e letteralmente si risolve. Si ricordi che poi deve valere per almeno una costante (vuol dire, sostituisco un numero, ad esempio $c=1$ ed $n_0 =$ qualche numero)

Soluzione. È opportuno richiamare la definizione dei limiti asintotici superiore, inferiore e stretto:

- (i) $\Omega(g(n)) = \{f(n) \mid \exists c > 0. \exists n_0. \forall n \geq n_0. 0 \leq c g(n) \leq f(n)\}$
- (ii) $O(g(n)) = \{f(n) \mid \exists d > 0. \exists n_0. \forall n \geq n_0. 0 \leq f(n) \leq d g(n)\}$
- (iii) $\Theta(g(n)) = \{f(n) \mid \exists c, d > 0. \exists n_0. \forall n \geq n_0. 0 \leq c g(n) \leq f(n) \leq d g(n)\}$

Procediamo dunque con la soluzione delle varie domande

1. $f(n) = O(g(n))$ sse $g(n) = \Omega(f(n))$
Proviamo che se $f(n) = O(g(n))$ allora $g(n) = \Omega(f(n))$. Supponiamo dunque che valga $f(n) = O(g(n))$, ovvero esistono $d > 0$ e n_0 tali che per ogni $n \geq n_0$ vale

$$0 \leq f(n) \leq d g(n)$$

Dividendo per d (possibile perché $d > 0$) si ottiene che per ogni $n \geq n_0$ vale

$$0 \leq \frac{1}{d} f(n) \leq g(n)$$

e quindi, ricordando la definizione (i), $g(n) = \Omega(f(n))$, utilizzando come costante moltiplicativa $c = \frac{1}{d} > 0$.

L'implicazione inversa è totalmente analoga.

2. $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

Mostriamo separatamente le due inclusioni.

- $\Theta(g(n)) \subseteq O(g(n)) \cap \Omega(g(n))$:
Sia $f(n) = \Theta(g(n))$. Dalla definizione (iii) otteniamo che esistono $c, d > 0$, n_0 tali che per ogni $n \geq n_0$ vale

$$0 \leq c g(n) \leq f(n) \leq d g(n)$$

È immediato dalle definizioni (ii) e (i) dedurre che $f(n) = \Omega(g(n))$ (con costante moltiplicativa c) e $f(n) = O(g(n))$ (con costante moltiplicativa d). Quindi $f(n) = O(g(n)) \cap \Omega(g(n))$.

- $O(g(n)) \cap \Omega(g(n)) \subseteq \Theta(g(n))$:

Sia $f(n) = O(g(n)) \cap \Omega(g(n))$. Dato che $f(n) = \Omega(g(n))$ esistono $c > 0$, n'_0 tali che per ogni $n \geq n'_0$ vale

$$0 \leq c g(n) \leq f(n)$$

Analogamente, dato che $f(n) = O(g(n))$ esistono $d > 0$, n''_0 tali che per ogni $n \geq n''_0$ vale

$$0 \leq f(n) \leq d g(n)$$

Quindi, detto $n_0 = \max(n'_0, n''_0)$ si ha che per ogni $n \geq n_0$

$$0 \leq c g(n) \leq f(n) \leq d g(n)$$

e pertanto $f(n) = \Theta(g(n))$, come desiderato.

3. $f(n) = \Theta(f(n))$

Immediato dalla definizione (iii), utilizzando come costanti moltiplicative $c = d = 1$ e n_0 qualsiasi.

4. $f(n) = O(g(n))$ e $g(n) = O(h(n))$ implica $f(n) = O(h(n))$

Sia $f(n) = O(g(n))$ e $g(n) = O(h(n))$. Quindi, dalla definizione (ii), esistono $d' > 0$, n'_0 tali che per ogni $n \geq n'_0$ vale:

$$0 \leq f(n) \leq d' g(n)$$

e analogamente esistono $d'' > 0$, n''_0 tali che per ogni $n \geq n''_0$ vale:

$$0 \leq g(n) \leq d'' h(n)$$

Pertanto, posto $n_0 = \max(n'_0, n''_0)$ si ha che per ogni $n \geq n_0$ risulta

$$0 \leq f(n) \leq d' g(n) \leq d' d'' h(n)$$

ovvero $f(n) = O(h(n))$, con costante moltiplicativa $d = d' d''$.

5. $f(n) = \Theta(g(n))$ sse $\Theta(f(n)) = \Theta(g(n))$ sse $g(n) = \Theta(f(n))$

Osserviamo che $f(n) = \Theta(g(n))$ sse $g(n) = \Theta(f(n))$ segue dai punti (1) e (2). Infatti,

$$\begin{array}{ll} f(n) = \Theta(g(n)) \text{ sse} & [\text{usando (2)}] \\ f(n) = \Omega(g(n)) \text{ e } f(n) = O(g(n)) \text{ sse} & [\text{usando (1) due volte}] \\ g(n) = O(f(n)) \text{ e } g(n) = \Omega(f(n)) \text{ sse} & [\text{usando (2)}] \\ g(n) = \Theta(f(n)) & \end{array}$$

Resta solo da dimostrare $f(n) = \Theta(g(n))$ sse $\Theta(f(n)) = \Theta(g(n))$. Proviamo separatamente le due implicazioni.

Sia $f(n) = \Theta(g(n))$. Dobbiamo provare che $\Theta(f(n)) = \Theta(g(n))$. Se $h(n) = \Theta(f(n))$ allora per (4) $h(n) = \Theta(g(n))$, quindi vale $\Theta(f(n)) \subseteq \Theta(g(n))$. L'inclusione opposta segue per simmetria, dato che abbiamo appena dimostrato che $f(n) = \Theta(g(n))$ sse $g(n) = \Theta(f(n))$.

Sia ora $\Theta(f(n)) = \Theta(g(n))$. É sufficiente ricordare che per (3) vale $f(n) = \Theta(f(n))$, da cui si deduce che $f(n) = \Theta(g(n))$.

Altri esercizi per casa:

$$(a) \quad T(m) = 2T\left(\frac{m}{2}\right) + \log m$$

$$(b) \quad T(m) = 2T\left(\frac{m}{2}\right) + m^2$$

$$(c) \quad T(m) = 2T\left(\frac{m}{2}\right) + m \log m$$

Ricordiamo che si basa al MT:

$T(n) = a \cdot T(n/b) + f(n)$, per cui:

Caso (1) $\rightarrow f(n) = O(n^{(\log_b(a) - \epsilon)))$, allora $T(n) = \Theta(n^{(\log_b(a))})$
Quindi, vince $n^{(\log_b(a))}$.

Caso (2) $\rightarrow f(n) = \Theta(n^{(\log_b(a))})$, allora $T(n) = \Theta(n^{(\log_b(a))} \cdot \log(n))$
Quindi pareggio

Caso (3) $\rightarrow f(n) = \Omega(n^{(\log_b(a) + \epsilon)))$ e vale la regolarità:
esiste un k tra 0 ed 1 tale che $a \cdot f(n/b) \leq k \cdot f(n)$ allora $T(n) = \Theta(f(n))$
Quindi, vince $f(n)$.

Comunque, si tratta di confrontare $f(n)$ rispetto a $n^{(\log_b(a))}$.

(a) Rispetto allo schema generale
 $a=2, b=2$

$$n^{(\log_2(2))}=n$$

$$m^{\log_2 2} = m$$

$$f(m) = \log m$$

$$\leadsto f(m) = \log m = O(m)$$

e per ogni $0 < \epsilon < 1$

$$\log m = O(m^{1-\epsilon})$$

dove $O(n^{(1-\epsilon)})$ è dato da $O(n^{(\log_b(a) - \epsilon)})$
(basta sostituire $n^{(\log_b(a))}$ a questa di fianco)

Quindi siamo nel caso (1)

$$T(m) = \Theta(m)$$

Banalmente, qui tra:

$$f(n) = \log(n)$$

$$n^{(\log_b(a))} = n$$

Quello che tende più velocemente a 0, per $n \rightarrow \infty$ è $\log(n)$.

Quindi, è più grande $n \rightarrow$ caso (1).

(b) Rispetto allo schema generale

$$a=2$$

$$b=2$$

$$m^{\log_2 2} = m^1 = m$$

$$f(m) = m^2$$

si ha che

$$f(m) = m^2 = \Omega(m)$$

Quindi, il più grande è $f(n)$.

Deve valere anche la condizione di regolarità.

e di più

$$= \Omega(m^{1+\epsilon})$$

dove $O(n^{(1+\epsilon)})$ è dato da $O(n^{(\log_b(a) + \epsilon)})$
(basta sostituire $n^{(\log_b(a))}$ a questa di fianco)

per $0 < \epsilon < 1$

$\leadsto$

Inoltre vale la condizione di regolarità:

$$a f\left(\frac{m}{b}\right) \leq K f(m) \quad \text{per } K < 1$$

$$2 \left(\frac{m}{2}\right)^2 \leq K m^2$$

$$\frac{m^2}{2} \leq K m^2 \quad \text{ovv. } K = \frac{1}{2}$$

perciò si conclude

$$T(m) = \Theta(m^2)$$

Se vale la condizione di regolarità, siamo per certo nel caso 3.

(c) Rispetto allo schema generale

$$a = 2 \quad b = 2$$

$$m^{\log_b a} = m$$

$$f(m) = m \log m$$

Il più grande tra i due sarebbe "n" (per $\lim_{n \rightarrow \infty}$, che infatti risulta 0). Pensiamo di essere nel caso 3, ma:

- è inammissibile perché la differenza tra $n \log(n)$ e $n \log_b(a)$ non è polinomiale. (cioè, il limite tra $f(n)$ ed $n^{\log_b(a)}$ va ad infinito e non vale per nessun epsilon)

Vale che

$$f(m) = m \log m = \Omega(m)$$

ma invece

$$\neq \Omega(m^{1+\epsilon}) \quad \forall \epsilon > 0$$

quindi non possiamo usare il Master Theorem

(Non vale neppure la condizione di regolarità)

$$a f\left(\frac{m}{b}\right) = 2 \frac{m}{2} \log \frac{m}{2} = m \log \frac{m}{2}$$

vorremmo (*)

Appunto, non vale neanche questa condizione, varrebbe 1 il calcolo, ma in realtà dovrebbe essere 0.

$$a f\left(\frac{m}{b}\right) = m \log \frac{m}{2} \leq K f(m) = K m \log m \quad K < 1$$

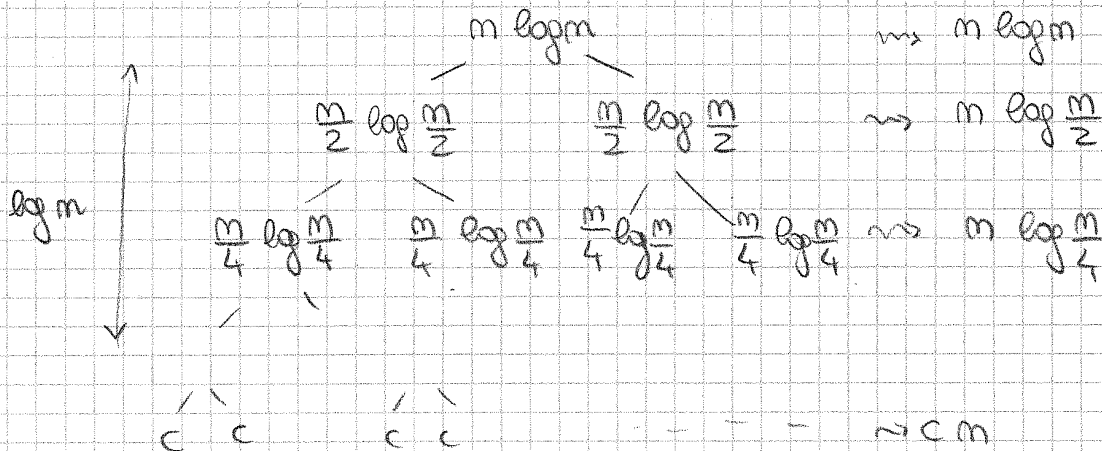
ma

$$\frac{m \log \frac{m}{2}}{m \log m} = \frac{\log m - \log 2}{\log m} \rightarrow 1$$

quindi (*) non può essere vera asintoticamente.

Proviamo il metodo di sostituzione. Per formulare un'ipotesi sulle forme della soluzione

Se non vale MT, dobbiamo usare la sostituzione (cioè, "immagino" come potrebbe svilupparsi la ricorrenza. Per fare ciò, prendo di solito il termine noto e creo un albero alla "merge sort" per capire che succede).



Fondamentalmente, si divide "n" sempre a metà fino alle foglie. Tutto questo in scala logaritmica data "n" costante. Quindi "n" è una costante fuori dalla serie e quello che cambia è il fattore logaritmico. Esso va da 0 (foglia) fino a $\log(n)-1$, (altezza albero)

$$m \sum_{j=0}^{\log m - 1} \log \frac{m}{2^j} + cm$$

Rapporto tra logaritmi = Sottrazione tra logaritmi (proprietà fond. logaritmi)

(sommando base 2)

$$= m \sum_{j=0}^{\log m - 1} (\log m - \log 2^j) + cm$$

Vede $\log(2^j)$ come $\log_J(2^j)$, quindi diventa $\log(2)$, quindi $\log(n)$

$$= m \log m \log m - m \sum_{j=0}^{\log m - 1} j + cm$$

$$= m (\log m)^2 - m \frac{\log m (\log m - 1)}{2} + cm$$

$$\Rightarrow \Theta(m (\log m)^2)$$

Fatta questa assunzione, dobbiamo dimostrarla (essendo theta, da sopra e da sotto (O grande ed omega), applicando la definizione).

Dimostriamo

$$1) T(m) = O(m (\log m)^2)$$

$$2) T(m) = \Omega(m (\log m)^2)$$

1) Dobbiamo verificare l'esistenza di $c > 0$ t.c.

$$T(m) \leq c m (\log m)^2 \quad \text{asintoticamente}$$

procediamo induttivamente (trovando il caso base)

$$T(m) = 2 T\left(\frac{m}{2}\right) + m \log m$$

$$\text{IP ind.} \rightarrow \leq 2 c \frac{m}{2} \left(\log \frac{m}{2}\right)^2 + m \log m$$

$$= cm (\log m - \log 2)^2 + m \log m$$

Di nuovo rapporto logaritmi come sottrazione;
nella riga dopo, semplice quadrato di binomio del logaritmo

$$= cm ((\log m)^2 - 2 \log 2 \log m + (\log 2)^2) + m \log m$$

$$= cm (\log m)^2 - m (\log m (2c \log 2 - 1) - c (\log 2)^2) \leq cm (\log m)^2$$

Raccoglimento a fattori comune rispetto ad "n" e poi risolve tutto il calcolo

osservando i termini, se deve essere minore della parte a destra, allora per c maggiore della sua costante, questo vale sicuramente.

(essendo $\log(2)$ più grande, per $n \rightarrow \infty$ di $cn(\log(n))^2$)

chiaramente vera se $c > \frac{1}{2 \log 2}$ e m "abbastanza grande"

2) Dobbiamo mostrare che esiste $d > 0$ tale

$$T(m) \geq dm (\log m)^2 \quad \text{asimbotticamente}$$

induttivamente:

$$T(m) = 2T\left(\frac{m}{2}\right) + m \log m$$

$$\text{IP ind} \rightarrow \geq 2d \frac{m}{2} (\log \frac{m}{2})^2 + m \log m$$

come prima

$$= dm (\log m)^2 - m (\log m (2c - 1) - c)$$

$$\geq dm (\log m)^2$$

di nuovo, guardando i termini la risolve:
risolvendo per c, si ottiene un mezzo.

Dato che deve essere maggiore della parte a destra,
allora essendo il $\log(n)$ più grande per $n \rightarrow \infty$, vale per $c < \frac{1}{2}$

stavo per se il termine è negativo, quindi $c < \frac{1}{2}$

Esercizio

Risolvere la ricorrenza

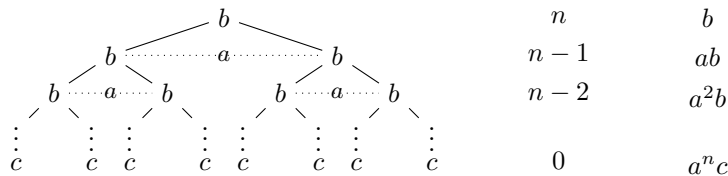
$$T(n) = aT(n-1) + b$$

Dovendo distinguere vari casi a seconda del valore di a è opportuno dare una soluzione esatta, specificando un caso base

$$T(n) = \begin{cases} c & \text{se } n = 0 \\ aT(n-1) + b & \text{se } n > 0 \end{cases}$$

Come detto sopra, qui non abbiamo molte informazioni ed espando seguendo l'albero delle ricorrenze rispetto al termine noto, contando sul fatto che "a" segue per potenza "b", (quindi, a^2 , a^3 , .. $a^{(n-1)}$).

Per ipotizzare una soluzione, da provare poi per sostituzione, vediamo l'albero delle ricorrenze



Questo ci conduce dunque all'ipotesi

$$T(n) = b + ab + a^2b + \dots + a^{n-1}b + a^n c = b \sum_{j=0}^{n-1} a^j + a^n c$$

che può essere verificata per induzione

- ($n = 0$) In questo caso

$$T(0) = (\sum_{j=0}^{-1})b + a^0 c = c$$

- ($n > 0$) In questo caso

$$\begin{aligned} T(n) &= aT(n-1) + b = && \text{La serie è da 0 ad } n-1, \text{ ma siccome mette in mezzo anche } (a^{(n)}-c) \text{ diventa per } j=0 \text{ ad } n-2 \\ &= a\left(\sum_{j=0}^{n-2} a^j\right)b + a^{n-1}c + b && [\text{ipotesi induttiva}] \\ &= \left(\sum_{j=0}^{n-2} a^{j+1}\right)b + a^n c + b && \text{Dato che } (n-1) \text{ è } (j+1), \text{ per la proprietà delle serie lo porta dentro, come } a^{(j+1)}. \\ &= \left(\sum_{i=1}^{n-1} a^i\right)b + a^n c + b && \text{Cambia poi indice, mettendo un "i" che va da 1 ad } n-1, \text{ per ipotesi induttiva (i = j+1).} \\ &= \left(\sum_{i=1}^{n-1} a^i\right)b + b + a^n c \\ &= \left(\sum_{i=0}^{n-1} a^i\right)b + a^n c && \text{Il "b" è trascurabile e gli ultimi passaggi sono degli aggiustamenti di termini.} \end{aligned}$$

come desiderato.

Per valutare l'andamento asintotico distinguiamo vari casi a seconda del valore di a :

- ($a = 1$) Vale Qua easy --> mette dentro $a=1$ e ricava $(bn+c)$

$$T(n) = (\sum_{j=0}^{n-1} 1^j)b + 1^n c = (\sum_{j=0}^{n-1} 1)b + c = bn + c$$

pertanto $T(n) = \Theta(n)$.

- ($a < 1$) Vale

Sapendo che "a" è minore di 1, significa che $a^n c \leq c$. Il termine $(1/(1-a))$ è il termine generale della serie geometrica (quindi, generica serie con a^i e che va da $i=0$ a n)

$$T(n) = \left(\sum_{j=0}^{n-1} a^j \right) b + \underbrace{a^n c}_{\leq c} \leq \left(\sum_{j=0}^{\infty} a^j \right) b + c = \frac{1}{1-a} b + c$$

Come detto, la serie geometrica:
<https://www.matematicaonvideo.it/wp-content/uploads/Serie-geometrica.png>

pertanto $T(n) = \Theta(1)$.

- ($a > 1$) Vale

$$T(n) = \left(\sum_{j=0}^{n-1} a^j \right) b + a^n c = \frac{a^n - 1}{a - 1} b + a^n c$$

Qua è più sottile; per $a > 1$, la serie va ad infinito come si vede dal link. Per infinito, intendiamo il limite superiore, che qui è $(n-1)$. Quindi, vuol dire scrivere la serie per $a^{(n-1)}$, da cui il risultato.

pertanto $T(n) = \Theta(a^n)$.

Per l'ultimo caso, supponiamo di voler mostrare direttamente il limite asintotico, ovvero

1. $T(n) = \Omega(a^n)$ Si dimostra applicando la definizione $\rightarrow T(n) \geq c a^n$
2. $T(n) = O(a^n)$

Mentre il punto (1) è banale, per il punto (2) la prova per induzione ovvia fallisce. Infatti si vuole che per una costante $d > 0$, asintoticamente valga

$$T(n) \leq d a^n$$

La prova induttiva procede come segue:

$$T(n) = aT(n-1) + b \leq a d a^{n-1} + b = d a^n + b \not\leq d a^n \quad \text{Come si vede, non vale}$$

Dobbiamo modificare l'ipotesi induttiva e lo facciamo portando in sottrazione qualcosa che poi nel passo induttivo sarà moltiplicato per la costante (Rafforzamento dell'ipotesi induttiva, termine di prima meno una costante che gli sarà moltiplicata)

$$T(n) \leq d(a^n - 1)$$

Ora la prova induttiva funziona:

$$\begin{aligned} T(n) &= aT(n-1) + b = \\ &\leq a d(a^{n-1} - 1) + b \quad [\text{ipotesi induttiva}] \\ &= d(a^n - a) + b \\ &= d(a^n - 1) - d(a - 1) + b \quad [\text{uso } a^n - a = a^n - 1 - (a - 1)] \\ &\leq d(a^n - 1) \end{aligned}$$

per $d \geq \frac{b}{a-1}$. Pertanto $T(n) = O(a^n - 1) = O(a^n)$. Qui, il calcolo funziona.

Mergesort con allocazione singola

Realizzare una versione del mergesort nella quale l'array di supporto è allocato staticamente (quindi la procedura sarà del tipo `MergeSort(A,B, ...)` dove `A` è l'array di input, mentre `B` è un array della stessa dimensione utilizzato per la memorizzazione temporanea

- *Versione 1:* l'array `B` è utilizzato per memorizzare temporaneamente le parti di array già ordinate delle quali si deve effettuare il merge;
- *Versione 2:* la procedura `Merge` fa alternativamente il merge dei sottoarray già ordinati da `A` verso `B` e vice versa, di modo da evitare la fase di copia dei sottoarray dei quali fare il merge.

Soluzione:

Versione 1. In questo caso la soluzione è molto semplice. È sufficiente allocare l'array `B` staticamente. Quindi `B` diventa un argomento della procedura `MergeSort` ricorsiva. Viene usato in `Merge` per contenere temporaneamente i due sottoarray da fondere.

chiamata base:

```
MergeSort (A)
    n = A.length
    allocate B[1..n]           // alloca l'array B di supporto
    MergeSortRic (A, B, 1, n)  Lo passa alla procedura ricorsiva
                                per calcolare ricorsivamente sulle due metà di array
```

procedura ricorsiva

```
MergeSortRic (A, B, p, r)
    if p < r
        q = (p+r)/2
        MergeSortRic (A, B, p, q)
        MergeSortRic (A, B, q+1, r)
        Merge (A, B, p, q, r)
```

Merge: usa l'array di supporto per memorizzare i due sottoarray da fondere

```
Merge (A, B, p, q, r)
    for i = p to q
        B[i] = A[i]
    for j = q+1 to r
        B[j] = A[j]
                                Alloco sui due indici per l'array di supporto B
                                la prima e la seconda metà

    i=p                         Inizializzo gli indici della prima e seconda metà
    j=q+1
    for k = p to r
        if (i <= q) and ((j > r) or (B[i] <= B[j])):
            A[k] = B[i]
            i ++
        else:
            A[k] = B[j]
            j ++
    }
```

1) se devo ancora ordinare la prima metà
oppure
2) se devo ordinare la seconda metà
&
se la prima metà è minore della seconda (quindi, è
correttamente ordinato)

Allora:
1) l'array `A` prende come valore quello più piccolo della prima
metà di array di `B` (salvata prima)
2) l'array `A` prende come valore quello più piccolo della
seconda metà di array di `B` (salvata prima)

Versione 2. Si definisce una procedura di mergesort `MergeSortAlt(A,B,p,r,destA)` con un parametro addizionale, `destA`, un flag booleano, che indica se la destinazione dell'ordinamento è A (quando vero) oppure B (quando falso). Occorre dunque avere l'accortezza di negare il flag ad ogni passaggio ricorsivo. Inoltre, nel caso base, se la destinazione dell'ordinamento è A non c'è niente da fare, se invece è B, l'unico elemento di cui si compone il sottoarray va copiato in B. Il resto è immutato rispetto a prima.

chiamata base:

```
MergeSort (A)
    n = A.length
    allocate B[1..n]          // alloca l'array B di supporto
    MergeSortRic (A, B, 1, n, true)
```

```
# MergeSortAlt(A, B, p, r, destA)
# se destA=true -> ordina A[p,r] con risultato in A[p,r]
# altrimenti    -> ordina A[p,r] con risultato in B[p,r]
```

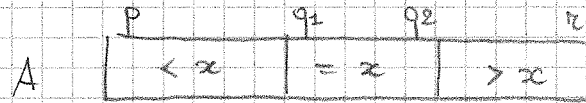
```
MergeSortAlt (A, B, p, r, destA)
    if p < r      se la prima metà è minore della seconda
        q = (p+r)/2      salvo l'indice medio
        MergeSortAlt (A, B, p, q, not destA)  esamino la prima e la seconda metà di array
        MergeSortAlt (A, B, q+1, r, not destA)
        if destA
            Merge (B, A, p, q, r)  Grazie al flag, capisco se eseguire il merge:
                                   - sulla prima metà (se destA è vero)
                                   - sulla seconda metà (se destA è falso)
        else
            Merge (A, B, p, q, r)
    else
        if not destA
            B[p]=A[p]  Come scritto sopra, questo è il caso base:
                      se la destinazione dell'ordinamento è A non c'è niente
                      da fare, se invece è B, l'unico elemento di cui si compone il sottoarray va copiato in B.
```

Merge: fa il merge di S(ource) in T(arget)

```
Merge (S, T, p, q, r):      Funzionamento uguale a quanto descritto
                             a pagina precedente
    i=p
    j=q+1
    for k = p to r
        if (i <= q) and (not (j <= r) or (S[i] <= S[j]))
            T[k] = S[i]
            i ++
        else:
            T[k] = S[j]
            j ++
    }
```

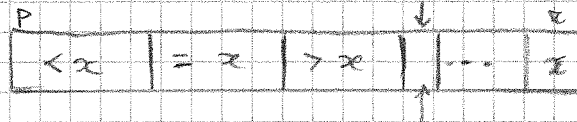
Tri Partition (A, p, r)

ritorno q_1 e q_2 che delimitano la zona di elementi uguali al pivot



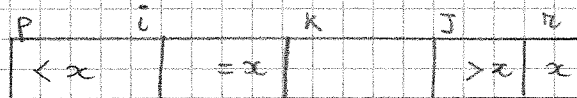
come procedere?

Nota = se mantengo tre zone contigue ho problemi ad aggiornarle



$x < x$ dovrei fare due scambi...

meglio utilizzare il seguente schema



$$A[p, i] < x$$

$$A[i+1, k-1] = x$$

$$A[j, r-1] > x$$

Usiamo tre partizioni per migliorare Quick Sort (che, come sappiamo, si basa sull'utilizzo di un pivot). In particolare, ci basiamo su tre partizioni: con gli indici iniziali tra la fine della prima partizione e la fine della seconda partizione (per minimizzare gli scambi tra le partizioni).

Tripartition (A, p, q)

$x = A[k]$

// pivot

$i = p-1$

Inizializzo gli indici come visto sopra
e il pivot

$k = p$

$j = r$

while $k < j$

if $A[k] < x$

$i++$

$A[i] \leftrightarrow A[k]$

$k++$

else if $A[k] > x$

$j--$

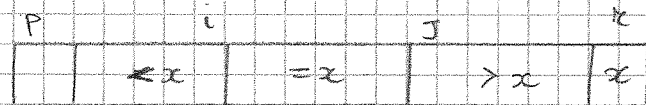
$A[k] \leftrightarrow A[j]$

// non incremento k!

else

$k++$

$A[j] \leftrightarrow A[i]$



(per ordinare l'array cerco di scambiare la metà che contiene x con la metà maggiore ed "accumulare" x al centro dell'array.

return (i+1, j)

Restituiamo da "i+1" a "j" per essere sicuri di beccare la metà giusta
rispetto al pivot (la parte uguale, in particolare)

QuickSort (A, p, r)

if $r - p > 1$

$q_1, q_2 = \text{TriPartition} (A, p, r)$

QuickSort (A, p, q_1-1)

QuickSort (A, q_2+1 , r)

Select

Realizzare un algoritmo **Select(A,k)** che dato un array non ordinato di interi $A[1,n]$ ed un indice $1 \leq k \leq n$, restituisce l'elemento dell'array che sarebbe in posizione k nell'array ordinato (o meglio, uno degli elementi che potrebbe essere in posizione k , dato che se ci sono ripetizioni, l'elemento non è univocamente determinato). Ad esempio, se $A = [1, 4, 2, 8, 5]$ si vuole **Select(A,3)** = 4 mentre **Select(A,5)** = 8.

Proporre soluzioni di con complessità:

1. $O(n \log n)$
2. $O(n + k \log n)$
3. $O(n \log k)$

Soluzione

1. Si può ottenere una soluzione di tempo $O(n \log n)$ banalmente, ordinando l'array e quindi ritornando l'elemento di posizione k . Ovvero

```
Select(A, k)
  n = A.length
  Mergesort(A,1,n)
  return A[k]
```

Essendo ($n \log n$), si usa banalmente un algoritmo di ordinamento che ha quella complessità (merge sort) e ritorna l'elemento

2. Per ottenere una soluzione di costo $O(n + k \log n)$ posso trasformare A in un min-heap ed estrarre k volte l'elemento più piccolo, ovvero:

```
Select(A, k)
  BuildMinHeap(A)
  for i=1 to k-1
    ExtractMin(A)
  return A[1]
```

Come descritto sotto, costruire un min heap ha costo lineare e poi l'estrazione del minimo sfrutta l'altezza dell'albero/heap, con complessità logaritmica, ritornando l'elemento utile, qualora selezionato, come primo elemento.

L'invariante di ciclo, utile a dimostrare la correttezza del procedimento, è che $A[1, n-i+1]$ contiene gli $n - i + 1$ elementi più grandi di A .

Il costo di **BuildMinHeap(A)** è $\Theta(n)$, mentre **ExtractMin(A)** costa $O(\log n)$ e viene eseguita k volte, quindi, in totale $O(n + k \log n)$.

3. Per ottenere una soluzione con complessità $O(n \log k)$ si può procedere nel modo seguente:
 - Si costruisce un max-heap che contenga i primi k elementi di A . Per questo si utilizza **BuildMaxHeap(A,k)** dove il parametro addizionale k indica il numero di elementi di A che vanno inclusi nel max-heap (si opera come se la dimensione di A fosse k).
 - Ognuno dei rimanenti $n - k$ elementi di A è confrontato con il massimo dei k elementi nel max-heap corrente, e se più piccolo, lo sostituisce.
 - Alla fine, il max-heap conterrà i k elementi più piccoli di A , il cui massimo $A[1]$ è l'elemento che si troverebbe in posizione k nell'array ordinato, ovvero l'elemento cercato.

Si ottiene dunque

Quindi:

- si costruisce con costo lineare un max heap
- si rende l'albero un max heap confrontando l'elemento i -esimo con il primo (massimo), con complessità $(n \log(k))$, dato che tutto ciò viene fatto in base all'indice k passato come parametro.
Grazie allo scambio di indice e grazie al fatto che i incrementa e k resta fisso, allora è $(n-k) \log(k)$.

```
Select(A, k)
    BuildMaxHeap(A, k)      // costo  $\Theta(k)$ 

    for i=k+1 to n          // costo  $O((n-k) \log k)$  quindi  $O(n \log k)$ 
        if A[i] < A[1]
            A[i] <-> A[1]    // scambia A[i] e A[1]
            MaxHeapify(A, 1)
    return A[k]
```

L'invariante di ciclo è

- $A[1, k]$ max-heap
- $A[k+1, i-1] \geq A[1, k]$, ovvero il frammento esplorato della parte rimanente di array, contiene elementi tutti tutti maggiori o uguali a quelli in $A[1, k]$.

Quindi, quando il ciclo termina $A[k+1, n] \geq A[1, k]$, ovvero $A[1, k]$ contiene i k elementi più piccoli dell'array. Pertanto il massimo, in $A[1]$, è l'elemento in posizione k nell'array ordinato, e quindi è l'elemento cercato.

Il costo complessivo è $O(k + n \log k) = O(n \log k)$, dato che il termine $n \log k$ domina k .

4. Selezione in tempo medio lineare: QuickSelect

L'idea è qui quella di sfruttare la procedura di partizionamento vista per il Quicksort. Se la posizione q del pivot è esattamente l'indice k cercato abbiamo concluso. Altrimenti possiamo ricorrere su una delle due parti a seconda che k sia minore o maggiore di q . Esplicitamente:

```
QuickSelect(A, p, r, k)
    q = Partition(A, p, r)      // costo  $an + b$ 

    if (q == k)
        return A[q]
    else if (k < q)
        return QuickSelect(A, p, q-1, k)
    else
        return QuickSelect(A, q+1, r, k)
```

Fondamentalmente, se cadiamo nel caso "fortunato" di Quick Sort, la ricorrenza è $T(n) = O(n) + T(n/2)$ che nel caso migliore (fortunato appunto) è $O(n)$ a, alla peggio, è $O(n^2)$.

L'algoritmo è simile a QuickSort. La differenza è che, invece di ricorrere per entrambe le parti (dopo aver trovato il pivot), ricorre solo per la parte che contiene il k -esimo elemento più piccolo. La logica è semplice: se l'indice dell'elemento partizionato è superiore a k , allora si ricorre per la parte sinistra. Se l'indice è uguale a k , abbiamo trovato il k -esimo elemento più piccolo e torniamo indietro. Se l'indice è inferiore a k , ricorriamo alla parte destra. Questo riduce la complessità prevista da $O(n \log n)$ a $O(n)$, con un caso peggiore di $O(n^2)$.

È facile vedere che, come nel Quicksort, nel caso peggiore, ovvero quando si ottenga sistematicamente una delle due partizioni vuote, si ottiene la ricorrenza

$$T(n) = T(n-1) + an + b$$

con soluzione $T(n) = \Theta(n^2)$

Quando invece sistematicamente la partizione sia perfettamente bilanciata, si ottiene la ricorrenza

$$T(n) = T(n/2) + an + b$$

che, utilizzando il Master Theorem, ha come soluzione $T(n) = \Theta(n)$.

Si può verificare che anche il caso medio porta a complessità lineare. In un approccio semplificato, possiamo assumere che con uguale probabilità, si

termini o si ricorra su di una partizione di dimensione $1, 2, \dots, n-1$. Il valore medio della complessità può dunque essere espresso dalla ricorrenza:

$$T(n) = \frac{1}{n} \left(\sum_{i=0}^{n-1} T(i) \right) + an + b$$

E si può verificare per sostituzione che $T(n) = O(n)$, ovvero che $T(n) \leq cn$ asintoticamente per un'opportuna costante $c > 0$.

Infatti si ha

$$\begin{aligned} T(n) &= \frac{1}{n} \left(\sum_{i=0}^{n-1} T(i) \right) + an + b \\ &\leq \frac{1}{n} \left(\sum_{i=0}^{n-1} ci \right) + an + b \quad [\text{ipotesi induttiva}] \\ &= c \frac{1}{n} \frac{n(n-1)}{2} + an + b \\ &= c \frac{(n-1)}{2} + an + b \\ &= \frac{cn}{2} - \frac{c}{2} + an + b \\ &= \left(\frac{c}{2} + a \right) n - \frac{c}{2} + b \\ &\leq cn \end{aligned}$$

dove l'ultima disuguaglianza vale, asintoticamente, se $\frac{c}{2} + a < c$ e quindi per $c > 2a$. Questo conclude la prova.

Così come nel caso del QuickSort, il caso medio può essere indotto anche quando l'input non sia uniformemente distribuito, con una scelta randomizzata del pivot. La presenza di duplicati richiederà di utilizzare una tripartizione.

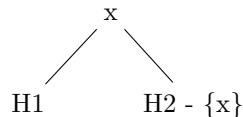
5. Selezione in tempo lineare

Si veda il libro di testo. [Randomized Partition](#), permette di utilizzare la versione randomizzata in modo da evitare che un particolare input sia soggetto al worst case ($O(n^2)$), partizionando in maniera bilanciata (magari, all'elemento più grande presente).

Fusione di Max-Heap

Realizzare una procedura `Fusion(H1,H2)` per fondere due heap `H1` e `H2`, il primo **completo**, il secondo **quasi completo**, della **stessa altezza**, che soddisfano la proprietà di max-heap, ottenendo un unico max-heap. Cercare una soluzione di costo $O(\log n)$ dove n è il numero di elementi totali dei due alberi.

Soluzione Assumiamo che `H1` e `H2` siano dati come strutture linked. Allora si può prendere l'ultima foglia x di `H2`, e usarla come radice comune dei due heap, che ha `H1` come figlio sinistro e quel che resta di `H2` come figlio destro.



A questo punto, la proprietà di max-heap è potenzialmente violata solo dalla radice e quindi può essere ripristinata usando `MaxHeapify`.

Fondiamo due max-heap considerando che il primo è un albero completo e il secondo è quasi completo (quindi, completo a tutti i livelli, tranne l'ultimo dove normalmente mancano a dx.

Spiegando l'algoritmo:
- cerco il parent nell'albero quasi completo
- se il parent è nil, H2 contiene solo la radice e quindi la foglia "x" sarà la radice di H2
- altrimenti, ha delle foglie e
- se la foglia è figlio sinistro, allora salviamo in "x" il parent del nodo sinistro
- se la foglia è figlio destro, allora salviamo in "x" il parent del nodo destro.
Si noti che oltre a salvare il parent, salviamo anche "p"; questo succede a causa del fatto che l'albero sia quasi completo (quindi, se non avessi altre foglie da salvare, le sto già salvando tutte).

```
fusion(H1, H2) // assume H1, H2 non vuoti
    p = numToParent(H2, H2.size) // determina il parent dell'ultima foglia di H2
    if p == nil // la foglia e' anche la radice
        // H2 contiene solo la radice
        x = H2.root
        H2.root = nil
    else
        if (H.size mod 2 == 0) // se il bit meno significativo e' 0
            x = p.left // la foglia e' figlio sinistro
            p.left = nil
        else
            x = p.right = p // altrimenti e' figlio destro
            p.right = nil
    end
    H2.size-- // ricorda che e' stata rimossa una foglia

    // crea un nuovo heap con radice x
    H.root = x
    H.size = H1.size + H2.size + 1
    x.left = T1.root H1.root
    x.right = T2.root H2.root

    MaxHeapify(x) // ripristina la proprieta' di max-heap

    return H
```

Correttezza:
- H1/H2 non vuoti
- prendiamo con la procedura `numToParent` il parent di H2
- se il parent è nil, formiamo un max heap che ha per radice l'unica foglia di H2
- se il parent non è nil, ci prendiamo i suoi figli `sx` e `dx`
In entrambi questi casi, ristabiliamo le proprietà di heap creando la radice, la `size`, i figli `sx` e `dx` e usiamo `MaxHeapify`.
Costo totale $O(\log(n))$.

Una volta fatto tutto questo, ogni volta togliamo una foglia da `H2` e quindi diminuiamo la sua `size` di 1.
Successivamente, si salva il nodo "x" come radice, la `size` dello heap è data da `size albero sx + size albero dx + 1` (per la radice) e i figli `sx` e `dx` sono rispettivamente le radici dei due heap, essendo max heap.

La complessità è data dalla somma della complessità di `numToParent(H, H.size)` e di `maxHeapify(x)` (il resto ha costo costante). Risulta dunque essere $O(\log n)$.

Si noti che se invece `H1` e `H2` fossero stati dati come array, la creazione di un nuovo array che contenga gli elementi di `H1` e `H2` ha almeno costo lineare. Si può provare che il costo resta almeno lineare anche se i due max-heap sono allocati in spazi consecutivi. L'idea dell'argomento è la seguente. Per fissare le idee, supponiamo che `H1` e `H2` siano memorizzati in un array `A[1...p...n]` con `A[1...p]` che contiene `H1` e `A[p+1...n]` che contiene `H2`. Le osservazioni fondamentali sono che:

- gli elementi di H_2 sono le foglie di A visto come heap, quindi hanno elementi di H_1 come antenati;
- se si assume che ogni elemento di H_1 sia strettamente minore di ogni elemento di H_2 , affinché un elemento di H_2 non venga spostato nella costruzione del max-heap complessivo, occorre che tutti i suoi antenati, che erano elementi di H_1 , vengano spostati;
- l'osservazione precedente implica che almeno metà degli elementi di H_1 devono essere spostati, cosa che richiede un numero di operazioni lineari in n .

Si osservi dunque che quando si utilizzi la rappresentazione dei max-heap come array non si può utilizzare la proprietà di max-heap di H_1 e H_2 in modo significativo, ovvero il costo resta analogo a quello che avrei semplicemente applicando `BuildMaxHeap` all'intero array $A[1 \dots n]$.

Ecco la funzione `numToParent(H,k)` che dato uno heap H e un numero di nodo k , restituisce il padre del nodo di posizione k . Per realizzarla si assume di avere una funzione `getBitVector(k)` che dato un numero n restituisce un array che contiene la rappresentazione binaria di n . Il costo di tale funzione sarà $O(\log k)$.

```
numToParent(H, k) // assume k >= 1
if k == 1
  return nil
else
  B = getBitVector(k) // trova i bit della cella di array che ci interessa
  i = 1 // trova la cifra piu' significativa a 1
  while (B[i]==0)
    i++
  x = H.root
  for j=i+1 to B.length-1 // Esaminiamo tutta la sequenza binaria, sapendo che
    if B[j] == 0           // Se un bit di B è pari a 0, cerchiamo a sx
      x = x.left
    else
      x = x.right         // Se un bit di B è pari a 1, cerchiamo a dx
  return x
```

② Dato un array $A[1, n]$ di interi una inversione è una coppia di indici i, j $i < j$ $A[i] > A[j]$



Nel contesto di questo esercizio, si assume per inversione il numero di elementi di un array tali che "un elemento in posizione minore sia più grande di un elemento in posizione maggiore dell'array". Si chiede di contare quanti siano questi elementi.

L'idea è di avere un algoritmo basato su Mergesort tale da avere due partizioni su cui cercare ricorsivamente le inversioni ed avere la metà che già rispetta questa proprietà (la metà fa da "spartiacque" ed accumuliamo cercando ricorsivamente a dx e a sx tutte le inversioni, in modo logaritmico, ordinando "in place" l'array).

Dare un algoritmo divide et impera

$Inv(A)$

che restituisce il numero di Inversioni in A

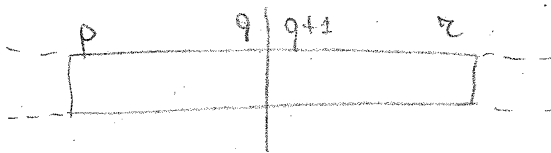
OSSERVAZIONE



una inversione in $A[p, r]$ è

$$p \leq i < j \leq r \quad A[i] > A[j]$$

se dividiamo $A[p, r]$



potranno avere

- $p \leq i < j \leq q \rightarrow$ inversione in $A[p, q]$
- $q+1 \leq i < j \leq r \rightarrow$ " " " $A[q+1, r]$
- $p \leq i \leq q, q+1 \leq j \leq r \rightarrow$ inversione "a cavallo"

quindi

$$\# Inv(A[p, r]) = \# Inv(A[p, q]) + \# Inv(A[q+1, r]) +$$

$$+ \# \{ (i, j) \mid i \in [p, q] \quad A[i] > A[j] \quad j \in [q+1, r] \}$$

Questo porta all'algoritmo

$Inv(A, p, r)$

$\sim$ conta le inversioni e ordina!

if $p < r$

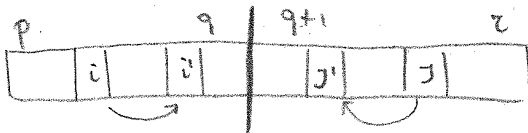
$$q = \frac{p+r}{2}$$

return $Inv(A, p, q) +$
 $Inv(A, q+1, r) +$

$InvMerge(A, p, q, r)$ $\sim$ calcolo (*)

Per contare le inversioni, ordinando, somma le chiamate ricorsive sulle due metà di array e poi esegue il merge. Il fatto di usare merge sort garantisce di rispettare ancora le inversioni, tale che un indice "i" maggiore di un indice "j" sia ancora così su un array ordinato (perché fatto in modo crescente).

Come realizzare InvMerge? 3 osservazioni



a) dopo essere stati ordinati $A[p, q]$ e $A[q+1, r]$ contemporo gli stessi elementi di prima (ordinamento in loco)

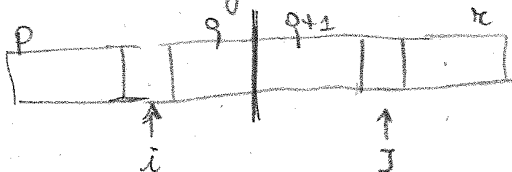
↓

(b) se (i, j) con $i \in [p, q]$ era inversione prima dell'ordinamento
 $j \in [q+1, r]$

delle i', j' le nuove posizioni a cui sono stati spostati gli elementi (i', j') è ancora inversione (e vice versa!)

$\sim$ la quantità (*) è invariata!

(c) nei sottoarray ordinati



se (i, j) è inversione (ovvero $A[i] > A[j]$)

allora (i', j) è inversione per ogni $i \leq i' \leq q$

$\sim q - i + 1$ inversioni

InvMerge (A, p, q, r)

$$m_1 = q - p + 1$$

$$m_2 = r - q$$

$$L[1, m_1] \leftarrow A[p, q]$$

$$R[1, m_2] \leftarrow A[q+1, r]$$

$$L[m_1+1] = R[m_2+1] = +\infty$$

$$i = j = 1$$

$$\text{Inv} = 0$$

for k = p to r

if $L[i] \leq R[j]$

$$A[k] = L[i]$$

i++

else

$$A[k] = R[j]$$

j++

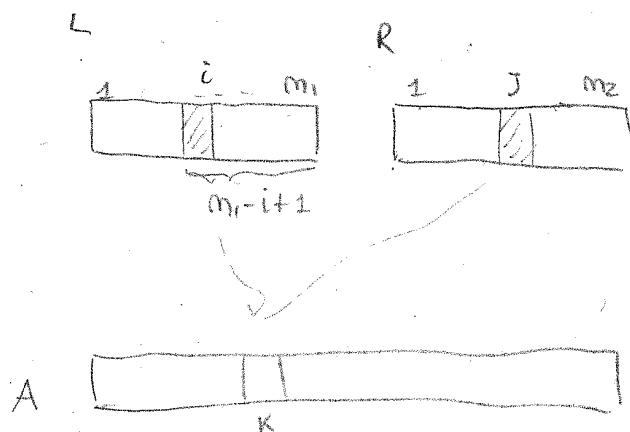
$$\text{Inv} = \text{Inv} + m_1 - i + 1$$

return Inv

L'algoritmo a fianco è praticamente identico

a MergeSort, quindi:

- inizializza gli indici iniziali e finali e le due partizioni di dx e di sx
- esegue gli ordinamenti delle due parti sx e dx
- il conteggio delle inversioni avviene solamente nell'ordinamento della parte dx, perché si assume che tutto ciò che c'è a sx sia già ordinato in modo crescente e quindi semplicemente sommo al numero di inversioni tutti gli elementi già scorsi a sx (togliendo però l'indice attuale, perché ci si sta ancora lavorando)
- ciò detto, ritorno il numero di inversioni



Correttezza:

- array vuoto, non esistono inversioni (inv=0)
 - array non vuoto e
 - lo divide in due partizioni e cerca il numero di inversioni
 - non essendo vuoto, finché ci sono elementi,
 - se la partizione di sx è da ordinare, scambio e ordino gli elementi in ordine crescente
 - quando questa condizione è falsa ordino a sx e l'ordinamento crescente mi garantisce di avere già un numero di inversioni pari agli elementi a sx
- Il numero di inversioni rispetta la proprietà di ordinamento e anche di crescita dell'array, quindi viene restituita.

Linked Max-Heaps

Realizzare la struttura dati *Max-Heap* mediante strutture linked.

Soluzione. Un generico nodo x conterrà i campi:

- $x.left$: figlio sinistro
- $x.right$: figlio destro
- $x.p$: genitore

Questo esercizio richiede una piccola modifica alla struttura max heap realizzandoli come fossero strutture linkate; questo significa che ogni nodo avrà le proprietà di conoscere tutto il percorso binario che permette di arrivarvi e, in ogni momento, sa chi è il suo parent/genitore. Questa porta ad una piccola modifica nelle funzioni (che evidenzio), al di là della funzione apposita che trova il parent del nodo.

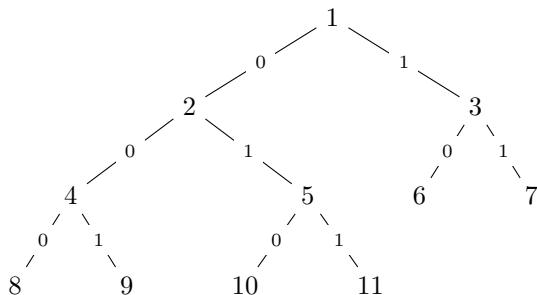
La radice avrà $x.p = nil$, mentre le foglie $x.left = x.right = nil$. Quindi un max-heap è un albero H , con campi

- $H.root$
- $H.size$.

Per quanto riguarda le operazioni:

- $Max(H)$, $MaxHeapify(H, x)$ e $MaxHeapifyUp(H, x)$ restano sostanzialmente inalterate.
- $Insert(H, x)$

L'elemento va inserito mantenendo la proprietà di albero quasi completo, quindi a destra dell'ultima foglia, se l'ultimo livello non è completo, o come prima foglia di un nuovo livello, se l'ultimo livello è completo. Occorre dunque capire qual è l'ultima foglia dell'ultimo livello. Per questo è opportuno osservare quanto segue. Dato uno heap (nella figura, il numero indica la posizione del nodo, non la chiave), ed etichettando gli archi che connettono al figlio sinistro e destro con 0 e 1 rispettivamente otteniamo:



allora, se esprimiamo il numero n associato ad un nodo in binario questo risulta del tipo

$$1 \underbrace{xxxxxxxxxxxxxxxx}_{\text{cammino radice-nodo}}$$

dove $xxxxxxxxxxxxxxxx$ è la sequenza di etichette nel cammino dalla radice al nodo.

Dunque la posizione nella quale inserire un nuovo nodo, se n è il numero attuale di nodi, si otterrà, esprimendo in binario $n + 1 = (1xxxxxxxxxxxxxxxx)_2$ ed usando $xxxxxxxxxxxxxxxx$ per scendere nell'albero: 0 scendi a sinistra, 1 scendi a destra.

Più precisamente, possiamo realizzare una funzione $numToParent(H, k)$ che dato uno heap H e un numero di nodo k , restituisce il padre del nodo di posizione k . Per realizzarla si assume di avere una funzione $getBitVector(k)$ che dato un numero n restituisce un bitvector che contiene la rappresentazione binaria di n . Il costo di tale funzione sarà $O(\log k)$.

```

numToParent(H, k)                                // assume  $k \geq 1$ 
    if k == 1
        return nil
    else
        B = getBitVector(k)

        i = 1                                    // trova la cifra piu' significativa a 1
        while (B[i]==0)
            i++

        x = H.root
        for j=i+1 to B.length-1
            if B[j] == 0
                x = x.left
            else
                x = x.right

        return x

```

Questa funzione:

- determina il percorso binario (un array lineare di 0/1 di tutte le foglie) dello heap
- prende il bit attuale più significativo (più a sx)
- ci salviamo la radice
- se il bit è 0, andiamo a sx
- se il bit è 1, andiamo a dx
- ritorniamo il parent del nodo in posizione k

A questo punto è facile realizzare la procedura di inserimento, che crea il nuovo nodo, noto il padre. Il nodo sarà figlio sinistro o figlio destro a seconda che il bit meno significativo della posizione sia 0 o 1, rispettivamente.

```

insert(H, x)
    x.left = x.right = nil                        // il nodo sara' una foglia

    H.size++                                     // incrementa la size
                                                // questa e' la posizione del nuovo nodo

    x.p = numToParent(H, H.size)                // Determina e assegna il parent

    if x.p <> nil
        if (H.size mod 2 == 0)                  // se il bit meno significativo e' 0
            x.p.left = x                        // il nodo e' figlio sinistro
        else
            x.p.right = x                       // altrimenti e' figlio destro

    maxHeapifyUp(x)                             // ripristina la proprieta' di max-heap

```

Deve solo sapere chi è il suo parent (modifica evidenziata); per il resto, pone le foglie = nil, aumenta la size per inserire il nuovo nodo e cerca di capire se andrà a sx o a dx, ripristinando le proprietà di max-heap.

La complessità è data dalla somma della complessità di `numToParent(H, H.size)` e di `maxHeapifyUp(x)` (il resto ha costo costante). Risulta dunque essere $O(\log n)$.

La funzione `ExtractMax(H)` è analoga alla `Insert`. Il massimo si trova nella radice. Quindi si individua l'ultima foglia e la si sostituisce alla radice.

```

extractMax(H)
    x = numToParent(H, H.size)    // determina il parent dell'ultima foglia

    if x == nil                    // l'albero contiene la sola radice
        H.root = nil
    else

        y.p = nil                 // si prepara a inserire y come root
        y.left = H.root.left
        y.right = H.root.right

        max = H.root              // ricorda il massimo, contenuto nella radice
        H.root = y

        maxHeapify(y)             // ripristina la proprieta' di max-heap

    H.size--                      // in ogni caso si riduce size

```

Anche in questo caso la complessità risulta dunque $O(\log n)$.

Anagramma

Mostrare che la funzione di hash $h(k) = k \bmod m$, quando $m = 2^p - 1$ e le chiavi k consistono di w parole di p bit è invariante rispetto a permutazioni delle parole costituenti la chiave.

Soluzione. L'osservazione fondamentale è che possiamo vedere la chiave come un numero in base 2^p , con le parole che la costituiscono (che variano nel range $0 \dots 2^p - 1$ che rappresentano le sue cifre).

Si conclude dunque osservando che dato un numero n espresso in base b e dette $a_k, \dots, a_1, a_0$ le sue cifre, ovvero

$$n = a_k \dots a_1 a_0$$

allora

$$n \bmod (b-1) = \sum_{j=0}^k a_j \bmod (b-1) \quad (1)$$

Da questo deriva immediatamente che se due numeri n_1 e n_2 si ottengono l'uno dall'altro permutando le cifre, dunque hanno cifre identiche, varrà $n_1 \bmod (b-1) = n_2 \bmod (b-1)$.

La proprietà (1) può essere dimostrata per induzione su k .

- ($k = 0$) Banale.
- ($k \rightarrow k+1$) Sia $n = a_{k+1}a_k \dots a_1a_0$. Detto $n' = a_k \dots a_1a_0$, per ipotesi induttiva, vale $n' \bmod (b-1) = \sum_{j=0}^k a_j \bmod (b-1)$. Poiché $n = a_{k+1}b^{k+1} + n'$, utilizzando le semplici proprietà del resto ovvero

- (1) $(x+y) \bmod b = (x \bmod b + y \bmod b) \bmod b$
- (2) $(x \cdot y) \bmod b = (x \bmod b \cdot y \bmod b) \bmod b$

abbiamo che

$$\begin{aligned} n \bmod (b-1) &= (a_{k+1}b^{k+1} + n') \bmod (b-1) \\ &= (a_{k+1}b^{k+1} \bmod (b-1) + n' \bmod (b-1)) \bmod (b-1) && \text{Proprietà distributiva rispetto al primo passaggio} \\ &= ((a_{k+1} \bmod (b-1))(b \bmod (b-1))^{k+1}) \bmod (b-1) + n' \bmod (b-1) \bmod (b-1) \\ &= ((a_{k+1} \bmod (b-1))1^{k+1}) \bmod (b-1) + n' \bmod (b-1) \bmod (b-1) && \text{Stiamo usando la (2) in entrambi questi passaggi} \\ &= (a_{k+1} \bmod (b-1) + n' \bmod (b-1)) \bmod (b-1) && \text{1 alla (k+1) è 1 ed è trascurabile} \\ &= (a_{k+1} \bmod (b-1) + \sum_{j=0}^k a_j \bmod (b-1)) \bmod (b-1) && \text{Da sopra, } n' = a_k \dots a_1a_0, \text{ quindi è una serie} \\ &= \sum_{j=0}^{k+1} a_j \bmod (b-1) && \text{E quindi, avendo il termine } a_{(k+1)}, \text{ si trasforma tutto in una serie da 0 a } k+1 \end{aligned}$$

e questo conclude la prova.

Dire che la hash è invariante rispetto alle permutazioni significa che, avendo un numero "n", allora la permutazione delle sue cifre è data da una serie di tutti i numeri modulo base di cui è formato (perché ci sarà almeno un numero diverso da essa).

Se k=0 funziona
Per il caso induttivo, vuol dire che vale per tutti i "j"

ABR-InOrder

Questo esercizio fondamentalmente è una dimostrazione di correttezza in due modi:
1) applicando InOrder e trovando le proprietà di ABR
2) dire che abbiamo un ABR e questo funziona perché stiamo usando InOrder

Dimostrare che un albero binario è un Albero Binario di Ricerca se e solo se una visita simmetrica visita i nodi in ordine crescente rispetto alla chiave.

Soluzione. Dimostriamo separatamente le due implicazioni.

- Sia T un albero binario e supponiamo che la procedura **InOrder**,

```
InOrder(x)
  if x <> nil
    InOrder(x. left)
    print x
    InOrder(x. right)
```

Banalmente, la commento in poche righe:

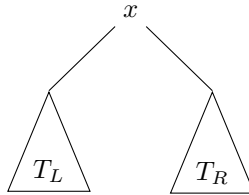
- se l'albero è diverso da vuoto, eseguire InOrder sui nodi sinistri, assicura che a sinistra ci sarà una sequenza ordinata (quindi minore di x)
- stampo " x " radice
- se l'albero è diverso da vuoto, eseguire InOrder sui nodi destri, assicura che a maggiore ci sarà una sequenza ordinata (quindi maggiore di x)

Quindi, valgono le proprietà degli ABR come si vede.

applicata alla radice, stampi i nodi dell'albero in ordine crescente di chiave.

Dimostriamo che T è un ABR per induzione sull'altezza h dell'albero.

- ($h = 0$) L'albero è vuoto e quindi è un ABR.
- ($h > 0$) In questo caso, detta x la radice, l'albero ha la forma



La procedura **InOrder(x)** produce per ipotesi una sequenza ordinata, che, per come funziona la procedura sarà:

$$\underbrace{x_1, \dots, x_l}_{\text{InOrder}(x.\text{left})} \quad x \quad \underbrace{y_1, \dots, y_r}_{\text{InOrder}(x.\text{right})}$$

Dunque **InOrder(x.left)** produce la sequenza ordinata $x_1, \dots, x_l$ dei nodi di T_L , che quindi, per ipotesi induttiva, è un ABR. Allo stesso modo **InOrder(x.right)** produce la sequenza ordinata $y_1, \dots, y_r$ dei nodi di T_R , che quindi, per ipotesi induttiva, è un ABR. Infine, dato che l'intera sequenza è ordinata, $x.\text{key} \geq x_i.\text{key}$, ovvero la chiave di x , è maggiore o uguale a quella dei nodi in T_L . Similmente $x.\text{key} \leq y_j.\text{key}$, ovvero la chiave di x , è minore o uguale a quella dei nodi in T_R . Tutto questo permette di concludere che T è un ABR.

- Per l'implicazione opposta, supponiamo che T sia un ABR e mostriamo che **InOrder** applicata alla radice produce i nodi di T ordinati in senso crescente rispetto alla chiave.

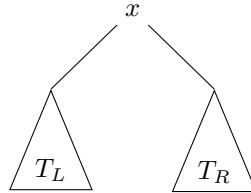
Anche in questo caso procediamo per induzione sull'altezza h dell'albero.

- ($h = 0$) L'albero è vuoto, **InOrder** produce la sequenza vuota che è il risultato atteso.
- ($h > 0$) In questo caso, detta x la radice, l'albero ha la forma

Ciò vale anche alla rovescia:

- se l'albero è vuoto, InOrder produce una sequenza ordinata
- se non fosse vuoto, è identico a prima, cioè:
- eseguire InOrder sui nodi sinistri assicura che a sinistra ci sarà una sequenza ordinata (quindi minore di x)
- eseguire InOrder sui nodi destri assicura che a maggiore ci sarà una sequenza ordinata (quindi maggiore di x)

Correttamente, valgono le proprietà di ABR.



La procedura `InOrder(x)` produce per ipotesi una sequenza ordinata, che, per come funziona la procedura sarà:

$$\underbrace{x_1, \dots, x_l}_{\text{InOrder}(x.\text{left})} \quad x \quad \underbrace{y_1, \dots, y_r}_{\text{InOrder}(x.\text{right})}$$

Per ipotesi induttiva, $x_1, \dots, x_l$ è la sequenza ordinata dei nodi di T_L e $y_1, \dots, y_r$ è la sequenza ordinata dei nodi di T_R . Infine, essendo T un ABR, $x.\text{key} \geq x_i.\text{key}$, dato che la chiave di x , è maggiore o uguale a quella dei nodi in T_L e similmente $x.\text{key} \leq y_j.\text{key}$, dato che la chiave di x , è minore o uguale a quella dei nodi in T_R . Tutto questo implica che l'intera sequenza è ordinata, come desiderato.

Max-Heap Ordinato

Dire se esiste un algoritmo di tempo lineare per elencare gli elementi di un max-heap in ordine decrescente. Descrivere l'algoritmo oppure motivare l'impossibilità di realizzarlo.

Soluzione. Un tale algoritmo non può esistere. Infatti, se esistesse, dato che è possibile costruire un max-heap in tempo lineare con la procedura **Max-Heapify**, se l'algoritmo richiesto esistesse, sarebbe possibile effettuare l'ordinamento in tempo lineare, mentre abbiamo dimostrato un limite inferiore $\Omega(n \log n)$ per l'ordinamento.

Dato che dobbiamo ristabilire le proprietà di max-heap, per forza bisogna ragionare "ad albero" e applicare le proprietà dette in modo logaritmico. Quindi, se fosse un array lineare, potremmo anche ordinarlo linearmente, ma di fatto dovendo ristabilirlo come Max-Heap, è impossibile crearne uno.

ABR lineare

Dire se esiste un algoritmo di tempo lineare per costruire un Albero Binario di Ricerca che sia quasi completo, a partire da un array A . Descrivere l'algoritmo oppure motivare l'impossibilità di realizzarlo.

Soluzione. Un tale algoritmo non può esistere. Infatti, se esistesse, dato che è possibile visitare gli elementi di un ABR in ordine crescente in tempo lineare, sarebbe possibile effettuare l'ordinamento in tempo lineare, mentre abbiamo dimostrato un limite inferiore $\Omega(n \log n)$ per l'ordinamento.

Tradotto: costruire linearmente un max heap, dato che serve un ABR quasi completo. Di fatto, non è possibile, dato che come detto, abbiamo bisogno di crearci i figli e questo comporta una analisi quasi logaritmica da un punto di vista temporale, cosa che sarebbe purtroppo impossibile.

ABR Insert ricorsiva

Realizzare una versione ricorsiva della procedura `Insert(T,x)` per gli alberi binari di ricerca.

Soluzione. La versione ricorsiva inserisce un nodo z nel sottoalbero radicato in un nodo x , avendo come parametro anche il parent *parent* di x . In stile “funzionale”, la funzione ricorsiva ritorna la radice dell’albero nel quale il nodo è stato inserito:

```
InsertRec(x, z, parent)
  if (x == nil)
    z.p = parent
    return z
  else
    if (z.key < x.key)
      x.left = InsertRec(x.left, z, x)
    else
      x.right = InsertRec(x.right, z, x)
    return x
```

Se "x" è nullo, allora ritorno il nodo da inserire "z" e pongo il suo parent al nodo che mi viene passato in input (quindi "parent")

Altrimenti, ragiono nel modo classico di ABR:

- confronto "z" ed "x"
- se "z" è minore di "x", il nodo sinistro sarà "z"
- se "z" è maggiore di "x", il nodo destro sarà "z"

Quindi la funzione di inserimento chiamerà la funzione ricorsiva sulla radice dell’albero:

```
Insert(T, z)
  T.root = InsertRec(T.root, z, nil)
```

La complessità è sempre $O(h)$ dove h è l’altezza dell’albero.

ABR con successore

Realizzare una implementazione degli alberi binari di ricerca nella quale il campo *parent* è sostituito dal campo *successore*, ovvero un nodo *x* avrà i campi *x.key*, *x.left*, *x.right* e *x.succ*, mentre non c'è *x.p*.

Soluzione.

Min, Max, Search

Le operazioni di Min, Max, Search restano invariate.

Per sapere sempre chi è il nodo successore (cioè il nodo successivo nell'attraversamento inOrder di un ABR), occorre modificare:

- la procedura di Insert (sapendo anche chi è il nodo predecessore, cioè il nodo precedente nell'attraversamento inOrder di un ABR)
- sapere sempre chi è il Parent (procedura apposita)
- modificare i riferimenti nella funzione di cancellazione e anche modificare la funzione Transplant (quella che sposta i nodi)

Insert

Per inserire un nodo *z*, come accade nell'implementazione standard, procedo come in una ricerca, scendendo dalla radice fino ad una foglia, mantenendo, oltre che un riferimento *y* al potenziale "parent" anche un riferimento *pred* al potenziale predecessore di *z* (ogni volta che si scende a destra lo si aggiorna, di modo da avere sempre il più prossimo antenato del quale siamo nel sottoalbero destro). Questo serve perché, una volta inserito *z*, detto *pred* il suo predecessore, il campo successore di *pred* dovrà riferire *z*, mentre il successore di *z* sarà quello che prima era il successore di *pred*.

Concretamente, lo pseudocodice può essere il seguente, dove sono commentate solo le "novità" rispetto alla versione ordinaria.

```
Insert(T,z)
  x = T.root
  y = nil
  pred = nil          // pred contiene l'antenato piu' prossimo
                      // del quale si e' nel sottoalbero dx
  while x <> nil
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
      pred = y        // aggiorna il potenziale predecessore

  if y == nil
    T.root = z
  else if z.key < y.key
    y.left = z
  else
    y.right = z

  if pred <> nil        // se ho un predecessore, il successore di z
    z.succ = pred.succ // diventa quello che era il successore di pred
    pred.succ = z      // mentre il successore di pred sarà z

  else                // se invece pred=nil, sono sempre sceso a
    z.succ = y         // sinistra quindi z e' il minimo, e il
                      // suo successore e' il padre
```

L'insert per l'ABR ragiona sugli stessi principi già discussi:

- creare la radice dell'albero e impostare la copia dell'elemento attuale a nil
- finché l'albero esiste (creiamo l'albero rispetto a z)
- ci copiamo la radice
- confronto "z" ed "x"
- se "z" è minore di "x", ci creiamo il nodo sinistro (non serve salvare il predecessore perché, a sinistra, per le proprietà di ABR è già "y")
- se "z" è maggiore di "x", ci creiamo il nodo destro e aggiorniamo il predecessore (quindi, il nodo precedente è letteralmente la copia della radice, cioè il parent del nodo appena inserito).

Una volta creato l'albero:

- se l'albero è nullo, la radice è "z"
- confronto "z" ed "x"
- se "z" è minore di "x", il nodo sinistro sarà "z"
- se "z" è maggiore di "x", il nodo destro sarà "z"

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Ora vediamo se esiste un predecessore:

1

- se non è nullo, una volta inserito "z", ci salviamo come nodo successore (nodo successivo dopo visita InOrder) quello di pred (che può essere "nil" o "y"), perché potremmo aver inserito la radice o inserito a destra
- se è nullo, allora il successore è il padre (perché ho inserito solo a sx e il prossimo nodo percorso è la radice secondo la logica InOrder).

Parent

Per trovare il parent di un nodo x , se assumo che si abbiano ABR nei quali tutte le chiavi siano distinte o nei quali, per ogni nodo w , gli elementi nel sottoalbero sinistro hanno chiave $<$ e nel sottoalbero destro $\geq$ di quella di w , posso operare come se stessi cercando la chiave del nodo x , avendo cura di mantenere traccia del parent z del nodo corrente y durante la ricerca.

```
Parent(T,x)
  y = T.root          // nodo corrente
  z = nil             // parent del nodo corrente

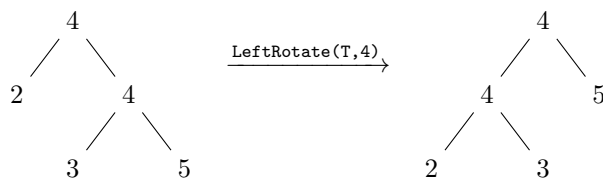
  while (y <> x)
    z = y
    if x.key < y.key
      y = y.left
    else
      y = y.right

  return z
```

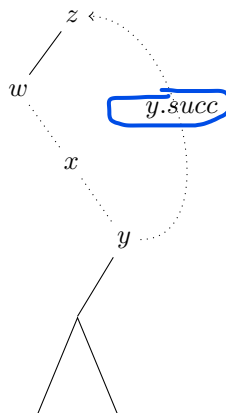
Per salvare il parent:
- creo la radice e il nodo corrente
- inizializzo il parent a nil
- finchè la radice è diversa dal nodo di cui trovare il parent
- salvo come parent la radice del nodo
- confronto "x" ed "y" e creo l'albero sulla base di "x"
- se "x" è minore di "y", creo il nodo sinistro
- se "x" è maggiore di "y", creo il nodo destro
- inseriti tutti i nodi, "z" sarà il parent essendo sempre il nodo precedente a quello attualmente inserito

La complessità risulta $O(h)$ dove h è l'altezza dell'albero.

Tuttavia, a meno che le chiavi non siano tutte diverse, l'assunzione che l'albero soddisfi la proprietà menzionata, anche se è stato costruito con inserimenti e cancellazioni, non è ragionevole, dato che viene invalidata dalle rotazioni



Come calcolare dunque il parent in assenza di questa assunzione? L'idea è la seguente: se x non ha sottoalbero destro, il suo successore è il suo antenato più prossimo, per il quale x è nel sottoalbero sinistro, quindi potrò partire da $x.succ$, scendere al figlio sinistro e quindi sempre a destra fino a che incontrerò x , determinando così il parent. Se invece x ha sottoalbero destro, posso fare la stessa cosa, partendo dal massimo del sottoalbero destro. Segue uno schema e quindi lo pseudo-codice:



Problema:
se ci sono le rotazioni, il parent non rimane tale e l'albero cambia.
Soluzione:
se "x" non ha sottoalbero destro, trovo il primo nodo antenato per cui "x" sia nel sottoalbero sinistro e quindi, grazie a questa cosa, scendendo a sinistra prima o poi troverò "x". Il suo parent è il massimo del sottoalbero dove sta "x"

```

Parent(T,x)
  y = x                      // trovo il massimo del sottoalbero destro di x
  while y.right <> nil        // (se presente)
    y = y.right

  z = y.succ                  // il successore di y e' l'antenato piu' prossimo
                              // di x per il quale x sia nel sottoalbero sinistro

  if z <> nil                 // quindi da y scendendo a sx e poi sempre a dx
    w = z.left               // incontro x
  else
    w = T.root               // se l'antenato in questione e' nil, z era il max
                              // e quindi x si incontra partendo dalla radice e
                              // andando sempre a dx

  if x == w
    return z                 Quindi:
                              - trovo il massimo del sottoalbero dx (se esiste)
                              - z è il nodo successivo dopo la visita InOrder rispetto
                              ad "y" (copia di "x")
                              - se esiste un successore
  while w.right <> x          Semplificazione notevole (si guardi il disegno sopra, ma è così):
    w = w.right              - incontro "x" a cui manca il sottoalbero dx e il suo parent è "w"
                              (perché "w" è il max nel sottoalbero)
                              - incontro "x" a cui manca il sottoalbero sx e il suo parent è "z"
                              (perché "z" è il max nel sottoalbero)

  return w

```

Nel caso peggiore compio un intero percorso dal nodo x fino ad una foglia e dalla radice fino ad x , quindi la complessità è ancora limitata dall'alttezza dell'albero $O(h)$.

Predecessore

Per determinare il predecessore di un nodo, come per il parent, se assumo che si abbiano ABR nei quali tutte le chiavi sono distinte o nei quali nel sottoalbero sinistro trovo solo nodi con chiave minore della radice, posso operare come se stessi cercando la chiave del nodo x , avendo cura di mantenere traccia del predecessore del nodo corrente. Senza questa assunzione posso usare la procedura ordinaria, nella quale, però, l'accesso al padre non avviene leggendo un campo, quindi in tempo $O(1)$, ma chiamando la procedura `Parent` che ha costo $O(h)$.

```

Pred(T,x)
  if x.left <> nil
    return Max(x.left)
  else
    p = Parent(T,x)
    while (p <> nil and x == p.left)
      x = p
      p = Parent(T,x)

  return p

```

Assunzione: tutte le chiavi distinte e albero ordinato
(tutte le chiavi a sinistra sono minori di "x")
Quindi:
- trovo il nodo massimo nel sottoalbero sx
- trovo il parent di "x"
- finché il parent non è nullo e finché "x" è figlio sx
- salvo in "x" il parent e "p" salva il nuovo parent
In questo modo, "p" sarà sempre il predecessore
(nodo precedente in una visita InOrder dell'albero)
ed "x" il nuovo figlio sx.

La complessità, dato che posso avere $O(h)$ chiamate di `Parent`, risulta $O(h^2)$.

Cancellazione

Per la cancellazione ho bisogno di una versione diversa della `Transplant` nella quale aggiorni solo il riferimento dal parent al figlio.

```

Transplant(T,u,v)
  p =Parent(T,u)

  if p == nil
    T.root = v
  else
    if (u == p.left)
      p.left = v
    else
      p.right = v

```

Modifichiamo la Transplant salvo il parent del nodo (ricordando che cerca di sostituire il nodo "u" e il suo sottoalbero corrispondente con un altro nodo "v" e il suo sottoalbero):

- se il parent è "nil", la radice di T è "v" (quindi vuol dire che l'albero nuovo avrà solo "v")
- se il parent non è "nil"
- se "u" è il figlio sx, allora lo sostituiamo con "v"
- se "u" è il figlio dx, allora lo sostituiamo con "v"

Quindi la cancellazione può essere:

```

Delete(T, z)
  pred = Pred(T, z)
  y = z.succ

  if (pred <> nil)
    pred.succ = y

  if (z.left == nil)
    Transplant(T, z, z.right)
  else if (z.right == nil)
    Transplant(T, z, z.left)
  else
    if (y <> z.right)
      Transplant(T, y, y.right)
      y.right = z.right

    Transplant(T, z, y)
    y.left = z.left

```

// determina il predecessore di z
// e il suo successore

// il predecessore di z dopo la cancellazione
// ha come succ l'attuale successore di z

// la parte rimanente e' analoga a quella std

Per la cancellazione:

- troviamo il predecessore e il successore di "z"
- se il predecessore è nullo, significa che l'albero T è vuoto e il suo successore è "y" (unico altro nodo presente e assegnat come successore la riga prima)

- ora dobbiamo cancellare e (uguale a quella standard)

- se il nodo sinistro rispetto a "z" è nullo, sostituisco "z" con "z.right"

- se il nodo destro rispetto a "z" è nullo, sostituisco "z" con "z.left"

- se esiste un nodo successore

- sostituisco "y" con "y.right" e il nodo destro di "y" (che è il successore di "z") è uguale a quello destro del nodo da inserire "z"

- ora sostituisco "z" con "y" (quindi il nodo da inserire viene sostituito con il successore

- sostituisco "y" con "y.left" e il nodo sinistro di "y" (che è il successore di "z") è uguale a quello sinistro del nodo da inserire "z"

La complessità è data dalla complessità del predecessore, quindi $O(h^2)$.

Calcola la LCS tra *spanking* e *amputation* (calcola solo la tabella $L[i, j]$ delle lunghezze).

		a	m	p	u	t	a	t	i	o	n
		0	0	0	0	0	0	0	0	0	0
s	0	0	0	0	0	0	0	0	0	0	0
p	0	0	0	1	1	1	1	1	1	1	1
a	0	1	1	1	1	1	2	2	2	2	2
n	0	1	1	1	1	1	2	2	2	2	3
k	0	1	1	1	1	1	2	2	2	2	3
i	0	1	1	1	1	1	2	2	3	3	3
n	0	1	1	1	1	1	2	2	3	3	4
g	0	1	1	1	1	1	2	2	3	3	4

Sia $n > 0$. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 1 & \text{se } i = j, \\ 2 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) \cdot M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

1. Si scriva una coppia di algoritmi INIT_M(n) e REC_M(i, j) per il calcolo memoizzato di $M(1, n)$.

```
INIT_M(n)
if n=1 then return 1
if n=2 then return 2
for i=1 to n-1 do
    M[i,i] = 1
    M[i,i+1] = 2
M[n,n] = 1
for i=1 to n-2 do
    for j=i+2 to n do
        M[i,j] = 0
return REC_M(1,n)
```

Casi base (matrice di 1 e 2 elementi):
 - (i = j) --> 1
 - (i+1 = j) --> 2

Inizializziamo come dobbiamo fare la matrice M, con un pattern tipico di questi casi:
 - con un ciclo (normalmente, con un indice solo), inizializziamo la matrice M seguendo i casi base:
 - (i = j) --> 1
 - (i = i + 1) --> 2
 - Inizializzo l'ultimo elemento (che userò per cominciare il mio algoritmo), quindi [n,n] = 1
 - Creo i cicli con entrambi gli indici che mi servono
 (con "i" che arriva fino ad "n-2" dato che tolgo i due casi base e "j" che, come si vede anche dalla formula, deve essere > "i+1")

```
REC_M(i,j)
if M[i,j] = 0 then
    M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) * REC_M(i,j-1)
return M[i,j]
```

2. Si calcoli il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 = 2 \sum_{i=1}^{n-2} n - i - 1 = 2 \sum_{k=1}^{n-2} k = (n-2)(n-1)$$

Essendo che $j = i+2$, allora mettendo indice comune "i" bisogna "togliere 2 all'indice i" e quindi, diventa "n-i-1".

Queste due prime serie seguono INIT_M

Per la sommatoria di Gauss, si mette convergenza a "k"

Il risultato della serie è (n-2) moltiplicato ad (n-1); facendosi la tabella si vede perché. Concettualmente, questo succede grazie alla presenza di "i", che ragiona sempre una riga in meno (n-1) di "j".

		i						
j	1	2	0	0	0	0	0	0
	0	1	2	0	0	0	0	0
	0	0	1	2	0	0	0	0
	0	0	0	1	2	0	0	0
	0	0	0	0	1	2	0	0
	0	0	0	0	0	1	2	0
	0	0	0	0	0	0	1	2
	0	0	0	0	0	0	0	1

Si inizializza i fino ad $n - 2$
 righe
 fino ad n
 &
 Si inizializza j fa $i + 2$ colonne
 fino ad n

La ricorrenza, per esempio, fa una cosa del genere (prodotto degli elementi evidenziati in arancione seguendo per esempio rispetto alla casella evidenziata in rosso):

1	2	0	0	0	0	0	0
0	1	2	0	0	0	0	0
0	0	1	2	0	0	0	0
0	0	0	1	2	0	0	0
0	0	0	0	1	2	0	0
0	0	0	0	0	1	2	0
0	0	0	0	0	0	1	2
0	0	0	0	0	0	0	1

Si consideri un insieme di 6 attività a_i , caratterizzate dai seguenti vettori s e f di tempi di inizio e fine: $s = (2,1,2,3,4,7)$ e $f = (4,4,6,11,12,13)$. Determinare l'insieme delle attività compatibili selezionate dall'algoritmo greedy GREEDY_SEL.

$\{a_1, a_5\}$

Dato un file sull'alfabeto $\{A,B,C,D,E\}$ con frequenze, rispettivamente, 12%, 11%, 53%, 10%, 14%, si determinino le 5 codeword ottenute con la codifica di Huffman.

Assumendo di etichettare con il bit 0 il lato che collega il nodo con frequenza minore, e con il bit 1 il lato che collega il nodo con frequenza maggiore, si ottiene

$e(A)=010$

$e(B)=001$

$e(C)=1$

$e(D)=000$

$e(E)=011$

Selezione attività

Attività	1	2	3	4	5	6	7	8	9	10	11	12	13
A1													
A2													
A3													
A4													
A5													
A6													

$A1 = \{2, 4\}$
 $A2 = \{1, 4\}$
 $A3 = \{2, 6\}$
 $A4 = \{3, 11\}$
 $A5 = \{4, 12\}$
 $A6 = \{7, 13\}$

Ricordando che:

“La scelta greedy consiste nello scegliere sempre l'attività successiva il cui tempo di completamento è minimo tra le attività rimanenti e il cui tempo di inizio è maggiore o uguale al tempo di completamento dell'attività precedentemente selezionata. Possiamo ordinare le attività in base al loro tempo di completamento, in modo da considerare sempre l'attività successiva come quella con il tempo di completamento minimo.”

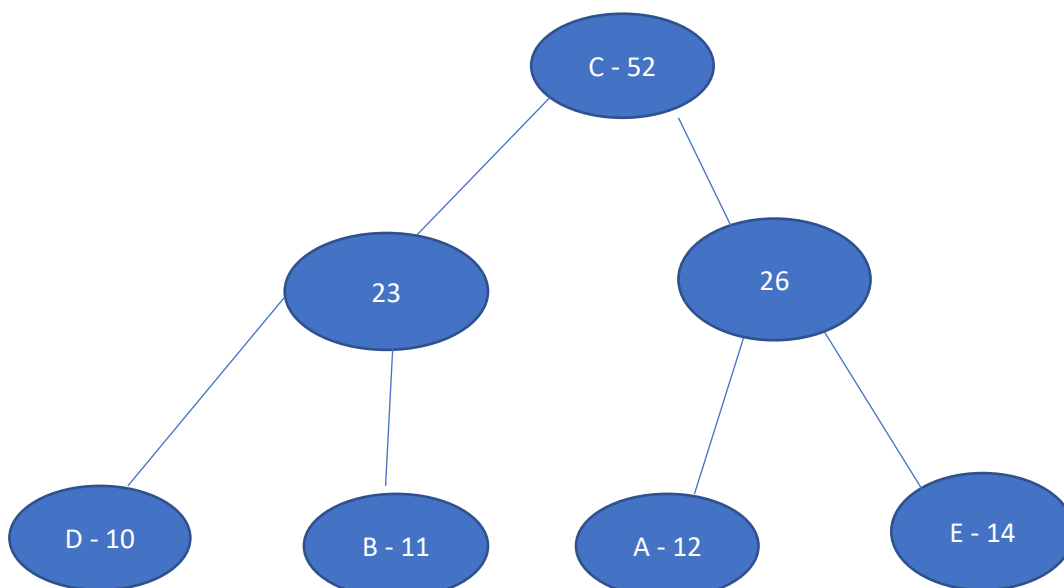
Concretamente:

- Scegliamo l'attività che inizia prima (più corta) → A1
- Scegliamo l'attività che sia più corta e abbiamo un inizio $\geq$ rispetto all'attività che inizia per prima → A5

Huffman (si modifica 53 a 52 per C altrimenti la somma di Huffman non corrisponde)

Si tratta di ordinare i caratteri in base alle frequenze e creare un albero, quindi:

A → 12% B → 20% C → 52% D → 10% E → 14%



Algoritmi e Strutture Dati

AA 2018/2019

Paolo Baldan
Dipartimento di Matematica
Università di Padova

27 settembre 2021

Segue una raccolta di esercizi svolti, suddivisi approssimativamente per aree tematiche. Gli esercizi sono spesso accompagnati da una bozza di soluzione, che contiene una indicazione su come procedere nella soluzione dell'esercizio. Spesso la soluzione è solo abbozzata ed è quindi da intendersi come una guida alla soluzione. In particolare, quando la domanda richieda definizioni o illustrazioni di procedimenti senza contributi originali, la soluzione non le include. Inoltre il documento è di recente redazione e può contenere sviste e piccoli errori. Segnalazioni in questo senso sono benvenute.

Si ringraziano per le segnalazioni (in ordine temporale): Bogdan Stanciu, Bianca Andreea Ciuche, Alberto Schiabel, Alberto Miola, Elisabetta Piombin, Jordan Gottardo, Sara Righetto, Ilaria Peron.

1 Limiti asintotici e ricorrenze

Forma generale ricorrenze:
 $T(n) = aT(n/b) + f(n)$

Domanda 1 Risolvere la ricorrenza $T(n) = 4T(n/2) + n \log n$ utilizzando il master theorem.

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 2$, $f(n) = n \log n$. Si osserva che $\log_b a = 2$ quindi $f(n) = O(n^{\log_b a - \epsilon})$ (per $0 < \epsilon < 1$) e quindi $T(n) = \Theta(n^2)$.

Letteralmente, si applicano gli estremi di MT e si vede che la ricorrenza tende al caso 3.
Completando l'esercizio faccio valere la condizione di regolarità e quindi vale per $k \geq 2$ (si veda il caso 3)

Domanda 2 Mostrare che la ricorrenza $T(n) = T(n/2) + T(n/4) + n$ ammette soluzione $T(n) = \Theta(n)$ utilizzando il metodo di sostituzione.

Soluzione: Si prova separatamente che asintoticamente

1. $T(n) \leq cn$, per un'opportuna costante c
2. $T(n) \geq dn$, per un'opportuna costante d

Ad esempio, per la prima,

$$\begin{aligned} T(n) &= T(n/2) + T(n/4) + n \\ &\leq n/2 c + n/4 c + n \\ &= (1 + 3/4 c)n \\ &\leq cn \end{aligned}$$

Dobbiamo usare il metodo di sostituzione.
Essendo un theta, vuol dire che dobbiamo provare che $T(n)$ è limitata superiormente ed inferiormente a meno di un'opportuna costante.
Il passaggio è dato dal prof;
si ricordi solo di sostituire "c" a $T(n)$ come si vede a lato nel calcolo della disuguaglianza.
Questo significa che (caso 1, ad esempio):
 $T(n/2) + T(n/4) + n \leq cn$
 $c(n/2) + c(n/4) + n \leq cn$ e risolvo

quando $1 + 3/4 c \leq c$, ovvero $c \geq 4$.

Domanda 3 Risolvere la ricorrenza $T(n) = 4T(n/2) + n^2 \sqrt{n}$ utilizzando il master theorem.

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 2$, $f(n) = n^2 \sqrt{n} = n^{\frac{5}{2}}$. Si osserva che $\log_b a = 2$ quindi $f(n) = \Omega(n^{\log_b a + \epsilon})$ (per $0 < \epsilon \leq \frac{1}{2}$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{2}) = 4 \frac{n^2}{4} \sqrt{\frac{n}{2}} = n^2 \sqrt{\frac{n}{2}} \leq cn^2 \sqrt{n}$ per $c \geq \frac{1}{\sqrt{2}}$ (che è < 1). Quindi $T(n) = \Theta(n^2 \sqrt{n})$. Per il confronto del limite, siamo nel caso 3 e applichiamo la condizione di regolarità risolvendo,

Domanda 4 Risolvere la ricorrenza $T(n) = T(n-2) + 2n$ utilizzando il metodo di sostituzione per dimostrare un limite asintotico stretto.

Soluzione: Mostriamo che $T(n) = \Theta(n^2)$ Si prova separatamente che asintoticamente

1. $T(n) \leq cn^2$, per un'opportuna costante c
2. $T(n) \geq dn^2$, per un'opportuna costante d

Ad esempio, per la prima,

$$\begin{aligned} T(n) &= T(n-2) + 2n \\ &\leq c(n-2)^2 + 2n \\ &= c(n^2 - 4n + 4) + 2n \\ &= cn^2 - ((4c-2)n - 4c) \\ &\leq cn^2 \end{aligned}$$

Esattamente come nel secondo esempio di questa serie, quando si dice "limite asintotico stretto" intendiamo un theta e bisogna risolvere matematicamente le disuguaglianze da sopra e da sotto.

quando $(4c-2)n - 4c \geq 0$, che vale per $c > \frac{1}{2}$ e $n \geq \frac{4c}{4c-2}$, ad es. $c = 1$ e $n \geq 2$.

Per la seconda, risulterà, dualmente $(4d-2)n - 4d \leq 0$, che vale, ad esempio, per qualunque n e $d = \frac{1}{2}$.

In questo caso, serve una soluzione esatta, quindi non andiamo a fare come sopra (cioè, calcolare da sopra e da sotto). Abbiamo il caso base $\rightarrow 3$ e il caso induttivo. Dato che questa ricorrenza non ha termini particolari ($n \log n$ ad esempio, che ci fa fare l'albero delle ricorrenze), usiamo il caso più basilare (lineare) $\rightarrow an + b$. Vale per il caso base e vale nel caso induttivo (serve per trovare a) $(T(n-1)+2)=T(n)+2$ $an+b+2=a(n+1)+b \rightarrow$ caso induttivo. Trovo $a = 2$ e con $b=3$ formo $T(n) = 2n + 3$

Domanda 5 Risolvere la ricorrenza

$$T(n) = \begin{cases} 3 & \text{se } n = 0 \\ T(n-1) + 2 & \text{se } n > 0 \end{cases}$$

utilizzando il metodo di sostituzione per determinare una soluzione esatta (non asintotica).

Soluzione: Mostriamo per induzione che $T(n) = an + b$, determinando a e b . Per $n = 0$ si ottiene $T(n) = 3 = b$, quindi $b = 3$. Nel caso induttivo, si ottiene

$$T(n+1) = T(n) + 2 = an + b + 2$$

Affinché $an + b + 2 = a(n+1) + b$ occorre che $a = 2$. Quindi, $T(n) = 2n + 3$.

Domanda 6 Sia data la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 3T(n/5) + T(n/6) + n & \text{se } n > 1 \end{cases}$$

Si fornisca un limite asintotico stretto per la soluzione.

Nota: quando c'è una ricorrenza di questo tipo (composta da uno o più termini), dobbiamo capire a che cosa il limite asintotico è stretto. Basta vedere, quasi sempre, il termine noto. Asintoticamente, tutto tende a quello.

Soluzione: Vale $T(n) = \Theta(n)$. Dimostriamo ad esempio che

$$T(n) = \Omega(n)$$

ovvero che asintoticamente, per un'opportuna costante $c > 0$

$$T(n) \geq cn$$

Infatti

$$\begin{aligned} T(n) &= 3T(n/5) + T(n/6) + n \leq \text{ per ipotesi induttiva} \\ &3cn/5 + cn/6 + n = \\ &c(23/30)n + n \end{aligned}$$

Calcolo espanso bene:
 $3cn/5 + cn/6 + n \geq cn$
 $18cn + 5cn + 30n \geq 30cn$
 $23cn - 30cn \geq -30n$
 $-7n \geq -30n$
 $c \leq 30/7$

vorremmo fosse $\geq cn$, che vale se $(1 - 23/30)c \leq 1$, ovvero $c \leq 30/7$.

Domanda 7 Sia data la seguente equazione di ricorrenza:

$$T(n) = 5T(\lfloor n/3 \rfloor) + 2n^2$$

Si fornisca un limite asintotico stretto per la soluzione.

Come prima, il limite asintotico è stretto rispetto al termine noto.

Essendo un limite che va ad infinito, siamo nel caso 3. Risolviamo facendo valere la condizione di regolarità.

Soluzione: Vale $T(n) = \Theta(n^2)$, tramite il master theorem. Infatti, secondo lo schema generale $a = 5$, $b = 3$, quindi occorre confrontare $n^{\log_3 5}$ e $f(n) = 2n^2$. Facile vedere che, essendo $\log_3 5 < 2$, vale $f(n) = \Omega(n^{\log_3 5 + \epsilon})$ per un opportuno $\epsilon > 0$. Inoltre vale la condizione di regolarità $af(n/b) \leq kf(n)$ per un opportuno $k < 1$, infatti

$$af(n/b) = 5f(n/3) = 5 * 2(n/3)^2 = 10/9n^2 \leq kf(n) = k2 * n^2$$

si riduce a $5/9 \leq k$, che vale, ad es. per $k = 5/9$.

Domanda 8 Sia data la seguente equazione di ricorrenza:

$$T(n) = T(n-1) + \log n$$

Si dimostri che $T(n) = O(n \log n)$.

Soluzione: Si procede una prova induttiva del fatto che $T(n) \leq cn \log n$.

$$\begin{aligned} T(n) &= T(n-1) + \log n \\ &\leq c(n-1) \log(n-1) + \log n \\ &\leq c(n-1) \log n + \log n \\ &= (cn - c + 1) \log n \\ &\leq cn \log n \end{aligned}$$

per $c \geq 1$.

Domanda 9 Sia data la seguente equazione di ricorrenza:

$$T(n) = T(n/2) + T(\sqrt{n}/2) + 2n$$

Si dimostri che $T(n) = \Theta(n)$.

Soluzione: Si procede una prova induttiva del fatto che $T(n) \leq cn$ e $T(n) \geq dn$, per opportune costanti $c, d > 0$, asintoticamente. Dimostriamo prima che $T(n) \leq cn$:

$$\begin{aligned}
T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{\sqrt{n}}{2}\right) + 2n \\
&\leq c\left(\frac{n}{2}\right) + c\left(\frac{\sqrt{n}}{2}\right) + 2n \quad \text{per ipotesi induttiva} \\
&= c\frac{n + \sqrt{n}}{2} + 2n \\
&\leq c(n + \frac{n}{2})/2 + 2n \quad \text{dato che } \sqrt{n} \leq \frac{n}{2} \text{ per } n \geq 4 \\
&\leq c(3n/4) + 2n \\
&= n(3/4c + 2)
\end{aligned}$$

per c tale che $3/4c + 2 \leq c$, ovvero $c \geq 8$.

Per la parte $T(n) \geq dn$, osserviamo che

$$\begin{aligned}
T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{\sqrt{n}}{2}\right) + 2n \\
&\geq d\left(\frac{n}{2}\right) + d\left(\frac{\sqrt{n}}{2}\right) + 2n \quad \text{per ipotesi induttiva} \\
&= d\frac{n + \sqrt{n}}{2} + 2n \\
&\geq d\frac{n}{2} + 2n \quad \text{dato che } \sqrt{n} \geq 0 \\
&\geq dn
\end{aligned}$$

per $d \leq 4$.

Domanda 10 Si consideri la ricorrenza

$$T(n) = T\left(\frac{4n}{5}\right) + \frac{n}{2} + \log n$$

Fornire limite asintotico stretto per la soluzione.

Caso 3: $f(n) \rightarrow \log(n)$ vs $n \log_b(a) \rightarrow 1$
 Deve quindi valere la condizione di regolarità, risolvendo per k .
 Quindi, il limite stretto è per $\theta(n \log_b(a)) \rightarrow \theta(n)$

Soluzione: Si può utilizzare il master theorem dato che la ricorrenza è nella forma

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

con $a = 1$, $b = 5/4$ e $f(n) = n/2 + \log n$.

Vale che $n^{\log_b a} = n^{\log_{5/4} 1} = n^0 = 1$, quindi

$$f(n) = \Omega(n^{\log_b a + \epsilon}) = \Omega(n^\epsilon)$$

per $0 < \epsilon < 1$.

Inoltre vale la condizione di regolarità $a f(n/b) \leq k f(n)$ per qualche $0 < k < 1$. Infatti

$$\begin{aligned}
a f(n/b) &= f(4n/5) \\
&= 2n/5 + \log 4n/5 \\
&= 2n/5 + \log n - \log(5/4) \\
&< k(n/2 + \log n)
\end{aligned}$$

che risulta soddisfatta per $k > 4/5$ (quindi $k/2 > 2/5$) e n abbastanza grande. Quindi $T(n) = \Theta(n + \log n) = \Theta(n)$.

Domanda 11 Si dimostri che la ricorrenza che segue ha soluzione $T(n) = \Theta(n)$

$$T(n) = \frac{1}{2}T(n-1) + n$$

Limite asintotico stretto; si prova per sostituzione la limitatezza da sopra ($\leq cn$) e da sotto ($\geq cn$)

Soluzione: Si prova separatamente che asintoticamente

1. $T(n) \leq cn$, per un'opportuna costante c
2. $T(n) \leq dn$, per un'opportuna costante d

Ad esempio, per la prima,

$$\begin{aligned} T(n) &= \frac{1}{2}T(n-1) + n \\ &\leq \frac{1}{2}cn + n \\ &= \left(\frac{1}{2}c + 1\right)n - \frac{1}{2}c \\ &\leq \left(\frac{1}{2}c + 1\right)n \\ &\leq cn \end{aligned}$$

dove l'ultima disuguaglianza vale xquando $c \geq 2$.

Per la seconda, risulterà, dualmente

$$\begin{aligned} T(n) &\geq \left(\frac{1}{2}d + 1\right)n - \frac{1}{2}d \\ &\geq dn \end{aligned}$$

vale, ad esempio, per qualunque $n \geq 1$ e $d \geq 1$.

Domanda 12 Dare la definizione di $\Omega(f(n))$. Dimostrare che la ricorrenza che segue ha soluzione $T(n) = \Omega(2^n)$

$$T(n) = \sum_{k=1}^{n-1} T(k) T(n-k)$$

Soluzione: Si deve provare che esiste una costante $c > 0$ e un n_0 tali che per ogni $n \geq n_0$

$$T(n) \geq c2^n$$

Si procede induttivamente:

$$\begin{aligned} T(n) &= \sum_{k=1}^{n-1} T(k) T(n-k) \\ &\geq \sum_{k=1}^{n-1} c2^k c2^{n-k} \\ &= \sum_{k=1}^{n-1} c^2 2^n \\ &= c^2 (n-1) 2^n \\ &\geq c^2 2^n \end{aligned}$$

Qui ci sta una serie, ma non ci spaventiamo. Banalmente, basta sostituire al termine della serie 2^n dove appare n e 2^k negli altri casi (dove ci sta $-k$, si mette $2^{-(k)}$.
Quando siamo nel punto "finale" di una serie, si mette sempre l'argomento (c^2) moltiplicato per quello a cui tende la serie $(n-1)$. A quel punto, abbiamo risolto: si trovano solo dei valori che rendono vera la disuguaglianza trovata, cioè: $c^2 2^{(n-1)} \geq c^2 2^n$, che vale appunto (si prova con dei numeri a caso) per $c \geq 1$ e per $n \geq 2$

per $n \geq 2$, e qualunque $c \geq 1$.

Domanda 13 Dare la definizione di $O(f(n))$. Dimostrare che la ricorrenza che segue ha soluzione $T(n) = O(n)$

$$T(n) = \frac{2}{3}T(n-1) + 2n$$

Quindi, porre tutta la disuguaglianza $\leq cn$

Soluzione:

Si deve provare che esiste una costante $c > 0$ e un n_0 tali che per ogni $n \geq n_0$

$$T(n) \leq cn$$

Si procede induttivamente:

$$\begin{aligned} T(n) &= \frac{2}{3}T(n-1) + 2n \\ &\leq \frac{2}{3}c(n-1) + 2n \\ &\leq cn \end{aligned}$$

Passaggi completi:

$$\frac{2}{3}cn - \frac{2}{3}c + 2n \leq cn$$

$$2cn - 2 + 6n \leq 3cn \text{ (moltiplico tutto per 3)}$$

$$-2 + 6n \leq cn$$

E quando non ha senso, come qui, andare avanti a calcolare, si sostituisce un valore e si vede per che cosa vale per $c \geq 3$.

Affinché l'ultima disuguaglianza valga, occorre che

$$c\left(\frac{1}{3}n + \frac{2}{3}\right) \geq 2n$$

Ora

$$c\left(\frac{1}{3}n + \frac{2}{3}\right) \geq \frac{2}{3}n \geq 2n$$

dove l'ultima disuguaglianza è certamente verificata per per ogni n , se $c \geq 3$.

Domanda 14 Dare una soluzione asintotica per la ricorrenza $T(n) = 3T(n/2) + n(n+1)$.

Confronto tra: $f(n) = n(n+1)$ e $n \log_b(a) = n \log_2(3)$. Di nuovo, è un limite che va ad infinito (caso 3), per cui deve valere la condizione di regolarità.

Soluzione: Si usa il master theorem con $a = 3$, $b = 2$, $f(n) = n(n+1)$. Si deve confrontare $f(n)$ con $n^{\log_b a} = n^{\log_2 3}$ ed essendo che $\log_2 3 < 2$ per qualunque $0 < \epsilon < 2 - \log_2 3$ si vede facilmente che $f(n) = \Omega(n^{\log_2 3 + \epsilon})$.

Per concludere che $T(n) = \Theta(f(n)) = \Theta(n^2)$ usando il caso 3 del master theorem occorre dimostrare la regolarità di $f(n)$, ovvero che $af(n/b) < kf(n)$ per $0 < k < 1$, asintoticamente. Si imposta

$$af(n/b) = 3n/2(n/2 + 1) < kn(n+1)$$

e si vede che $n/2 + 1 < 5n/8$ per $n > n_0 = 8$, quindi sotto questa condizione

$$af(n/b) = 3n/2(n/2 + 1) < 15/16n(n+1)$$

ovvero $k = 15/16$ va bene.

Questa è un poco più complessa e sinceramente poco utile: passare oltre.

Domanda 15 Data la ricorrenza $T(n) = T(n/2) + T(n/3) + \sqrt{n} + 2$ dimostrare che ha soluzione $T(n) = O(n)$. Il limite è stretto, ovvero vale anche $T(n) = \Omega(n)$? [Questa seconda parte della domanda, per un errore nella scelta dei coefficienti risultava più complessa di quanto preventivato, quindi non è stata considerata nella valutazione. Allego comunque la soluzione per completezza.]

Soluzione: Per dimostrare che $T(n) = O(n)$ occorre trovare $c > 0$ e $n_0 \in \mathbb{N}$ tale che per ogni $n \geq n_0$, $T(n) \leq cn$. Procediamo con un ragionamento induttivo

$$\begin{aligned} T(n) &= T(n/2) + T(n/3) + \sqrt{n} + 2 \\ &\leq cn/2 + cn/3 + \sqrt{n} + 2 \\ &= cn/2 + cn/3 + \sqrt{n} + 2 \\ &= 5/6cn + \sqrt{n} + 2 \\ &= 5/6cn + n + 2 \\ &\leq cn \end{aligned}$$

vale se $1/6cn \geq n + 2m$, quindi, ad esempio, per $c \geq 7$ e $n \geq 12$.

Il limite non è stretto. Un tentativo di dimostrazione induttiva che $T(n) \geq dn$ per qualche $d > 0$ fallisce. Tuttavia questo non consente di concludere.

Si può invece dimostrare che $T(n) = O(n^\alpha)$ con $\alpha = 5/6$. Infatti, ancora in modo induttivo:

$$\begin{aligned}
T(n) &= T(n/2) + T(n/3) + \sqrt{n} + 2 \\
&\leq cn/2 + cn/3 + \sqrt{n} + 2 \\
&= c(n/2)^\alpha + c(n/3)^\alpha + \sqrt{n} + 2 \quad [\text{dato che } \sqrt{n} + 2 \leq n^\alpha \text{ per } n \geq 5] \\
&\leq cn^\alpha((1/2)^\alpha + (1/3)^\alpha) + n^\alpha \\
&\leq cn^\alpha
\end{aligned}$$

vale se $c(1 - (1/2)^\alpha - (1/3)^\alpha)n^\alpha \geq n^\alpha$, quindi, osservando che la quantità $1 - (1/2)^\alpha - (1/3)^\alpha > 0$, è sufficiente scegliere $c \geq 1/(1 - (1/2)^\alpha - (1/3)^\alpha)$ e $n \geq 5$.

Questo prova che $T(n) = O(n^\alpha)$ e quindi non può essere $T(n) = \Omega(n)$.

Domanda 16 Data la ricorrenza $T(n) = 5T(n/3) + (n - 2)^2$, trovare la soluzione asintotica.

Siamo sempre nella forma generale di ricorrenza $T(n) = aT(n/b) + f(n)$ e confronto $(n-2)^2 = f(n)$ e $\log_3(5) = \log_b(a)$. Caso 3, condizione di regolarità e si risolve per k .

Soluzione: Si usa il master theorem con $a = 5$, $b = 3$, $f(n) = (n - 2)^2$. Si deve confrontare $f(n)$ con $n^{\log_b a} = n^{\log_3 5}$ ed essendo che $\log_3 5 < 2$ per qualunque $0 < \epsilon < 2 - \log_2 3$ si vede facilmente che $f(n) = \Omega(n^{\log_2 3 + \epsilon})$.

Per concludere che $T(n) = \Theta(f(n)) = \Theta(n^2)$ usando il caso 3 del master theorem occorre dimostrare la regolarità di $f(n)$, ovvero che $af(n/b) < kf(n)$ per $0 < k < 1$, asintoticamente. Si imposta

$$af(n/b) = 5(n/3 - 2)^2 < k(n - 2)^2$$

e si osserva

$$5(n/3 - 2)^2 < 5((n - 2)/3)^2 = 5/9(n - 2)^2$$

per cui si può scegliere $k = 5/9 < 1$ e n qualunque.

Domanda 17 Dare la definizione di $\Omega(f(n))$. Mostrare che se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora $f(n) = \Omega(h(n))$.

Nei seguenti due casi, si devono dimostrare le condizioni date usando le definizioni formali (con i limiti).

$f(n) = \Omega(g(n)) \rightarrow 0 < c_1 g(n) \leq f(n)$

$g(n) = \Omega(h(n)) \rightarrow 0 < c_2 h(n) \leq g(n)$

Soluzione: Si ha che Metto insieme e ottengo che: $0 < c_2 h(n) \leq c_1 g(n) \leq f(n)$ (con $h(n)$ sotto dato che poi uso $g(n)$ anche per f

$$\Omega(f(n)) = \{g(n) \mid \exists c > 0. \exists n_0 \in \mathbb{N}. \forall n \geq n_0. 0 \leq cg(n) \leq f(n)\}.$$

Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora esistono $c_1, c_2 > 0$, $n_1, n_2 \in \mathbb{N}$ tali che per ogni $n \geq n_1$

$$0 \leq c_1 g(n) \leq f(n) \tag{1}$$

e per ogni $n \geq n_2$

$$0 \leq c_2 h(n) \leq g(n) \tag{2}$$

Ne consegue che per ogni $n \geq \max\{n_1, n_2\}$, moltiplicando (2) per c_1 si ha

$$0 \leq c_1 c_2 h(n) \leq c_1 g(n) \leq f(n)$$

ovvero, indicato con $n_0 = \max\{n_1, n_2\}$ e $c = c_1 c_2$, per ogni $n \geq n_0$,

$$0 \leq ch(n) \leq f(n)$$

ovvero $f(n) = \Omega(h(n))$.

Qui metto insieme le costanti c_1 e c_2 tali che valga per tutte le funzioni considerate, partendo da $h(n)$. "Togliendo di mezzo" c_1 e c_2 (metto una costante sola, tale che sia il massimo e quindi $g(n)$ diventa "inclusa" in $h(n)$ --> essendo che metto una costante sola e $h(n)$ ne ha due, $g(n)$ diventa trascurabile. A quel punto, notiamo che $h(n)$ è un limite inferiore (omega).

Domanda 18 Dare la definizione di $\Theta(f(n))$. Mostrare che $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$.

Soluzione: Per la prima parte

$$\Theta(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c, c_2 > 0. \forall n \geq n_0. 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)\}.$$

Per dimostrare che $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$, ricordiamo che

1. $O(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c > 0. \forall n \geq n_0. 0 \leq g(n) \leq c f(n)\}$
2. $\Omega(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c > 0. \forall n \geq n_0. 0 \leq c f(n) \leq g(n)\}.$

Proviamo separatamente le due inclusioni. Se $g(n) \in \Theta(n)$ allora chiaramente $g(n) \in \Omega(f(n))$, grazie all'esistenza di n_0 e c_1 , e similmente $g(n) \in O(f(n))$, grazie all'esistenza di n_0 e c_2 .

Per il contrario, se $g(n) \in \Omega(f(n))$ e $g(n) \in O(f(n))$, per la prima esistono $c_1 > 0$, $n_1 \in \mathbb{N}$ tali che per ogni $n \geq n_1$ vale

$$0 \leq c_1 f(n) \leq g(n)$$

e per la seconda esistono $c_2 > 0$, $n_2 \in \mathbb{N}$ tali che per ogni $n \geq n_2$ vale

$$0 \leq g(n) \leq c_2 f(n)$$

Quindi, detto $n_0 = \max(n_1, n_2)$ per ogni $n \geq n_0$,

$$0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)$$

ovvero $f(n) \in \Theta(f(n))$.

Anche qua bisogna "mettere insieme i pezzi usando le definizioni".

Quindi, avremo che

$\Theta(f(n)) =$

$O(c_1 f(n)) \cap \Omega(g(n))$

intersecato con

$O(g(n)) \cap \Omega(c_2 f(n))$

e mettendo insieme avremo che

$O(c_1 f(n)) \cap \Omega(g(n)) \cap O(g(n)) \cap \Omega(c_2 f(n))$

Quindi, si nota che $g(n)$ sta "in mezzo" tra gli $f(n)$ e si nota che ciò corrisponde ad un limite stretto $\Theta(f(n))$.

2 Divide et impera e Ricorsione

Esercizio 1 Dato un array di interi $A[1..n]$, chiamiamo *gap* un indice $i \in [1, n]$ tale che $A[i + 1] - A[i] > 1$.

- Mostrare per induzione su n che un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ (quindi $n \geq 2$) contiene almeno un gap.
- Fornire lo pseudocodice di una procedura ricorsiva *divide et impera gap* che dato un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ restituisce un gap in A .
- Valutare la complessità della funzione, utilizzando il master theorem.

Soluzione: La prova per induzione è la seguente:

- $(n = 2)$ Banale, il gap è in $i = 1$.

1) Per contenere almeno un gap vuol dire che la somma di due indici consecutivi è maggiore di 1. Il caso base è banale; il caso induttivo richiede di "buttarsi in mezzo" gli indici n ed $n+1$.

- $(n > 2)$ Sia $A[n] - A[1] \geq n$. Allora se $A[n] - A[n-1] > 1$, si ha che $i = n-1$ è un gap e abbiamo concluso. Altrimenti, si osserva che $A[n] - A[1] = A[n] - A[n-1] + A[n-1] - A[1]$. Quindi $A[n-1] - A[1] = (A[n] - A[1]) - (A[n] - A[n-1]) \geq n-1$, quindi per ipotesi induttiva $A[1..n-1]$ contiene un gap.

La prova funziona anche se invece che spezzare l'array in $A[1..n-1]$ e $A[n-1..n]$, lo si spezza in $A[1..[n/2]]$ e $A[[n/2]..n]$. Da questo segue l'algoritmo:

```
gap(A,p,r)    // p < r (almeno due elementi) , A[r] - A[p] >= r-p+1
  if p = r-1
    return p
  else
    q = (p+r)/2
    if A[q] - A[p] > q-p+1
      gap(A,p,q)
    else
      // note che allora A[r] - A[q] > r-q+1
      gap(A,q,r)
```

Se l'array A contiene meno di due elementi, allora ritorna il primo elemento

Altrimenti, si trova l'elemento che sta a metà e
- se l'elemento di metà meno l'elemento estremo inferiore
continuano a rispettare la proprietà di gap --> allora, ricorro sulla prima metà

Nuovamente, altrimenti
- se l'elemento di metà meno l'elemento estremo superiore
continuano a rispettare la proprietà di gap --> allora, ricorro sulla seconda
metà

Si ha che $T(n) = T(n/2) + 1$, quindi il master theorem, dato che $n^{\log_b a} = n^{\log_2 1} = n^0 = \Theta(1)$, ci dice che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

La correttezza è già argomentata grazie al punto 1;
essendo, come si vede, una cosa simile a Merge Sort, allora avrà complessità logaritmica.

Esercizio 2 Scrivere una procedura di tipo *divide et impera over* che dato un array di interi distinti $A[1..n]$, ordinato in modo crescente, e un intero x restituisce l'indice del più piccolo elemento in A strettamente maggiore di x . Se nessun elemento di A soddisfa la condizione, si restituisca $n+1$. Valutare la complessità dell'algoritmo.

Soluzione: Si realizza una funzione ricorsiva *over(A,p,r,x)* che restituisce il minimo indice $i \in [p, r]$ tale che $A[i] > x$ se un tale indice esiste, altrimenti $r+1$.

```
over(A,p,r,x)
  if r-p < 0
    return r+1
  else
    q = floor((r+p)/2)
    if A[q] <= x
      return over(A,q+1,r,x)
    else
      return over(A,p,q-1,x)
```

Se l'array A contiene meno di due elementi, allora ritorna il primo elemento
(r+1 perché così, se r fosse vuoto, allora prendo il primo elemento di sicuro)

Altrimenti, trovo la metà e:
- se l'elemento medio è $\leq x$, ricorro sulla seconda metà per trovare il primo elemento
strettamente maggiore di x
- se l'elemento medio è $> x$, ricorro sulla prima metà per trovare il primo elemento
strettamente maggiore di x
Correttezza argomentata sotto e idea simile a MergeSort --> complessità $\Theta(\log(n))$

La correttezza può essere dimostrata per induzione sulla lunghezza n del sottoarray, ovvero $n = |r - p + 1|$. Quando $n = 0$ quindi $r < p$ l'intervallo $[p, r]$ è vuoto, e correttamente la funzione ritorna $r + 1$. Se invece $n > 0$, ovvero $r > p$, sia $q = \lfloor p + r/2 \rfloor$. Distinguiamo due casi:

- se $A[q] > x$ allora, per ipotesi induttiva, la chiamata $\text{over}(A, p, q-1, x)$ ritorna il minimo indice i in $[p, q-1]$ tale che $A[i] > x$, se un tale indice esiste, e in questo caso è anche il minimo indice in $[p, r]$ con questa proprietà. Se nessun elemento in $A[p, q-1]$ è maggiore di x correttamente ritorna $q - 1 + 1 = q$ che $[p, q-1]$ sarà il minimo indice in $[p, r]$ di un elemento maggiore di x .
- se $A[q] \leq x$ allora la chiamata $\text{over}(A, q+1, r, x)$ ritorna il minimo indice i in $[q+1, r]$ tale che $A[i] > x$, se un tale indice esiste (e in questo caso sarà anche il minimo indice in $[p, r]$), e $r + 1$ altrimenti.

La complessità è data dalla ricorrenza $T(n) = T(n/2) + c$ e quindi $T(n) = \Theta(\log n)$ dal master theorem.

Esercizio 3 Realizzare una procedura di tipo divide et impera $\text{Max}(A, p, r)$ per trovare il massimo nell'array $A[p, r]$. Si assuma che l'array non sia vuoto ($p \leq r$). Scrivere lo pseudocodice e valutare la complessità con il master theorem.

Soluzione:

```
Max(A, p, r)
  if p = r
    return A[p]
  else // p < r
    q = (p+r)/2
    m1 = Max(A, p, q)
    m2 = Max(A, q+1, r)
    if m1 < m2
      return m2
    else
      return m1
```

Max: troviamo il massimo in un array divide ed impera.

Se l'array è composto di un solo elemento, ritorniamo esattamente quello, altrimenti

- se abbiamo almeno due elementi, prendiamo l'elemento medio e recuperiamo il minimo della prima metà e della seconda metà.

- semplicemente, se è più grande il massimo della seconda metà $m2$, ritorniamo proprio $m2$

- altrimenti, ritorniamo $m1$, quindi il massimo della prima metà

Supponendo che l'array sia ordinato, incrementa solo sulla seconda metà; per questo, rimane (p, q) sulla prima ricorsione,

Dato che ricorriamo successivamente sulle due metà avremo una complessità del tipo $T(n) = \theta(\log(n))$, con una ricorrenza del tipo $T(n) = T(n/2) + c$

Esercizio 4 Sia $A[1..n]$ un array di interi distinti ordinato in senso crescente. Dimostrare che dato un qualunque indice i , se $A[i] > i$ allora $A[j] > j$ per ogni $j > i$ e analogamente se $A[i] < i$ allora $A[j] < j$ per ogni $j < i$.

Utilizzare l'osservazione per realizzare una funzione $\text{Fix}(A)$ che dato l'array di interi $A[1..n]$ ordinato senza ripetizioni restituisce un indice i tale che $A[i] = i$, se esiste, e 0 altrimenti. Valutarne la complessità.

Soluzione: Per la prima parte sia i un indice tale che $A[i] > i$, ovvero $A[i] \geq i + 1$. Si osserva che, se $i < n$ allora $A[i + 1] > A[i] \geq i + 1$ ovvero $A[i + 1] > i + 1$. Usando questo fatto, si conclude con un ragionamento induttivo (che per ogni $j \geq 1$ vale $A[i + j] \geq i + j$).

A questo punto, un algoritmo può essere il seguente

```
Fix(A)
  return Fix-rec(A, 1, n)
```

Pattern che ogni tanto si usa, per comodità ricorsiva: passiamo ad una funzione ausiliaria tutto l'array da scansionare ed il primo indice e poi "facciamo robe".

```
Fix-rec(A, p, r)
```

```

if p <= r
  q = p+r/2
  if A[q] = q
    return q
  else if A[q] < q
    return Fix(A,q+1,r)
  else
    return Fix(A,p,q-1)
else
  return 0

```

Fix:

trovare un indice tale che $A[i] = i$, dovendo dimostrare che dato un qualunque indice i , se $A[i] > i$ allora $A[j] > j$ per ogni $j > i$ e analogamente se $A[i] < i$ allora $A[j] < j$ per ogni $j < i$.

Se questo fatto è vero (correttezza):

- nella prima metà dell'algoritmo, avremo tutti gli indici "i" per cui $A[i-1] < A[i]$ e ricorriamo nella prima metà di array. Questo vale per il caso $i=0$ (caso base) ed il passo induttivo, che porta ad avere $A[j] > i$, essendo ordinato.
 - nella seconda metà dell'algoritmo, avremo tutti gli indici "j" per cui $A[j-1] < A[j]$ e ricorriamo nella prima metà di array. Questo vale per il caso $j=0$ (caso base) ed il passo induttivo, che porta ad avere $A[j] > i$, essendo ordinato.
 - ricorsivamente, se succedesse questa cosa, esisterà almeno un indice Aq tale per cui, ricorrendo su due metà ordinate, troviamo l'elemento che serve.

Ricorsione su due sottometà di array --> complessità logaritmica.

Il costo è dato dalla ricorrenza $T(n) = T(n/2) + c$ che ha soluzione $T(n) = \Theta(\log n)$.

Domanda 19 Scrivere una funzione ricorsiva `subseq(X,Y,m,n)` che date due sequenze $X[1..m]$ e $Y[1..n]$, di lunghezza m e n rispettivamente, verifica se X è una sottosequenza di Y e restituisce un valore booleano conseguente. Valutarne la complessità.

Soluzione: La soluzione è:

subseq: dati due array $X[1..m]$ e $Y[1..n]$ determina se X è una sottosequenza di Y

```

subseq(X,Y,m,n):
  if m == 0
    return True
  elif m <= n
    if X[m] == Y[n]:
      return subseq(X,Y,m-1,n-1)
    else:
      return subseq(X,Y,m,n-1)
  else
    return False

```

Dobbiamo verificare che X sia sottosequenza di Y :

Esempio:

$X = 456$

$Y = 1234567$

(Analisi e correttezza)

Dato che abbiamo due array e ci vengono forniti due indici, li usiamo per ricorrere sulle due metà di array.

Caso base: $m=0$, array vuoto ed è certamente vero che il vuoto sia sottosequenza di y .

Caso induttivo: $m \leq n$

A questo punto, cerchiamo su entrambi gli array in modo ricorsivo, usando Y .

Se X ed Y sono uguali, allora ricorro su entrambi gli array, mantenendo la proprietà di uguaglianza (vale per l'indice m e continua a valere per i successivi)

Se X e Y sono diversi, allora ricorro all'indietro su Y , dato che X è un sottoarray/sottosequenza di X e riusciamo ad avere la proprietà di uguaglianza in loco.

Per quanto riguarda la complessità si ha che ogni chiamata, esclusa la parte ricorsiva, ha costo costante e nel caso peggiore scorro tutta la seconda stringa, quindi ci sono n chiamate. Quindi il costo è $O(n)$.

Non stiamo scansionando dividendo l'array in due metà e con una ricorsione per condizione, esaminiamo linearmente le due strutture. Complessità lineare.

Esercizio 5 Un array $A[1..n]$ di numeri si dice *alternante* se non ha elementi contigui identici (ovvero per ogni $i \leq n-1$ vale $A[i] \neq A[i+1]$) e inoltre per ogni $i \leq n-2$, vale che $a_i < a_{i+1} > a_{i+2}$ oppure $a_i > a_{i+1} < a_{i+2}$. Ad esempio gli array $[1, 2, -1, 3, 2]$ e $[5, 1, 2, -1, 3, 2]$ sono alternanti, mentre non lo sono $[1, 2, 3]$ e $[1, 1, 2]$. Scrivere una funzione ricorsiva `alt(A,n)` che dato un array $A[1..n]$ di numeri verifica se è alternante. Valutarne la complessità.

Soluzione:

Alternante:

se abbiamo un indice "i" significa che:
 $X > i < Y$ oppure $X < i > Y$

alt: dato un array $A[1..n]$ verifica se è alternante

```

alt(A,n)
  return altRec(A,n,0) or altRec(A,n,1)

```

Come prima, passiamo ad una procedura ausiliaria, che considera una ricorsione con una variabile "dir" che indica la direzione in cui impostare la ricorsione.

```

altRec(A,i,dir)
  # verifica se A[1..i] è alternante.
  # usa un ulteriore parametro per indicare se la sequenza alternante deve

```

```
# concludersi crescendo (0) o decrescendo (1).
```

```

if i==1
    return True
else
    if dir==0
        return altRec(A,i-1,1) and (A[i-1] < A[i])
    else
        return altRec(A,i-1,0) and (A[i-1] > A[i])

```

Infatti, il caso base è vero (un indice solo, certamente alternante con sé stesso).
 Il caso induttivo considera che:
 - se $dir = 0$, allora l'elemento precedente è minore ad "i", allora ricorriamo sull'elemento precedente dell'array e vale true per il fatto che il precedente è minore di i
 - se $dir = 1$, allora l'elemento precedente è maggiore ad "i", allora ricorriamo sull'elemento precedente dell'array e vale true per il fatto che il precedente è maggiore di i.

Quando troviamo true, significa che alternativamente la sequenza va "su" e "giù" e, grazie ai "true/false", certamente abbiamo che continuiamo a trovare indici che rispettano la proprietà:
 banalmente, significa che non ci sono elementi uguali e continuando a ricorrere, vedo che tutti gli elementi sono diversi, grazie al fatto che consideriamo due metà distinte di array.

Dato che la parte non ricorsiva è di costo costante, la complessità si può esprimere come $T(n) = 1 + T(n-1)$ che porta a $T(n) = \Theta(n)$.

Una soluzione più compatta:

```

# alt: dato un array A[1..n] verifica se e' alternante
alt(A,n)
    return altRec(A,n,true) or altRec(A,n,false)

```

```

altRec(A,i,dir)
    # verifica se A[1..i] e' alternante.
    # usa un ulteriore parametro per indicare se la sequenza alternante deve
    # concludersi crescendo (true) o decrescendo (false).

```

```

if i==1
    return True
else
    return altRec(A,i-1,not dir)
           and (A[i-1] <> A[i])
           and (dir xnor (A[i-1] > A[i])) # usa il fatto che
                                           # true  xnor V = not V
                                           # false xnor V = V

```

Soluzione compatta che fa la stessa cosa:
 ricorre considerando che gli elementi precedenti sia diversi e decrescente
 e
 in base alla variabile "dir" determina se a coppie gli elementi crescono "true=1"
 o decrescono "false=0".

3 Ordinamento

Esercizio 6 Fornire lo pseudocodice di una procedura $\text{min}(A, B)$ che dati due array A e B che siano uno la permutazione dell'altro ne trova l'elemento minimo confrontando esclusivamente elementi di A ed elementi di B (non si possono confrontare due elementi di A o due elementi di B tra loro, e non si possono fare copie degli array, ovvero la funzione deve operare con spazio costante). Valutare la complessità della funzione.

Soluzione: L'idea è quella di confrontare gli elementi di A e B , scorrendoli con due indici i, j avanzando in B quando $A[i] \leq B[j]$ e in A altrimenti. Sul lato di A l'avanzamento si fermerà ad un elemento $A[i]$ che sarà minore o uguale di tutti gli elementi di B . Il vero e proprio invariante è un po' complicato.

```

min(A,B)
  i=j=1
  // invariante: (forall h \in [0,i) exists k in [0,j] A[h] > B[k]) or
                  (i <= n) and (forall k \in [0,j) exists h in [0,i] A[h] <= B[k])

  while (j <= n)
    if A[i] <= B[j]
      j = j+1
    else
      i = i+1
  return A[i]
```

Dato che non possiamo confrontare elementi dello stesso array tra di loro o fare copie, l'unica idea è di usare due indici per analizzare i due array a coppie. Inizializzando gli indici $i = j = 1$, analizziamo che: se "j" aumenta, il minimo sarà in "i = j-1" altrimenti se "i" aumenta, il minimo sarà in "i". Ritorno poi l'indice minimo dell'array, che sarà sull'indice i-esimo (perché "i" e "j" partono dallo stesso indice e se vale la proprietà per il caso base $i = j = 1$, allora sapremo che esiste certamente un indice minimo, dato dall'indice "i" o "i-1" della scansione. Scansioni in tempo costante (confronti) esaminando due array linearmente con un ciclo --> Lineare

Si noti che l'invariante implica $i \leq n$, quindi questo controllo nel ciclo non è necessario.

Il numero di iterazioni è al massimo $2n$ dato che uno dei due indici (i oppure j) viene sempre incrementato. Dunque la complessità è $\Theta(n)$.

Domanda 20 Realizzare una funzione $\text{RevCountingSort}(A, B, n, k)$ che, dato un array $A[1..n]$ contenente interi nell'intervallo $[0..k]$, restituisce in $B[1..n]$ una sua permutazione ordinata in modo decrescente utilizzando una variante del counting sort. Valutarne la complessità.

Soluzione:

```

RevCountingSort (A, B, n, k)
  C[1..k] = 0
  for i = 1 to n
    C[A[i]] = C[A[i]] + 1

  for j = k-1 downto 0 do
    C[j] = C[j] + C[j+1]

  for j = 1 to n
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[j]] - 1
```

A differenza di CountingSort, abbiamo "n" come variabile da gestire. Quindi: - inizializziamo l'array di conteggio - contiamo quante volte appaiono i singoli elementi in A e li salvo in C - Qui cambia la musica: siccome è una permutazione, scansiono l'array C partendo dalla fine e ritorno una disposizione degli elementi ordinati in modo decrescente. - metto dentro all'array B i numeri guardando gli indici ed i valori che ho messo in C.

Esercizio 7 Realizzare una funzione $\text{Anagramma}(x, y)$ che date due stringhe x e y sull'alfabeto $\{0, 1\}$, verifica se x è un anagramma di y restituendo conseguentemente **true** o **false**. Una stringa è vista come un array di caratteri, con lunghezza data dall'attributo `len`. Ad esempio, la stringa x è la sequenza di caratteri $x[1] \ x[2] \ \dots \ x[x.\text{len}]$. Valutare la complessità.

Si assume che "x" ed "y" siano della stessa lunghezza "len"

Soluzione:

- i. L'implementazione è la seguente:

112233 = x

221133 = y

```
Anagramma(x,y)
    diff[0..1] = 0      # diff[v] = difference between number of v's in x and
                        # number of v's in y, up to now

    for i=1 to x.len
        diff[x[i]]++
        diff[y[i]]--

    if (diff[0] == 0) and (x.len = y.len)
        # adding (diff[1] == 0) is useless as implied by the first two conditions
        return true
    else
        return false

# nota: il controllo sulla lunghezza poteva essere anticipato
```

Esempio concreto di anagramma:

calendario = locandiera --> due stringhe che, con lo stesso numero di caratteri formano due parole di senso compiuto.

Quindi, come detto, serve contare i caratteri e lo faremo in modo lineare, sfruttando l'idea "in-place" di Counting Sort.

Quello che faccio è molto semplice:

- inizializzo un array di supporto chiamato "diff" pieno di zeri

- esaminiamo sia "x" che "y" e confronto gli elementi;

se hanno lo stesso numero di caratteri per ogni singolo carattere, allora

"diff" sarà pari a 0. Se questo capita, allora abbiamo a che fare con un anagramma

altrimenti è una permutazione, ma non un anagramma.

Correttezza:

- caso base, indice vuoto, array anagramma vero

- caso induttivo, conto i caratteri:

se "diff" è pari a 0, tutti gli elementi sono un anagramma, partendo dal caso i-esimo

se "diff" è diverso da 0, non abbiamo a che fare con un anagramma

- ii. Complessità chiaramente $\Theta(n)$.

Domanda 21 Realizzare una funzione `Double(A,n)` che, dato un array `A[1,n]` ordinato in senso crescente, verifica se esiste una coppia di indici `i, j` tali che `A[j] = 2 * A[i]`. Restituisce la coppia se esiste e (0,0) altrimenti. Scrivere lo pseudocodice e valutare la complessità.

Soluzione:

Il codice può essere:

`A = [1 5 7 8 9 10]`

```
double(A,n) {
    n = A.length
    // assume A[1,n] ordinato
    // ritorna una coppia i, j
    // tale che 2*A[i] = A[j]

    i=1
    j=1
    while (i<=n) and (j<=n) and (2*A[i] <> A[j])
        if (2*A[i] > A[j])
            j++
        else
            i++

    if (i<=n) and (j<=n)
        return (i,j)
    else
        return (0,0)
```

`Double`: si cerca se esiste coppia di indici per cui un elemento con indice "i" è il doppio dell'elemento con indice "j".

Quindi:

- inizializzo gli indici e trovo la lunghezza dell'array

- finché non abbiamo esaminato entrambi gli array

e non esiste ancora una coppia di indici per cui un elemento con indice "i" è il doppio dell'elemento con indice "j".

- se il doppio dell'elemento con indice "i" è maggiore di "j", occorre incrementare "j"

- se il doppio dell'elemento con indice "i" è minore di "j", occorre incrementare "i"

Trovati gli indici, si ritorna la coppia (se esiste) altrimenti ritorna una coppia di zeri.

È facile vedere che si mantiene l'invariante $\forall i' \in [0, i). \forall j' \in [0, j). 2 * A[i'] \neq A[j']$. Da questo la correttezza segue immediatamente. Il costo è lineare (il numero di iterazioni è pari ad al più $2n$, ed ogni iterazione ha costo costante, quindi $T(n) = O(n)$).

Esercizio 8 Si dica che un array `A[1..n]` è bi-ordinato se esiste un indice `k` tale che `A[1..k]` è ordinato in senso crescente e `A[k..n]` è ordinato in senso decrescente. Realizzare un algoritmo

`top(A,n)` che dato in input un array `A` bi-ordinato con elementi distinti ne determina il valore massimo. Ad esempio su `A = (1 2 3 14 13 4)` restituisce 14. Valutarne la complessità.

Soluzione:

`top(A,n)` Lo passiamo ad una funzione per calcolo ricorsivo.
`top-rec(A,i,j)`

```
top_rec(A,i,j)
  if (i=j)
    return A[i]
  else
    q = (i+j)/2
    if A[q] <= A[q+1]
      return top_rec(q+1,j)
    else
      return top_rec(i,q)
```

Bi-ordinato:
 partendo da un array con elementi distinti,
 rispetto ad un indice "k", la prima metà è crescente
 e la seconda è decrescente.

L'idea è quella della scansione logaritmica in due metà
 di merge-sort (complessità dell'algoritmo).

Quindi:

- usiamo la scansione in due metà, trovando "q", elemento medio
- se l'elemento della prima metà è minore dell'elemento sulla seconda metà, allora ricorriamo spostando un indice avanti verso "j" (cioè, si ha un elemento crescente sulla prima metà)
- se l'elemento della prima metà è maggiore dell'elemento sulla seconda metà, allora l'indice attuale è quello i-esimo e ci spostiamo verso "q". (cioè, si ha un elemento decrescente sulla seconda metà)

Tutto questo assicura che ci spostiamo solo usando "q" e, quando troviamo un elemento uguale a sé stesso, abbiamo trovato che tutta la prima metà è crescente e tutta la seconda metà è decrescente. Quindi, una volta trovato l'indice per cui in una delle due metà non si ha più crescita o decrescenza (non sono più diverse), si è trovato l'indice utile.

Esercizio 9 Scrivere una funzione `perm(m,n)` che dati due numeri interi m e n , maggiori o uguali di 0, verifica se uno dei due numeri può essere ottenuto permutando le cifre dell'altro. Ad esempio 915 e 159 sono uno la permutazione dell'altro mentre 911 e 19 no. Attenzione al ruolo degli zeri, ad es. 150 è la permutazione di 51, dato che $51 = 051$. Valutarne la complessità. (*Suggerimento:* Il counting sort può fornire una ispirazione.)

Soluzione:

perm: dati due interi positivi m e n verifica se uno e' la
 # permutazione dell'altro.

```
perm(m,n):
  # use an array for counting the difference in the number of digits
  # of the two numbers. Note that we do not need to count the 0's as
  # different number of 0's can be compensated by putting 0's in front
```

```
# D[i] = difference count for cypher i
for i=1 to 9
  D[i] = 0
```

```
while (m > 0)
  # compute the last digit of m
  d = m mod 10
  if (d > 0)
    D[d]++
  m = m div 10
```

```
while (n > 0)
  # compute the last digit of m
  d = n mod 10
  if (d > 0)
    D[d]--
  n=n div 10
```

Permutazione: stessi elementi presenti (anche considerando gli zeri)
 di due stringhe, ma in ordine diverso tra di loro.

In pratica, essendo stringhe numeriche:
 - inizializziamo tutte le posizioni pari ai numeri presenti per D a 0
 (quindi, per contare dentro D tutti i numeri presenti tra 0 e 9)
 - finché $m > 0$, prendiamo il numero "m" e se questo è positivo, significa che esiste una nuova cifra di "m" di cui esaminare la permutazione e contiamo questa cifra.
 - finché $n > 0$, prendiamo il numero "n" e se questo è positivo, significa che esiste una nuova cifra di "n" di cui esaminare la permutazione e sottraiamo questa cifra.

L'idea è come "diff" di qualche esercizio prima:
 - sommare per tutte le cifre di "m"
 - sottrarre per tutte le cifre di "n"
 - se D ha 0 per tutte le cifre, siamo in presenza di una permutazione e si ritorna un indice > 9 (perché per tutti i numeri possibili, si ha una permutazione che li coinvolge tutti interamente).

```
# returns True if D contains all zeros
i = 1
while (i<=9) and (D[i]==0)
    i++

return (i>9)
```

I primi due cicli while hanno complessità $\Theta(\log m)$ e $\Theta(\log n)$, mentre l'ultimo è $\Theta(9) = \Theta(1)$. Quindi la complessità è $\Theta(\log m + \log n) = \Theta(\log \max\{m, n\})$.

4 Heap

Domanda 22 Scrivere una funzione $sndmin(A)$ che dato in input un array A organizzato a min-heap, restituisce il successore della radice, ovvero il minimo elemento dello heap maggiore della radice. Se un tale elemento non esiste genera un errore. Assumere che A sia non vuoto e gli elementi in A siano tutti distinti.

Soluzione: È sufficiente ricordare che ogni nodo di un min-heap è minore o uguale ai discendenti per dedurre che il successore della radice, se esiste, è uno dei due figli $A[2]$ o $A[3]$.

```
sndmin(A)
  if A.heapsize > 2
    return min (A[2], A[3])
  else if A.heapsize = 2
    return A[2]
  else
    return error
```

Esercizio banale:

serve il minimo elemento dello heap maggiore della radice, ma abbiamo un min heap, quindi vuol dire che:
 - se lo heap ha più di due elementi, è il minimo tra il secondo ed il terzo
 - se lo heap ha esattamente due elementi, è il secondo elemento
 - altrimenti non esiste

Complessità lineare.

Correttezza: Fattuale dalle osservazioni fatte.

Domanda 23 Scrivere una funzione $IsMaxHeap(A)$ che dato in input un array di interi $A[1..n]$ che verifica se A è organizzato a max-heap e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: Versione ricorsiva

```
IsMaxHeap(A,i,j) // verifica se l'array A[i,j] e' un max-heap
                  (o meglio, se contiene un max-heap radicato in i,
                  dato che al passo ricorsivo non tutto [i,j]
                  conterra' il sottoalbero scelto)
```

```
  l = 2*i
  r = 2*i+1
  if l<j
    leftOk = A[i] >= A[l] and IsMaxHeap(l, j)
  else
    leftOk = true

  if r<j
    rightOk = A[i] >= A[r] and IsMaxHeap(r, j)
  else
    rightOk = true
```

Per capire se è un max heap, si crea il nodo figlio sx e il nodo figlio dx.

Se il nodo sinistro è minore del nodo ad indice "j", allora si rispetta la proprietà di Max Heap avendo il nodo parent "i" maggiore del nodo attuale e ristabilendo la proprietà di minore a sinistra, avendo che "i" è il nuovo indice "i" (e quindi ricorsivamente, il figlio sinistro è più piccolo del parent)

Se il nodo sinistro è maggiore del nodo ad indice "j", allora si rispetta la proprietà di Max Heap avendo il nodo parent "r" maggiore del nodo attuale e ristabilendo la proprietà di minore a destra, avendo che "r" è il nuovo indice "i". (e quindi ricorsivamente, il figlio destro è più piccolo del parent)

return leftOk and rightOk Se la proprietà vale come discusso su tutti i nodi, si ritorna un booleano che è true.

Per quanto riguarda la complessità si osservi che $T(n) = c + 2T(n/2)$ per un'opportuna costante c e quindi, utilizzando il master theorem, si deduce che la complessità è $O(n)$.

Versione iterativa

```
IsMaxHeap(A,n) // verifica se l'array A[1..n] e' un max-heap
  i=2
  while (i <= n) and (A[i] <= A[i/2])
    i++
  return (i==n+1)
```

Iterativamente, vediamo max heap linearmente e finché vale la proprietà di max/heap, quindi il nodo parent maggiore/uguale del nodo attuale, si incrementa "i" e si ritorna l'indice a cui ci si è fermati.

Domanda 24 Fornire lo pseudocodice della procedura `HeapExtractMin(A)` per estrarre il minimo elemento da un min-heap A , realizzato tramite un array come visto a lezione (la dimensione corrente dello heap è data dall'attributo $A.heapsize$). Discutere la complessità.

Soluzione:

```

Heap-Extract-Min(A)    // A e' un min-heap
    if A.heapsize < 1
        error \underflow"
    else
        min = A[1]
        A[1] = A[A.heapsize]
        A.heapsize = A.heapsize - 1
        Min-Heapify(A,1)
        return min

```

L'estrazione del minimo è praticamente come l'estrazione del massimo vista a lezione; quindi, si considera una size dello heap > 1 .
Se questo accade, il minimo è la radice dello heap e continuo ricorsivamente a scambiare la radice con l'ultimo elemento, diminuendo la size e ristabilendo volta per volta le proprietà di minheap, ritornando il minimo.

```

MinHeapify(A,i)
    l = 2i, r = 2i+1
    min = i
    if l <= A.heapsize and A[l] < A[m]
        min = l
    if r <= A.heapsize and A[r] < A[m]
        min = r
    if min <> i
        A[i] <-> A[m]
        MinHeapify(A,m)

```

Stabilendo figlio sx e figlio dx, mettiamo che il minimo sia ad indice "i" e continuiamo a vedere i figli come minore del nodo padre e:
- se vale questa proprietà a sx e il nodo è $\leq$ alla size dello heap, il minimo è l'elemento a sinistra "l"
- se vale questa proprietà a dx e il nodo è $\leq$ alla size dello heap, il minimo è l'elemento a destra "r"
Se il minimo non è il parent (continuando ad iterare sarà il nodo radice), allora continuiamo a chiamare la procedura `MinHeapify`.

Domanda 25 Dare la definizione di max-heap. Dato un array $A[1..12]$ con sequenza di elementi [60, 6, 45, 95, 30, 24, 15, 80, 19, 38, 21, 70] si indichi il risultato della procedura `BuildMaxHeap` applicata ad A . Si descriva sinteticamente come si procede per arrivare al risultato.

Soluzione: L'array risultante (si vedano le procedure sul libro) è:

[95, 80, 70, 60, 38, 45, 15, 6, 19, 30, 21, 24]

Lo si inserisca su <https://visualgo.net/en/heap> nella sezione Create e si vede tutto.

5 Alberi e ricorsione

Domanda 26 Dato un albero nel quale i nodi contengono una chiave, si definisca *costo* di un cammino dalla radice ad una foglia, come la somma delle chiavi dei nodi che compaiono nel cammino. Scrivere una funzione `MaxPath(T)` che opera nel modo seguente. Prende in input un albero binario T , con radice $T.root$, e nodi x che hanno come campi $x.k$, $x.l$ e $x.r$, ovvero una chiave, il puntatore al figlio sinistro e destro, rispettivamente. Resituisce la il costo del cammino di costo massimo dalla radice ad una foglia. Valutarne la complessità.

Soluzione:

```
MaxPath(x)
  if x==nil
    return 0
  elseif x.l = nil
    return x.key + MaxPath(x.r)
  elseif x.r = nil
    return x.key + MaxPath(x.l)
  else
    return x.key + max { MaxPath(x.l), MaxPath(x.r) }
```

Costo: percorrere un albero binario nella sua interezza.
 Se l'albero è nullo --> costo nullo
 Se l'albero sx è nullo --> costo della chiave + ricorsione a dx
 Se l'albero dx è nullo --> costo della chiave + ricorsione a sx
 Se ho entrambi i sottoalberi nulli -->
 costo della chiave + max {ricorsione a dx - ricorsione a sx}
 Complessità: $O(h)$, dove "h" è l'altezza dell'albero

Domanda 27 Realizzare una procedura `Level(T)` che dato un albero binario T , con radice $T.root$, e nodi x con campi $x.left$, $x.right$ e $x.key$, rispettivamente figlio destro, figlio sinistro e chiave intera, ritorna il numero di nodi per i quali la chiave $x.key$ è minore o uguale al livello del nodo (la radice ha livello 0, i suoi figli livello 1 e così via). Valutare la complessità.

Soluzione:

```
// ritorna il numero di nodi y del sottoalbero radicato in x, tali che
//      y.key <= livello
```

```
Level(x, level)
  if x == nil
    return 0
  else
    left = Level(x.left, level+1)
    right = Level(x.right, level+1)
    if x.key <= level
      return left + right + 1
    else
      return left + right
```

Esaminiamo in profondità l'albero:
 - se il nodo è nullo, ritorno 0 (in quanto non ci sono nodi la cui chiave è $\leq$ al livello nodo)
 - se il nodo non è nullo, allora,
 ricorro sul sottoalbero sx e aumento il livello (conto i nodi radicati a sx in "left")
 &
 ricorro sul sottoalbero dx e aumento il livello (conto i nodi radicati a dx in "right").
 Fatto tutto questo:
 - se la chiave è $\leq$ a l livello esaminato, ritorno tutti i nodi esaminati a sx e dx più la chiave stessa (facente parte dell'analisi in profondità)
 - altrimenti, ritorno il livello di sx e il livello di dx

```
// chiamata di base
Level(T)
  return Level (T.root, 0)
```

Si tratta di una visita dell'albero, quindi la complessità è lineare $\Theta(n)$.

Domanda 28 Sia T un albero binario i cui nodi x hanno i campi $x.left$, $x.right$, $x.key$. L'albero si dice un *sum-heap* se per ogni nodo x , la chiave di x è maggiore o uguale sia alla somma delle chiavi nel sottoalbero sinistro che alla somma delle chiavi nel sottoalbero destro.

Scrivere una funzione `IsSumHeap(T)` che dato in input un albero T verifica se T è un sum-heap e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: La soluzione può essere

```
IsSumHeap(T)                                Chiamata di base per la procedura ricorsiva
    return IsSumHeap-rec(T.root)
```

```
IsSumHeap-rec(x) // verifica se l'albero radicato in x e' un sum-heap
                  e ritorna true/false e la somma delle chiavi nel
                  sottoalbero radicato in x
```

```
if x = nil
    return true, 0
else
    isSumHeapL, sumL = IsSumHeap-rec(x.left)
    isSumHeapR, sumR = IsSumHeap-rec(x.right)
    return (x.key >= sumL) and (x.key >= sumR),
           x.key + sumL + sumR
```

Sum-heap: mantiene la proprietà di heap (parent $\geq$ left-right) e la chiave di "x" è $\geq$ alla somma delle chiavi a sx e dx.

Quindi:

- ricorro a sx, tenendo un booleano di controllo che verifica se la somma dei nodi continua a rispettare la proprietà di heap
- ricorro a dx, tenendo un booleano di controllo che verifica se la somma dei nodi continua a rispettare la proprietà di heap
- ritorna vero se la chiave di "x" è $\geq$ della somma dei nodi di sx e di dx e ritorna la somma di tutti i nodi.

Complessità logaritmica.

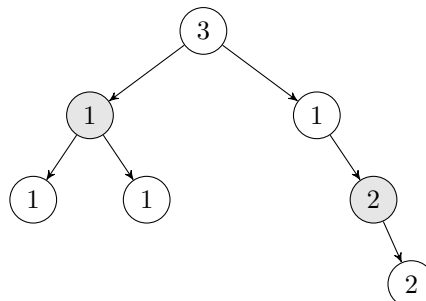
Per quanto riguarda la complessità, se l'albero è bilanciato, $T(n) = c + 2T(n/2)$ per un'opportuna costante c e quindi, utilizzando il master theorem, si deduce che la complessità è $\Theta(n)$. Se l'albero non è bilanciato, si può osservare che si tratta di una visita (costo lineare) o più precisamente scrivere la ricorrenza

$$T(n) = T(k) + T(n - k - 1) + c$$

e provare, per sostituzione, che $T(n) = an + b$ è soluzione per opportune costanti a, b .

Esercizio 10 Un nodo x di un albero binario T si dice *fair* se la somma delle chiavi nel cammino che conduce dalla radice dell'albero al nodo x (escluso) coincide con la somma delle chiavi nel sottoalbero di radice x (con x incluso). Realizzare un algoritmo ricorsivo `printFair(T)` che dato un albero T stampa tutti i suoi nodi fair. Supporre che ogni nodo abbia i campi $x.left$, $x.right$, $x.p$, $x.key$. Valutare la complessità dell'algoritmo.

Un esempio: i nodi grigi sono fair



Soluzione: L'algoritmo può essere il seguente:

```
printFair(x,path)    // x = node of the tree
                     // path=sum of the keys in the path from the root to x
```

```

// Action: print the fair nodes and
// returns the sum of the keys in the subtree

if (x == nil)
    return 0

left  = printFair(x.l, path + x.key)
right = printFair(x.r, path + x.key)
sumTree = left + right + x.key
if (path == sumTree)
    print x
return sumTree

```

Fair: la somma delle chiavi nel cammino dalla radice è la somma delle chiavi nel sottoalbero di radice "x" (quindi, escluso il nodo stesso, la somma corrisponde esattamente alla chiave nei sottoalberi)

Similmente a prima:

- ricorro a sx e sommo i nodi fair a sx (tutte le chiavi) e la chiave stessa, idealmente anche stampando i nodi a sx
- ricorro a dx e sommo i nodi fair a dx (tutte le chiavi) e la chiave stessa, idealmente anche stampando i nodi a dx

Se la somma corrisponde a tutti i nodi, ritorno in una sola variabile tutti i nodi fair.

e viene chiamato come `printFair(T.root, 0)`.

Si tratta di una visita, quindi con costo $O(n)$ (più precisamente ottenibile con il master theorem come soluzione della ricorrenza $T(n) = 2T(n/2) + c$).

Esercizio 11 Sia dato un albero i cui nodi contengono una chiave intera $x.key$, oltre ai campi $x.l$, $x.r$ e $x.p$ che rappresentano rispettivamente il figlio sinistro, il figlio destro e il padre. Si definisce *grado di squilibrio* di un nodo il valore assoluto della differenza tra la somma delle chiavi nei nodi foglia del sottoalbero sinistro e la somma delle chiavi dei nodi foglia del sottoalbero destro. Il grado di squilibrio di un albero è il massimo grado di squilibrio dei suoi nodi.

Fornire lo pseudocodice di una funzione `sdegree(T)` che calcola il grado di squilibrio dell'albero T (si possono utilizzare funzioni ricorsive di supporto). Valutare la complessità della funzione.

Soluzione:

```

// computes the sum of the leaf nodes of the subtree and the sdegree
// for the node x (returns two values)

sdegree(x)

if (x == nil)
    sum = 0
    degree = 0
elif (x.left == nil) and (x.right == nil)    # leaf
    sum = x.key
    degree = 0
else
    suml, degreeel = sdegree(x.l)
    sumr, degreeer = sdegree(x.r)
    sum = suml + sumr
    degree = max { degreeel, degreeer, abs(suml - sumr) }

return sum, degree

```

Squilibrio:
Differenza tra la somma delle chiavi delle foglie a sx e la somma delle chiavi delle foglie a dx.

- Se la radice è nulla, la somma e lo squilibrio sono 0.
- Se entrambi i sottoalberi sono nulli, la somma equivale alla chiave di "x" ed il grado di squilibrio è pari a 0.
- Se i sottoalberi non sono nulli, ricorro a sx e dx calcolando con la ricorsione il grado di squilibrio e la somma dei nodi, tali da restituire in "degree" il max di tutto lo squilibrio dei vari nodi.

Esercizio 12 Si consideri un albero binario T , i cui nodi x hanno i campi $x.l$, $x.r$, $x.p$ che rappresentano il figlio sinistro, il figlio destro e il padre, rispettivamente. Un *cammino* è una

sequenza di nodi $x_0, x_1, \dots, x_n$ tale che per ogni $i = 1, \dots, n$ vale $x_{i+1}.p = x_i$. Il cammino è detto *terminabile* se $x_n.l = \text{nil}$ oppure $x_n.r = \text{nil}$. Diciamo che l'albero è 1-bilanciato se tutti i cammini terminabili dalla radice hanno lunghezze che differiscono al più di 1. Scrivere una funzione `bal1(T)` che dato in input l'albero T verifica se è 1-bilanciato e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: La soluzione può basarsi su di una funzione ricorsiva che calcola per un nodo x la lunghezza massima e minima dei cammini terminabili da x

```
bal1(T)    // verifica se l'albero radicato in x e' 1-bilanciato,
           // ovvero i cammini dalla radice terminati da nil hanno
           // lunghezze che differiscono al piu di 1

           // si basa su di una funzione ricorsica che calcola la
           // lunghezza minima e massima dei cammini.
min,max = bal1rec(T.root)  Vogliamo verificare se l'albero è 1-bilanciato (quindi, i cammini dalle radici
                           alle foglie differiscono al max di 1).
                           Quindi, ricorriamo su tutti i nodi e ritorniamo due variabili (min-max) la cui differenza
                           permette di intuire il bilanciamento degli alberi. Infatti:
                           - ricorriamo a sinistra e salviamo il nodo più a sx e il nodo più a dx (min e max) del
                           sottoalbero sx
                           - ricorriamo a destra e salviamo il nodo più a dx e il nodo più a dx (min e max) del
                           sottoalbero dx
                           Assieme al nodo chiave, sommiamo 1 al percorso a sx e al percorso a dx, così
                           prendiamo chiave e tutti i nodi.
return max-min <= 1

bal1rec(x)
if x = nil
    return (0,0)
else
    (left_min,left_max) = bal1(x.left)
    (right_min,r_right_max) = bal1(x.right)
    return max(left_min, right_min) + 1, max(left_max,right_max) + 1
```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n-k-1)$ per un'opportuna costante c e quindi, la complessità è $O(n)$. Il caso peggiore di esame di tutto l'albero, in complessità, è proprio lineare (vado da tutte le parti, quasi linearmente come su un array)

Esercizio 13 Sia T un albero binario i cui nodi x hanno i campi $x.left$, $x.right$, $x.key$. L'albero si dice k -bounded, per un certo valore k , se per ogni nodo x la somma delle chiavi lungo ciascun cammino da x ad una foglia è minore o uguale a k .

Scrivere una funzione `k-bound(T,k)` che dato in input un albero T e un valore k verifica se T è k -bounded e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione:

```
k-bound(T,k)
    return k-bound-rec(T.root)

k-bound-rec(x,k)    // Verifica se l'albero radicato in x e' k-bounded
                    // Ritorna true/false e la somma delle chiavi in un
                    // cammino da x alla radice con somma massima

if x = nil
    return true, 0
else
    if x.left = nil
        isKBound, max = k-bound-rec(x.right,k)
        max = max + x.key
    else if x.right = nil
        isKBound, max = k-bound-rec(x.left,k)
        max = max + x.key
```

K-bounded: la somma delle chiavi in una cammino da "x" ad una foglia è $\leq k$
 Quindi, come sempre:
 - albero nullo, è k-bounded e la somma delle chiavi è 0
 - sottoalbero sx nullo, verifico se a dx è k-bounded e max è la somma dei nodi a dx, la cui somma la chiave
 - sottoalbero dx nullo, verifico se a sx è k-bounded e max è la somma dei nodi a sx, la cui somma la chiave

```

else
    isKBoundL, maxL = k-bound-rec(x.left,k)
    isKBoundR, maxR = k-bound-rec(x.right,k)
    max = max { maxR, maxL } + x.key
    isKBound = isKBoundL and isKBoundR and max <= k

return isKBound, max

```

Altrimenti, tutti i due i sottoalberi sono validi, verifico da entrambe le parti se sono k-bounded e sommiamo in max tutti i nodi a sx e a dx più la chiave, ritornando il valore booleano e la somma dei nodi.

La complessità è $\Theta(n)$. Il caso peggiore di esame di tutto l'albero, in complessità, è proprio lineare (vado da tutte le parti, quasi linearmente come su un array)

Domanda 29 Scrivere una funzione *complete*(*T*) che dato in input un albero binario verifica se è completo (ovvero ogni nodo interno ha due figli e tutte le foglie hanno la stessa distanza dalla radice).

Soluzione:

```

complete(x) // verifica se l'albero radicato in x e' completo: ritorna
              // l'altezza dell'albero in caso positivo e -1 in caso negativo
if x = nil
    return 0
else
    countl = complete(x.left)
    countr = complete(x.right)

    if (countr == -1) or (countl == -1) or (countl <> countr)
        return -1
    else
        return countl+1

```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n-k-1)$ per un'opportuna costante c e quindi, la complessità è $O(n)$.

Domanda 30 Scrivere una funzione *diff*(*T*) che dato in input un albero binario *T* determina la massima differenza di lunghezza tra due cammini che vanno dalla radice ad un sottoalbero vuoto. Ad esempio sull'albero ottenuto inserendo 1, 2 e 3 produce 2, su quello ottenuto inserendo 2, 1, 3 produce 0. Valutarne la complessità.

Soluzione: Il programma si basa su una funzione ricorsiva *diff-rec*(*x*) che restituisce il la lunghezza del minimo e massimo cammino per il sottoalbero radicato in *x*

```

diff(T)
    min,max = diffRec(T.root)
    return

diff-rec(x)
    if x = nil
        return 0,0
    else
        minl, maxl = diff-rec(x.left)
        minr, maxr = diff-rec(x.right)
        return min(minl,minr)+1, max(maxl,maxr)+1

```

Diff = ritorna la massima differenza tra i cammini dalla radice ad un sottoalbero vuoto. Quindi, passiamo tutto ad una funzione ricorsiva a cui passiamo la radice e usiamo due parametri, "min" e "max" sono il nodo più a sx e più a dx per capire qual è il giusto cammino.

Se il nodo "x" è nullo, allora si ritorna la differenza nulla, altrimenti, ricorriamo a sx e a dx e troviamo il min e max da entrambe le parti e sottraendo a "diff" (la radice) e i nodi incontrati. A quel punto, ritorniamo il minimo e il massimo tra i cammini esaminati + 1 (perché ci sta anche il nodo con "x"), avendo in questo modo l'intervallo differenza tra i nodi di sx e dx anche incompleti.

Per quanto riguarda la complessità si osservi che si tratta di una visita dell'albero, quindi $\Theta(n)$.

Esercizio 1 (8 punti) Realizzare una funzione booleana `triple(A)` che dato un array di numeri $A[1..n]$ verifica se esistono tre indici (non necessariamente diversi) i, j e k , tali che $A[i] + A[j] = A[k]$ e restituisce un corrispondente valore booleano. Ad esempio per $[85, 10, 29, 93, 95]$ restituisce true, dato che $85 + 10 = 95$, mentre per $[70, 45, 83, 21, 68]$ restituisce false. Si suggerisce di ordinare l'array. Valutare la complessità.

Soluzione: Assumiamo inizialmente che l'array A sia ordinato. In questo caso, si può cercare, per ogni indice k , una coppia (i, j) tale che $A[k] = A[i] + A[j]$.

assume $A[1..n]$ ordinato

cerca tre indici i, j, k tali che $A[i] + A[j] = A[k]$

```
Triple(A)
  n = A.length
  found = false
  k=1
  while (not found) and (k <= n)
    # cerca se ci sono due indici tali che  $A[i] + A[j] = A[k]$ 
    i=1
    j=n
    # invariante:
    #   per ogni  $i' < i$ , e  $j'$  qualunque  $A[i'] + A[j'] < A[k]$ 
    #   per ogni  $j' > j$ , e  $i'$  qualunque  $A[i'] + A[j'] < A[k]$ 
    while (i <= j) and (A[i] + A[j] < A[k])
      if A[i] + A[j] < A[k]
        i = i+1
      else
        j = j-1
    found = i <= j

    k=k+1

  return found
```

Triple: si cerca se esiste un trio di indici per cui un elemento con indice "k" è dato dalla somma degli elementi con indice "i" e quello con indice "j".

Quindi:

- inizializzo gli indici e trovo la lunghezza dell'array
- finché non abbiamo ancora trovato Triple e l'indice "k" di analisi è minore della lunghezza dell'array
- finché l'indice "i" è minore uguale a "j" e non abbiamo ancora
- per cui un elemento con indice "k" è dato dalla somma degli elementi con indice "i" e quello con indice "j".
- se la somma degli elementi con indici "i" e "j" è minore di quello con indice k, aumentiamo "i"
- se la somma degli elementi con indici "i" e "j" è maggiore di quello con indice k, diminuiamo "j"

La variabile "found" recupera il valore degli indici "i" e "j", capendo se è stato trovato o meno quello che cerchiamo; similmente, alla fine, recuperiamo anche il valore di "k" e lo ritorniamo.

(Idea correttezza)

Sapendo quindi che nei cicli sono diversi, se $i \leq j$ sappiamo che una coppia di elementi è minore di "k" se " $i < j$ " invece se $i > j$ sappiamo che una coppia di elementi è maggiore di "k" se " $i > j$ ".

La complessità è $O(n^2)$: infatti, il ciclo interno, ad ogni iterazione incrementa i o decrementa j , quindi il numero massimo di iterazioni è n , e viene ripetuto al più n volte per effetto del ciclo esterno.

Se l'array non è ordinato, si può ordinarlo preventivamente, cosa che si può fare con costo $O(n \log n)$ e quindi senza alterare la complessità asintotica complessiva.

Esercizio 1 (9 punti) Realizzare una funzione `avgTree(T)` che dato un albero binario T con chiavi numeriche, verifica se, per ogni nodo che abbia discendenti, la chiave del nodo è maggiore o uguale della media delle chiavi dei discendenti e ritorna conseguentemente un valore booleano (la radice dell'albero è $T.root$ e ogni nodo x ha i campi $x.left$, $x.right$ e $x.key$).

Soluzione: L'idea è quella di effettuare una visita dell'albero, in modo ricorsivo, raccogliendo per ogni sottoalbero le seguenti informazioni:

- se è soddisfatta la condizione sulla media richiesta
- la somma delle chiavi
- il numero dei nodi.

Quanto mi trovo su di un nodo, analizzo i sottoalberi sinistro e destro. Quindi, avendo a disposizione la somma delle chiavi ed il numero dei nodi per i due sottoalberi posso verificare la condizione sulla media per il nodo in esame. Lo pseudocodice della soluzione può essere il seguente:

```

avgTree(T)
    v, n, s = avgTreeRec(T.root)
    return v

```

Avgtree = verifica se la chiave del nodo è $\geq$ alla media delle chiavi dei discendenti e ritorna un booleano, il numero dei discendenti e la somma delle chiavi
Solita chiamata ricorsiva base, passando la radice

```

# avgTreeRec(x):
# verifica se il sottoalbero radicato in x e' un avgTree e ritorna tre valori:
# - un booleano,
# - il numero dei discendenti
# - la somma delle loro chiavi

```

```

avgTreeRec(x)

```

```

if x = nil
    return true, 0, 0

```

Se l'albero è nullo, si ritorna vero e tutto a 0

```

else
    # ispeziono i sottoalberi sinistro e destro
    vl, nl, sl = avgTreeRec(x.left)
    vr, nr, sr = avgTreeRec(x.right)

```

```

# se non ci sono discendenti (nl+nr =0) oppure la chiave del nodo in
# esame e' maggiore o uguale della media delle chiavi dei discendenti
# la condizione e' soddisfatta
v = (nl+nr =0) or (x.key >= (sl+sr)/(nl+nr))

```

Il flag verifica se ci sono discendenti (la somma dei nodi dovrebbe essere diversa da 0, ma se uguale a 0, non ci sono discendenti e verifica se la media (somma dei nodi / numero dei nodi) è $\leq$ a x.key

```

# il numero di nodi dell'albero radicato in x e' dato dalla somma del
# del numero di nodi nel sottoalbero dx e nel sottoalbero sx piu' uno
# (il nodo x stesso)
n = nl + nr + 1

```

Somma dei nodi di sx e dx più la chiave

```

# la somma delle chiavi dell'albero radicato in x e' dato dalla somma del
# delle chiavi nel sottoalbero dx e in quello sx piu' x.key
s = sl + sr + x.key

```

Somma delle chiavi di sx e dx più la chiave

```

return v, n, s

```

Si tratta di una visita e quindi la complessità è $\Theta(n)$. Esamino tutto l'albero e si ha complessità lineare.

Domanda A (8 punti) Definire formalmente la classe $\Theta(g(n))$. Dimostrare le seguenti affermazioni o fornire un controesempio:

- se $f(n), f'(n) \in \Theta(g(n))$ allora $f(n) + f'(n) \in \Theta(g(n))$;
- $f(n), f'(n) \in \Theta(g(n))$ allora $f(n) * f'(n) \in \Theta(g(n))$;

Soluzione: Per la definizione di $\Theta(f(n))$, consultare il libro.

Per (i), siano $f(n), f'(n) \in \Theta(g(n))$. Per definizione, esistono $c_1, c_2 > 0$ e n_0 tali che per ogni $n \geq n_0$ vale:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

e, analogamente esistono $c'_1, c'_2 > 0$ e n'_0 tali che per ogni $n \geq n'_0$ vale:

$$0 \leq c'_1 g(n) \leq f'(n) \leq c'_2 g(n)$$

contando che $f(n)$ è dato dal punto prima

Quindi, per ogni $n \geq \max\{n_0, n'_0\}$ abbiamo:

Metto insieme i pezzi singolarmente, sotto forma di somma (punto 1) e disuguaglianze

$$(c_1 + c'_1)g(n) = c_1 g(n) + c'_1 g(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2)g(n)$$

dato che $c_1 + c'_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Per la parte (ii), l'affermazione è falsa. Basta considerare $f(n) = f'(n) = n \in \Theta(n)$, ma ovviamente $f(n) * f'(n) = n^2 \notin \Theta(n)$.

Esercizio 1 (9 punti) Realizzare una funzione booleana `checkSum(A,B,C,n)` che dati tre array $A[1..n]$, $B[1..n]$ e $C[1..n]$ con valori numerici, verifica se esistono tre indici i, j, k con $1 \leq i, j, k \leq n$ tali che $A[i] + B[j] = C[k]$. Valutarne la complessità.

Soluzione: L'idea consiste nell'ordinare i vettori B e C e a questo punto verificare per ogni elemento $A[i]$ del primo array se ne esiste uno del secondo $B[j]$ la cui somma produce un elemento $C[k]$ del terzo. Il fatto che gli array B e C siano ordinati permette di scorrerli una sola volta per ogni elemento del primo array.

```
checkSum(A,B,C,n):
    # ordina B e C
    Sort(B)
    Sort(C)

    i = 1
    found = false

    while (i <= n) and not found
        # per ogni A[i] verifica se per qualche elemento B[j] vale che
        # A[i]+B[j]=C[k] scorrendo B e C

        j = k = 1
        while (j <= n) and (k <= n) and (A[i] + B[j] <> C[k])
            if (A[i] + B[j] < C[k]):
                j++
            else
                k++
        if (j <= n and k <= n)
            found = true
        else
            i++

    return found
```

Checksum: verifica se esistono tre indici "i", "j", "k" tale che $A[i] + B[j] = C[k]$.
Inizializzo i tre indici e continuo finché "j" e "k" sono $\leq n$ e finché non abbiamo la somma Checksum.
Quindi:
- se la somma è minore di $C[k]$, allora incrementiamo "j"
- se la somma è maggiore di $C[k]$, allora incrementiamo "k"
- se entrambi gli indici sono $\leq n$, abbiamo trovato il valore, altrimenti incrementiamo l'indice "i".

L'invariante del ciclo esterno è che se `found` è falsa, allora per ogni $i' < i$, e qualunque siano j' e k' , vale $A[i'] + B[j'] \neq C[k']$. Se invece `found` è vera, allora $i, j, n \leq n$ e $A[i] + B[j] = C[k]$.

Per il ciclo interno, abbiamo che per ogni $j' < j$ e per ogni $k' < k$ vale $A[i] + B[j'] \neq C[k']$. Per il mantenimento dell'invariante, unico fatto da osservare è che quando $A[i] + B[j] < C[k]$ posso incrementare j mantenendo l'invariante. Infatti so che per ogni $j' < j$, dato che B è ordinato, vale $B[j'] \leq B[j]$ e quindi $A[i] + B[j'] \leq A[i] + B[j] < C[k]$. Il ragionamento quando $A[i] + B[j] > C[k]$ e incremento k è duale.

La complessità è $O(n^2)$ dato che ho due cicli annidati. In quello esterno, quando non esco i aumenta, quindi il numero di iterazioni è limitato da n . In quello interno, quando non esco, j oppure k aumentano, quindi il numero di iterazioni è limitato da $2n$. Complessivamente, nel caso peggiore, ho $n * 2n = 2n^2 = \Theta(n^2)$ iterazioni, ciascuna di costo costante.

A questo si somma il costo degli ordinamenti, che però posso realizzare in tempo $\Theta(n \log n)$ di modo che non incida sulla complessità asintotica.

Esercizio 1 (9 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , in tempo costante, la media dei valori memorizzati nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `avg(x)` che restituisce la media delle chiavi nel sottoalbero radicato in x e la procedura di inserimento di un nodo `insert(T,z)`.

Soluzione: L'idea è quella di arricchire la struttura facendo in modo che ogni nodo x , oltre ai campi usuali, abbia il campo $x.sum$ che contiene la somma delle chiavi per i nodi del sottoalbero radicato in x e $x.size$ che contiene il numero dei nodi in tale sottoalbero.

A questo punto è immediato realizzare la funzione `avg(x)` di tempo costante

```
avg(x)
  if x <> nil
    return x.sum/x.size
  else
    error
```

Per salvare la media delle chiavi, ci basta solo inserire un campo "sum" per la somma delle chiavi e "size" il numero dei nodi del sottoalbero, tale che la media sia la somma diviso il numero dei nodi.

Invece per l'inserimento, si modifica la procedura standard, che discende l'albero dalla radice fino alla posizione in cui inserire il nuovo nodo, facendo in modo che tutti i nodi attraversati e che quindi conterranno nel sottoalbero il nuovo nodo, abbiano i campi `sum` e `size` aggiornati.

```
Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x
    x.size++
    x.sum = x.sum + z.key
    if (z.key < x.key)
      x = x.left
    else
      x = x.right

  z.p = y
  if (y <> nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  z.sum = z.key
  z.size = 1
  z.left = z.right = nil
```

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
exp(n)
  if n = 0
    return 0
  else
    return exp(n-1) + exp(n-1) + 1
```

Qua ci danno una funzione e dobbiamo dimostrare induttivamente che funziona.
Quindi:
caso base --> mettiamo 0 come indice nella chiamata ricorsiva
caso induttivo --> si crea la relazione di ricorrenza, composta da 2 somme ricorsive su (n-1) e una costante (in questo caso 1).

Dimostrare induttivamente che la funzione calcola il valore $2^n - 1$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Omega(2^n)$. È anche $O(2^n)$? Motivare le risposte.

Per dimostrare il secondo pezzo, quindi Omega oppure O grande di 2^n , allora usiamo il metodo di sostituzione, dimostrando che vale per una generica costante "d".

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione su n . Se $n = 0$, la funzione ritorna 0 che è in effetti $2^0 - 1 = 1 - 1 = 0$. Se $n > 0$, allora per ipotesi induttiva, $exp(n-1) = 2^{n-1} - 1$. Quindi $exp(n) = exp(n-1) + exp(n-1) + 1 = 2^{n-1} - 1 + 2^{n-1} - 1 + 1 = 2 \cdot 2^{n-1} - 1 = 2^n - 1$, come desiderato.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, si effettuano due chiamate ricorsive con argomento $n - 1$ e tre somme (operazioni di tempo costante) si ottiene quindi

$$T(n) = 2T(n-1) + c$$

È facile dimostrare con il metodo di sostituzione che $T(n) = \Omega(2^n)$, ovvero che esistono $d > 0$ e n_0 tali che $T(n) \geq d2^n$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= 2T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq 2d2^{n-1} + c && \text{[ipotesi induttiva]} \\ &= d2^n + c \\ &\geq d2^n \end{aligned}$$

qualunque sia $d > 0$ e n .

Vale anche $T(n) = O(2^n)$. Tuttavia, il tentativo di provare direttamente per induzione che $d > 0$ e n_0 tali che $T(n) \leq d2^n$ per un'opportuna costante $d > 0$ e $n \geq n_0$ fallisce. Si riesce invece a dimostrare che $T(n) \leq d(2^n - 1)$. Infatti:

$$\begin{aligned} T(n) &= 2T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq 2d(2^{n-1} - 1) + c && \text{[ipotesi induttiva]} \\ &= d2^n - 2d + c \\ &= d(2^n - 1) - d + c \\ &\leq d2^n \end{aligned}$$

Esercizio 1 (9 punti) Realizzare una funzione $append(T, x, z)$ che dato un albero binario di ricerca T , un suo nodo foglia x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna `nil`. (Assumere che i nodi dell'albero abbiano gli usuali campi $x.left$, $x.right$, $x.p$, $x.key$ e la radice sia $T.root$.) Valutarne la complessità.

Soluzione: Affinchè z possa essere inserito come figlio destro o sinistro di z , occorre in primo luogo che la chiave di z sia $\leq$ delle chiavi degli antenati di z che hanno x e quindi z nel sottoalbero sinistro e $\geq$ delle chiavi degli antenati di x che hanno x nel sottoalbero destro.

Quindi risalendo tali antenati, si verifica che la condizione sia soddisfatta. Se questo accade e $z.key$ è $\leq x.key$ potrà essere inserito come figlio sinistro di x , altrimenti come figlio destro. Altrimenti z non può essere figlio di x e si ritorna `nil`.

Lo pseudocodice può essere il seguente:

```
append(T,x,z)
```

```
# verifica che z.key sia <= delle chiavi degli antenati di x
# per i quali x e' nel sottoalbero dx e
y = x
```

```
while (y.p <> nil) and
      ( (y == y.p.left and z.key <= y.p.key) or
        (y == y.p.right and z.key >= y.p.key) )
  y = y.p
```

```
# se siamo arrivati alla radice, significa che la condizione e' soddisfatta
```

```
if (y.p == nil)
  # e z viene inserito come figlio dx o sx
  z.p = x
  if (z.key <= x.key)
    x.left = z
  else
    x.right = z
```

```
  return T
```

```
else
  # altrimenti si ritorna nil
  return nil
```

Append: verifichiamo se possiamo inserire "z" come figlio di "x" in T.

Per farlo, poniamo il solito nodo copia di "x" nel nodo "y"

Quindi:

finché il nodo "y" ha un parent, continuiamo a salire l'albero fino ad arrivare il più possibile vicino al parent: questo ci assicura di inserire effettivamente il nodo "z" come figlio di "y" (infatti, "y" diventa uguale al suo parent perché continuiamo a risalire l'albero).

Se siamo arrivati alla radice, quindi non possiamo andare più in alto di lei ($y.p == nil$), allora il parent di "z" sarà proprio "x" e capiamo a seconda del valore della chiave, dove andare ad inserire "z", se come figlio dx o sx di "x", ritornando tutto l'albero T, altrimenti si ritorna nil

La complessità è chiaramente $O(h)$, dove h è l'altezza dell'albero. Infatti, il ciclo while effettua un numero di iterazioni pari al numero di antenati di x , e quindi limitato dall'altezza dell'albero. E ciascuna iterazione ha costo costante.

Domanda A (8 punti) Si consideri la funzione ricorsiva $\text{search}(A, p, r, k)$ che dato un array A , ordinato in modo crescente, un valore k e due indici p, q con $1 \leq p \leq r \leq A.length$ restituisce un indice i tale che $p \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0.

```
search(A, p, r, k)
  if p <= r
    q = (p+r)/2
    if A[q] = k
      return q
    elseif A[q] < k
      return search(A, q+1, r, k)
    else
      return search(A, p, q-1, k)
  else
    return 0
```

Fondamentalmente, data la funzione, si dimostra per induzione che sia corretta. Questa vale, considerando che ordiniamo due sottometà di array a pezzi (esaminando le due metà separate, facendo valere la proprietà di ordinamento per ogni singolo indice presente). In tal modo, capiamo anche che, dato che esaminiamo due metà di array, la complessità è logaritmica.

Dimostrare induttivamente che la funzione è corretta. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

Soluzione: La prova di correttezza avviene per induzione sulla lunghezza $l = r - p + 1$ del sottoarray $A[p..r]$ di interesse. Se $l = 0$, ovvero $p > r$, la funzione ritorna correttamente 0, dato che non ci sono elementi nel sottoarray e quindi non ci possono essere elementi $= k$. Se invece $l > 0$ si calcola $q = \lfloor (p + r)/2 \rfloor$ e si distinguono tre casi:

- se $A[q] = k$ si ritorna correttamente k
- se $A[q] < k$, dato che l'array è ordinato certamente $A[j] \leq A[q] < k$ per $p \leq j \leq q$. Quindi il valore k può trovarsi solo nel sottoarray $A[q + 1, r]$. La chiamata $\text{search}(A, q + 1, r, k)$, dato che il sottoarray ha lunghezza minore di l , per ipotesi induttiva restituisce un indice i tale che $q + 1 \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. Per l'osservazione precedente, questo è il valore corretto da restituire anche per $\text{search}(A, p, r, k)$ e si conclude.
- se $A[q] > k$ si ragiona in modo duale rispetto al caso precedente.

Per quanto riguarda la ricorrenza, ignorando i casi base, dato che la funzione ricorre su di un array la cui dimensione è la metà di quello originale, si ottiene:

$$T(n) = T(n/2) + c$$

Rispetto allo schema standard del master theorem ho $a = 1$, $b = 2$ e $f(n) = c$. Ho dunque che $f(n) = 1 = \Theta(n^{\log_2 1}) = n^0 = 1$. Pertanto si conclude che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

L'idea dell'inserimento è:

- Aumentare la dimensione dell'heap di 1, in modo da poter memorizzare il nuovo elemento.

- Inserire il nuovo elemento alla fine dell'heap.

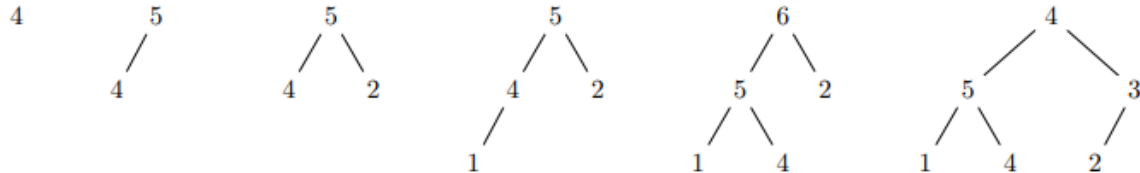
- Il nuovo elemento inserito può alterare le proprietà dell'Heap per i suoi genitori. Quindi, per mantenere le proprietà di Heap, si esegue l'heapificazione del nuovo elemento inserito (quindi, ogni nodo di un albero ha una chiave che è /maggiore o minore)-uguale alla chiave del suo genitore).

Max Heap: la radice è l'elemento massimo e i nodi rispettano la proprietà di max heap.

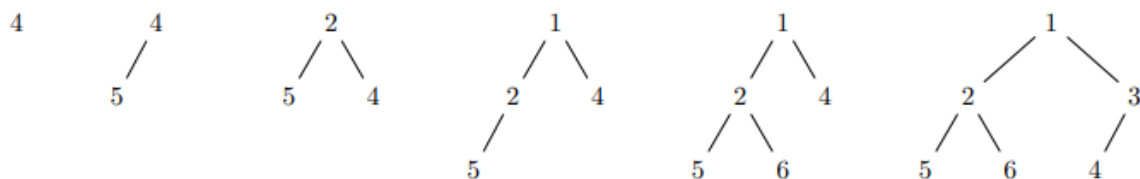
L'inserimento è completo (da sinistra a destra) e deve essere bilanciato.

Domanda B (5 punti) Si dia la definizione della struttura dati max-heap. Si supponga di inserire in un max-heap inizialmente vuoto, in successione, gli elementi 4, 5, 2, 1, 6, 3. Fornire la rappresentazione (ad albero) del max-heap così ottenuto. Si motivi la risposta mostrando la sequenza di max-heap ottenuti.

Soluzione: Per la definizione di max-heap e la procedura di inserzione di un elemento si rimanda al testo. La sequenza di max-heap ottenuta con l'inserimento degli elementi nell'ordine indicato è:



mentre invece la sequenza di min-heap è:



Esercizio 1 (9 punti) Realizzare una funzione `intersect(A1,A2,n)` che dati due array di interi `A1` e `A2`, organizzati a min-heap, con capacità `n`, restituisce un nuovo array `A`, ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in `A1` e `A2`. Nel caso gli array contengano più occorrenze dello stesso valore `v`, l'intersezione mantiene il numero minimo di occorrenze di `v` (ad es. se `A1` contiene i valori 1,2,2,2 e `A2` contiene i valori 1,1,2,2 allora `A` conterrà 1,2,2). Valutarne la complessità.

Soluzione: Per costruire l'intersezione, si procede nel modo seguente. Posso estrarre gli elementi di ciascun min-heap, in ordine crescente, semplicemente con una sequenza di estrazioni di minimo, ciascuna delle quali ha costo logaritmico. Se da `A1` estraggo v_1 e da `A2` estraggo v_2 , se $v_1 = v_2$, inserisco il valore comune nell'intersezione. Se invece $v_1 < v_2$, certamente v_1 non è nell'intersezione, dato che tutti gli elementi rimanenti in `A2` sono $\geq v_2$, quindi posso scartare v_1 , estrarre un nuovo elemento da `A1` e continuare. Se $v_1 > v_2$ procedo dualmente.

In questo modo ottengo gli elementi dell'intersezione in ordine crescente. L'ultima osservazione è che posso inserirli in questo modo in `A`, che risulterà un array crescente, che è un caso particolare di min-heap.

Lo pseudocodice può essere il seguente:

<pre> intersect(A1,A2,n) allocate A[1..n] i=0 while (A1.heapsize > 0) and (A2.heapsize > 0) v1 = Min(A1) v2 = Min(A2) if (v1 = v2) i++ A[i]=1 ExtractMin(A1) ExtractMin(A2) elseif (v1 < v2) ExtractMin(A1) else ExtractMin(A2) A.heapsize=i return A </pre>	Intersect: dati due min heap A1 e A2, restituisco un array che allochiamo A anche lui a min heap. // ultimo elemento occupato in A Finché esistono entrambi gli heap, essendo dei min-heap, allora estraiamo l'elemento minimo. Allora; - se l'elemento è uguale tra i due heap, aumento l'indice, la radice di A (che parte da 1) e poi continuiamo a estrarre il minimo elemento - se invece abbiamo l'elemento minore nel primo heap, lo estraiamo dal primo e lo mettiamo nell'array A - se invece abbiamo l'elemento minore nel secondo heap, lo estraiamo dal secondo e lo mettiamo nell'array A - la posizione di analisi di A è "i" e ritorniamo, a fine ciclo, A. Essendo un heap, avremo che il costo è $n \log n$.
--	---

La complessità di ogni **ExtractMin** è $O(\log n)$ mentre **Min** ha costo costante. Il numero di estrazioni è chiaramente limitato da $2n$ e questo porta a dedurre che il costo complessivo è $O(n \log n)$.

Domanda A (8 punti)

Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
val(n)
  if n = 0
    return 0
  else
    return val(n-1) + n + 1
```

Dimostriamo per induzione:

- caso base, indice " n " = 0 nella chiamata ricorsiva a " val ", banale

- caso induttivo, si prende il termine della funzione " $(n-1)$ " e la costante $(n+1)$.

Dato che dobbiamo dimostrare il theta di n , si dimostra per sostituzione e si dimostra che vale per due costanti, in questo caso " d " e " c ".

Dimostrare induttivamente che la funzione calcola il valore $(n+3)n/2$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Theta(n)$. Motivare le risposte.

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione su n . Se $n = 0$, la funzione ritorna 0 che è in effetti $(0+3)0/2 = 0/2 = 0$. Se $n > 0$, allora per ipotesi induttiva, $val(n-1) = (n+2)(n-1)/2$. Quindi $val(n) = val(n-1) + n + 1 = (n+2)(n-1)/2 + n + 1 = (n^2 + n - 2 + 2n + 2)/2 = (n^2 + 3n)/2 = (n+3)n/2$, come desiderato.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, si effettuano una chiamata ricorsiva con argomento $n-1$ e una somma (operazione di tempo costante) si ottiene quindi

$$T(n) = T(n-1) + c$$

È facile dimostrare con il metodo di sostituzione che $T(n) = \Theta(n)$. Dimostriamo, ad esempio, che $T(n) = \Omega(n)$, ovvero che esistono $d > 0$ e n_0 tali che $T(n) \geq dn$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq d(n-1) + c && \text{[ipotesi induttiva]} \\ &= dn - d + c \\ &\geq dn \end{aligned}$$

per una costante d tale che $0 < d \leq c$ e qualunque n .

Vale anche $T(n) = O(n)$, ovvero esistono $d > 0$ e n_0 tali che $T(n) \leq dn$ per un'opportuna costante $d > 0$ e $n \geq n_0$. Infatti:

$$\begin{aligned} T(n) &= T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq d(n-1) + c && \text{[ipotesi induttiva]} \\ &= dn - d + c \\ &\leq dn \end{aligned}$$

dove l'ultima disuguaglianza vale per una costante $d \geq c$ e qualunque sia n .

Esercizio 1 (9 punti) Scrivere una funzione $\text{Anc}(T, k1, k2)$ che dato un albero binario di ricerca T nel quale tutte le chiavi sono distinte, e due chiavi $k1, k2$ presenti in T , verifica se il nodo contenente $k1$ è antenato del nodo contenente $k2$. Valutare la complessità della funzione.

Soluzione: È sufficiente osservare che, dato che le chiavi sono uniche e $k1, k2$ sono certamente contenute in T , il nodo che contiene $k1$ è antenato di quello che contiene $k2$ se e solo se cercando $k2$ a partire dalla radice di T incontro la chiave $k1$. Si assume che un nodo sia antenato di sé stesso.

La funzione può dunque essere realizzata come una semplice variante della ricerca negli alberi binari di ricerca.

```
Anc(T, k1, k2)
  x = T.root

  while (x.key <> k1) and (x.key <> k2)
    if (k2 < x.key)
      x = x.left
    else
      x = x.right

  return (x.key == k1)
```

Anc: verifico se il nodo "k1" è antenato del nodo "k2".

Come al solito, ci salviamo in "x" la radice dell'albero. Cerchiamo quindi, avendo tutte le chiavi diverse, "k2" rispetto a "x":
- se "k2" è minore della chiave in "x", allora andiamo a sx in "x"
- se "k2" è maggiore della chiave in "x", allora andiamo a dx in "x"

Avendo che tutte le chiavi sono diverse, troveremo che partendo dalla radice, certamente "k1", con una complessità che è pari all'altezza dell'albero esaminato.

Se l'albero ha altezza h , nel caso peggiore non trovo $k1$ e la chiave $k2$ è una foglia a profondità h , che quindi raggiungo in h iterazioni. Pertanto la complessità è $O(h)$.

Domanda A (7 punti) Definire formalmente la classe $\Theta(g(n))$. Dimostrare le seguenti affermazioni o fornire un controesempio:

- se $f(n) \in \Theta(g(n))$ e $f'(n) \in O(g(n))$ allora $f(n) + f'(n) \in \Theta(g(n))$; (rispetto a $f(n)$)
- se $f(n) \in \Theta(g(n))$ e $f'(n) \in O(g(n))$ allora $f(n) - f'(n) \in \Theta(g(n))$;

Soluzione: Per la definizione di $\Theta(f(n))$, consultare il libro.

Per (i), sia $f(n) \in \Theta(g(n))$. Per definizione, esistono $c_1, c_2 > 0$ e n_0 tali che per ogni $n \geq n_0$ vale:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Inoltre, dato che $f'(n) \in O(g(n))$ esistono $c'_2 > 0$ e n'_0 tali che per ogni $n \geq n'_0$ vale:

$$0 \leq f'(n) \leq c'_2 g(n)$$

Quindi, per ogni $n \geq \max\{n_0, n'_0\}$ abbiamo:

$$0 \leq c_1 g(n) \leq f(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2) g(n)$$

dato che $c_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Quindi, essendo $k > 0$, per ogni $n \geq \max n_0$ abbiamo:

$$(c_1 + c'_1) g(n) = c_1 g(n) + c'_1 g(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2) g(n)$$

dato che $c_1 + c'_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Per la parte (ii), l'affermazione è falsa. Basta considerare $f(n) = f'(n) = n \in \Theta(n)$ e quindi in $O(n)$, ma ovviamente $f(n) - f'(n) \notin \Omega(n)$ e quindi $\notin \Theta(n)$.

Domanda B (7 punti) Scrivere una funzione `toTree(A)` che dato un array `A` organizzato a max-heap (dimensione `A.heapSize`), lo trasforma in un albero binario realizzato con strutture linked, ancora organizzato a max-heap e ritorna la radice di tale albero. Il nuovo albero è costituito da nodi `x` con i campi `x.p` (parent), `x.k` (chiave), `x.l` e `x.r` (figlio sinistro e figlio destro). Per allocare un nuovo nodo si assuma di avere a disposizione un costruttore `node()`. Valutare la complessità.

Soluzione: L'idea è semplicemente quella di visitare l'albero, memorizzato come array, in profondità, e riprodurlo nella struttura linked.

`toTree --> trasformare un max-heap in un albero binario.`

```
toTree(A)
    T.root = toTreeRec(A,1,nil)
    return T

// toTreeRec(A,i,x):
// dato un max-heap A ritorna la radice di un albero, copia "linked" del
// sottoalbero di A radicato in i, l'argomento x e' il nodo padre
toTreeRec(A,i,x)
    if i <= A.heapSize
        y = node()                // crea un nuovo nodo
        y.key = A[i]              // la chiave e' A[i]
        y.p = x                   // il parent e' x
        left = 2*i                // costruisce i sottoalberi sx e dx,
        right = 2*i+1             // che saranno figli sx e dx di y
        y.l = toTreeRec(A,left, y)
        y.r = toTreeRec(A,right,y)
    else
        return nil
```

Quindi, finché non abbiamo esaminato tutto lo heap:

- creo un nodo in cui la chiave è `A[i]` attuale
- il parent del nodo è dato da "x", all'inizio "nil" e mano a mano resta il genitore di "y"
- creiamo il nodo `sx` e `dx` (rispettivamente, secondo le proprietà di heap, saranno $(2 * i)$ e $(2 * i + 1)$)
- andiamo quindi a prenderci gli elementi `dx` e `sx` dell'albero seguendo un'idea ricorsiva, quindi passando "left" e "right" e il parent "y" come stabilito.

Si tratta di una visita dell'albero e quindi la complessità è $\Theta(n)$ dove n è il numero di elementi del max-heap.

Dato che è una visita completa di tutto l'albero, si esegue in tempo lineare, perché alla peggio guardiamo ogni singolo elemento.

Esercizio 1 (8 punti) Realizzare una funzione `missing(A,n)` che dato un array `A[1..n]` che contiene interi nell'intervallo `[1,n]` verifica se esiste qualche valore $v \in [1,n]$ che non compare in `A`, ovvero tale che $A[i] \neq v$ per ogni $i \in [1,n]$. Valutare la complessità.

Soluzione: Si osserva che in `A[1..n]` possono mancare dei valori se e solo se ci sono elementi duplicati.

Per verificare la presenza di duplicati, si scorre l'array `A` e si usa un array di booleani `B[1..n]` per ricordare i valori già incontrati. Più precisamente `B[v]` indica se è stato o meno incontrato il valore v nella parte di array `A` esplorata. Ci sarà un duplicato (e quindi un valore assente) solo se durante l'esplorazione si incontra un valore v che era già stato incontrato prima e quindi con `B[v]=true`.

```
missing(A,n)
  alloca B[1..n] array di booleani

  for i=1 to n
    B[i]=false

  i=1
  while (i<=n) and not B[A[i]]
    B[A[i]] = true
    i++

  return i<=n
```

Si usa una idea concettualmente simile a CountingSort, quindi usiamo un array inizializzato tutto a "false" chiamato `B`. In questo, verifichiamo per ogni indice di `A` se esiste un valore nell'intervallo "i" da 1 ad n che non compare. Nel qual caso, qualche elemento rimarrebbe a "false" e l'idea è di avere un indice "i" che esamina tutti gli elementi e verifica se vale questa proprietà.

Ritorniamo quindi l'indice "i" che dice "fino a dove vale la proprietà di possedere tutti i valori"; infatti, noi in Missing vogliamo sapere se manca qualche valore nell'intervallo, partendo da "false" in `B`.

L'invariante del ciclo è

- $A[1, i - 1]$ non contiene duplicati e
- $\forall v \in [1, n]. B[v] = true$ se e solo se $\forall j \in [1, i - 1]. A[j] \neq v$.

Quindi, se si esce perché $i = n + 1$, significa che $A[1, n]$ non contiene duplicati e quindi non ci sono valori assenti.

Se invece si esce con $i \leq n$, significa che $B[A[i]] = true$. Pertanto deve esistere $j < i$ tale che $A[j] = A[i]$, ovvero c'è un duplicato e quindi necessariamente un valore assente.

La complessità è chiaramente $\Theta(n)$, dato che il ciclo di inizializzazione di `B` costa n , e il `while` fa al massimo n iterazioni, ciascuna di costo costante.

Domanda A (7 punti) Si consideri la funzione ricorsiva $\text{search}(A, p, r, k)$ che dato un array A , ordinato in modo decrescente, un valore k e due indici p, q con $1 \leq p \leq r \leq A.length$ restituisce un indice i tale che $p \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. (Già esaminata a pag. 83)

```
search(A, p, r, k)
    if p <= r
        q = (p+r)/2
        if A[q] = k
            return q
        elseif A[q] > k
            return search(A, q+1, r, k)
        else
            return search(A, p, q-1, k)
    else
        return 0
```

Dimostrare induttivamente che la funzione è corretta. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

Soluzione: La prova di correttezza avviene per induzione sulla lunghezza $l = r - p + 1$ del sottoarray $A[p..r]$ di interesse. Se $l = 0$, ovvero $p > r$, la funzione ritorna correttamente 0, dato che non ci sono elementi nel sottoarray e quindi non ci possono essere elementi $= k$. Se invece $l > 0$ si calcola $q = \lfloor (p+r)/2 \rfloor$ e si distinguono tre casi:

- se $A[q] = k$ si ritorna correttamente k
- se $A[q] > k$, dato che l'array è ordinato in modo decrescente certamente $A[j] \geq A[q] > k$ per $p \leq j \leq q$. Quindi il valore k può trovarsi solo nel sottoarray $A[q+1, r]$. La chiamata $\text{search}(A, q+1, r, k)$, dato che il sottoarray ha lunghezza minore di l , per ipotesi induttiva restituisce un indice i tale che $q+1 \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. Per l'osservazione precedente, questo è il valore corretto da restituire anche per $\text{search}(A, p, r, k)$ e si conclude.
- se $A[q] < k$ si ragiona in modo duale rispetto al caso precedente.

Per quanto riguarda la ricorrenza, ignorando i casi base, dato che la funzione ricorre su di un array la cui dimensione è la metà di quello originale, si ottiene:

$$T(n) = T(n/2) + c$$

Rispetto allo schema standard del master theorem ho $a = 1$, $b = 2$ e $f(n) = c$. Ho dunque che $f(n) = 1 = \Theta(n^{\log_2 1}) = n^0 = 1$. Pertanto si conclude che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

Domanda B (6 punti) Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + 2(k \bmod (m-2))$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 13, 29, 19, 27, 8.

Soluzione: Si ottiene [Si guardi l'esempio a pagina 103 basato sul doppio hash.](#)

0	29
1	-
2	27
3	19
4	-
5	13
6	-
7	8

Esercizio 1 (9 punti) Si consideri un albero binario T , i cui nodi x hanno i campi $x.l$, $x.r$, $x.p$ che rappresentano il figlio sinistro, il figlio destro e il padre, rispettivamente. Un *cammino* è una sequenza di nodi $x_0, x_1, \dots, x_n$ tale che per ogni $i = 0, \dots, n-1$ vale $x_{i+1}.p = x_i$. Il cammino è detto *nil-terminated* se $x_n.l = \text{nil}$ oppure $x_n.r = \text{nil}$. Dato un certo k , diciamo che l'albero è k -bilanciato se tutti i cammini nil-terminated che iniziano dalla radice hanno lunghezze che differiscono al più di k . Scrivere una funzione $\text{bal}(T, k)$ che dato in input l'albero T e un valore k verifica se T è k -bilanciato e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: La soluzione può basarsi su di una funzione ricorsiva che calcola per un nodo x la lunghezza massima e minima dei cammini che iniziano da x e sono nil-terminated

```
bal(T,k)    // verifica se l'albero radicato in x e' k-bilanciato,
            // ovvero i cammini dalla radice terminati da nil hanno
            // lunghezze che differiscono al piu di k
```

```
            // si basa su di una funzione ricorsiva che calcola la
            // lunghezza minima e massima dei cammini nil-terminated
            // da un certo nodo.
```

```
min,max = nilTerm(T.root)
```

```
return max-min <= k
```

```
nilTerm(x)
```

```
if x = nil
```

```
    return (0,0)
```

```
else
```

```
    (left_min,left_max) = nilTerm(x.left)
```

```
    (right_min,r_right_max) = nilTerm(x.right)
```

```
    return min(left_min, right_min) + 1, max(left_max,right_max) + 1
```

Vogliamo verificare se l'albero è k -bilanciato (quindi, i cammini dalle radici alle foglie differiscono al max di k) --> variante dell'esercizio 1-bilanciato. Quindi, ricorriamo su tutti i nodi e ritorniamo due variabili (min-max) la cui differenza

permette di intuire il bilanciamento degli alberi. Infatti:

- ricorriamo a sinistra e salviamo il nodo più a sx e il nodo più a dx (min e max) del sottoalbero sx

- ricorriamo a destra e salviamo il nodo più a dx e il nodo più a dx (min e max) del sottoalbero dx

Assieme al nodo chiave, sommiamo 1 al percorso a sx e al percorso a dx, così prendiamo chiave e tutti i nodi.

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n - k - 1)$ per un'opportuna costante c e quindi, la complessità è $O(n)$.

Ricorriamo quindi su ogni nodo dell'albero, avendo alla peggio una complessità lineare.

La differenza principale è verificare se l'intervallo (dato dalle due variabili "min" e "max" sia $\leq k$, intuendo così se si tratta proprio di un albero k -bilanciato.

Caso 3 del MT e si risolve per una costante "k", ponendo la condizione di regolarità.

Domanda A (6 punti) Data la ricorrenza $T(n) = 6T(n/3) + (n-1)^2$, trovare la soluzione asintotica.

Soluzione: Si usa il master theorem con $a = 6$, $b = 3$, $f(n) = (n-1)^2$. Si deve confrontare $f(n)$ con $n^{\log_b a} = n^{\log_3 6}$ ed essendo che $\log_3 6 < 2$ per qualunque $0 < \epsilon < 2 - \log_3 6$ si vede facilmente che $f(n) = \Omega(n^{\log_3 6 + \epsilon})$.

Per concludere che $T(n) = \Theta(f(n)) = \Theta(n^2)$ usando il caso 3 del Master Theorem occorre dimostrare la regolarità di $f(n)$, ovvero che $af(n/b) < kf(n)$ per $0 < k < 1$, asintoticamente. Si imposta

$$af(n/b) = 6(n/3 - 1)^2 < k(n-1)^2$$

e si osserva

$$6(n/3 - 1)^2 = 6((n-3)/3)^2 < 6((n-2)/3)^2 = 6/9(n-2)^2 = 2/3(n-2)^2$$

per cui si può scegliere $k = 2/3 < 1$ e n qualunque.

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *toro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

		t	<u>o</u>	r	o
	0	0	0	0	0
a	0	0	0	0	0
r	0	0	0	1	1
m	0	0	0	1	1
<u>o</u>	0	0	1	1	2

La lunghezza della longest common subsequence tra *armo* e *toro* è quindi 2.

Esercizio 1 (10 punti) Dare la definizione di albero binario di ricerca. Scrivere una funzione `conta(T,a,b)` che dato un albero binario di ricerca `T` e due valori `a` e `b`, ritorna il numero di nodi `x` tali che $x.key \in [a, b]$. Mostrarne la correttezza e valutarne la complessità.

Soluzione: L'idea è quella di realizzare una funzione ricorsiva `contaRec(x,a,b)`, che dato un nodo `x`, restituisce il numero di nodi nel sottoalbero radicato in `x` con chiave nell'intervallo $[a, b]$. Quindi la si applica alla radice dell'albero.

```
conta(T,a,b)
    return contaRec(T.root, a,b)
```

```
contaRec(x, a, b)
    if x <> nil
        if b < x.key
            return contaRec(x.left)
        else if x.key < a
            return contaRec(x.right)
        else
            return 1 + contaRec(x.right) + contaRec(x.left)
```

Conta -> ritorna il numero di nodi tali che la chiave di "x" sia nell'intervallo "a" e "b".

Per fare ciò, comodamente, passiamo ad una funzione ricorsiva di supporto quanto da fare.

Quindi:

- se il nodo non è nullo

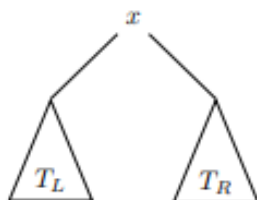
- se "b" è minore della chiave di "x", significa che "x" è più grande e dobbiamo ricorrere a sinistra, trovando i nodi che rispetto l'estremo superiore dell'intervallo (dato che "b" è l'estremo più grande e sicuramente a sx ci sono tutti i valori più piccoli)

- se "a" è maggiore della chiave di "x", significa che "x" è più piccolo e dobbiamo ricorrere a destra, trovando i nodi che rispetto l'estremo inferiore dell'intervallo (dato che "a" è l'estremo più piccolo e sicuramente a dx ci sono tutti i valori più grandi)

La ricorsione ci garantisce di trovare tutti i nodi (più il nodo con "x") che stanno dentro l'intervallo $[a, b]$

La correttezza della funzione ricorsiva può essere dimostrata per induzione sull'altezza h del sottoalbero radicato in x

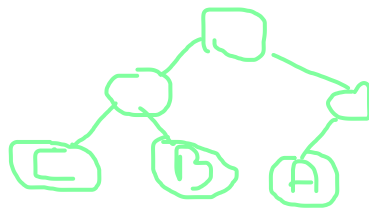
- ($h = 0$) In questo caso, $x = nil$, il sottoalbero è vuoto e quindi correttamente si ritorna 0.
- ($h > 0$) In questo caso l'albero ha la forma



Si distinguono tre casi:

- se $b < x.key$, sicuramente, per le proprietà degli ABR, tutti i nodi nel sottoalbero T_R hanno chiave maggiore di $x.key$ e quindi di b , e pertanto non sono nell'intervallo. Dunque si ritorna il numero di nodi in $[a, b]$ nel sottoalbero T_L , che per ipotesi induttiva è fornito da `contaRec(x.left)`.
- $x.key < a$ si ragiona in modo duale rispetto al caso precedente
- altrimenti, $x.key \in [a, b]$ e i nodi con chiave in $[a, b]$ possono trovarsi sia nel sottoalbero sinistro che nel sottoalbero destro. Si ritorna dunque $1 + \text{contaRec}(x.right) + \text{contaRec}(x.left)$, dove le chiamate `contaRec(x.right)` e `contaRec(x.left)` ritornano, per ipotesi induttiva, i nodi in $[a, b]$ nei sottoalberi T_R e T_L .

Nel caso peggiore, ad esempio quando tutti i nodi hanno chiave in $[a, b]$, la funzione esegue una visita completa dell'albero e quindi la complessità è $O(n)$ dove n è il numero dei nodi.



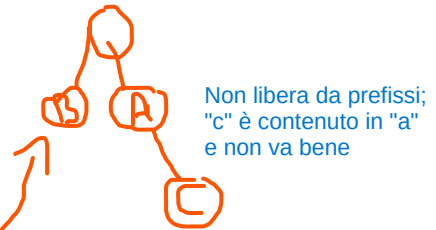
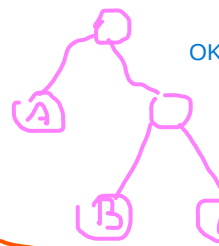
Per andare bene, (a) dovrebbe essere al livello superiore, perché in pratica Huffman è un albero somma senza nodi delle stesse frequenze allo stesso livello.

Esercizio 2 (9 punti) Si consideri un file definito sull'alfabeto $\Sigma = \{a, b, c\}$, con frequenze $f(a), f(b), f(c)$. Per ognuna delle seguenti codifiche si determini, se esiste, un opportuno assegnamento di valori alle 3 frequenze $f(a), f(b), f(c)$ per cui l'algoritmo di Huffman restituisce tale codifica, oppure si argomenti che tale codifica non è mai ottenibile.

1. $e(a) = 0, e(b) = 10, e(c) = 11$ →

2. $e(a) = 1, e(b) = 0, e(c) = 11$ →

3. $e(a) = 10, e(b) = 01, e(c) = 00$ →



Soluzione:

1. Questa codifica viene restituita dall'algoritmo di Huffman quando $f(b), f(c) < f(a)$: in questo caso i nodi associati a b e c vengono uniti creando un nuovo nodo interno, che poi viene unito al nodo associato ad a . Quindi, per esempio, $f(a) = 40, f(b) = 25, f(c) = 35$.
2. La codeword $e(a) = 1$ è un prefisso della codeword $e(c) = 11$, cioè questa codifica non è libera da prefissi, e quindi non è una codifica di Huffman.
3. L'albero associato a questa codifica non è pieno perché c'è un nodo interno con un solo figlio (quello nel cammino che porta alla foglia associata al carattere a). Quindi questa codifica non è ottima e quindi non è una codifica di Huffman.

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un array A di interi e due indici $1 \leq p \leq r \leq A.length$:

```
Minimum(A,p,r)
  if p=r
    return A[p]
  else
    q = (p+r)/2
    return min(Minimum(A,p,q), Minimum(A,q+1,r))
```

Correttezza induttiva:
 - $n=1$, ritorno l'unico elemento presente
 - $n > 1$, allora cerco nelle due metà di array alla MergeSort il minimo dei due sottoarray, con ricorrenza data da $2T(n/2) + c$, dato dall'avere due chiamate su due sottometà di array come descritto.

Dimostrare induttivamente che la funzione calcola il minimo del sottoarray $A[p..r]$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Theta(n)$, dove n indica la lunghezza del sottoarray. Motivare le risposte.

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione sulla lunghezza $n = r - p + 1$ del sottoarray $A[p..q]$. Se $n = 1$, il sottoarray di interesse contiene un solo elemento $A[p]$ che è correttamente l'elemento ritornato. Se invece $n > 1$, le due chiamate ricorsive $Minimum(A, p, q - 1)$ e $Minimum(A, q, r)$, su sottoarray di dimensione $< n$ ritornano il minimi in $A[p..q]$ e $A[q + 1, r]$. Viene ritornato il valore mimimimo tra i due, che srà correttamente il minimo valore in $A[p..r]$.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, se n è la lunghezza del sottoarray di interesse, si effettuano due chiamate ricorsive su sottoarray di dimensione $n/2$ e un'operazione di minimo (operazione di tempo costante) si ottiene quindi

$$T(n) = 2T(n/2) + c$$

Si applica la formula del MT e si ha un facile caso 1.

Per risolvere la ricorrenza possiamo applicare il Master Theorem (siamo in una istanza dello schema generale con $a = b = 2$ e $f(n) = c$). Si osserva che $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. Quindi $f(n) = c = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ qualunque sia $0 < \epsilon < 1$. Quindi siamo nel primo caso del Master Theprem e si conclude che $T(n) = \Theta(n^{\log_b a}) = \Theta(n)$.

Domanda B (6 punti) Realizzare una funzione `SearchUnique(T,k)` che dato un albero binario di ricerca `T` (che si assume non vuoto), verifica se la chiave `k` è presente in un unico nodo, e, in caso affermativo restituisce il nodo, altrimenti (ovvero se la chiave non è presente oppure è presente in più nodi) restituisce `nil`. Si possono usare le funzioni standard degli alberi binari di ricerca. Valutarne la complessità.

Soluzione: L'idea è quella di cercare la chiave `k` con l'usuale ricerca dei BST. Se la chiave è presente, detto `x` il nodo trovato, per verificare se sia unica, si controlla che sottoalbero destro e sinistro non contengano ulteriori occorrenze della chiave. Questo si può fare con due ulteriori ricerche. La usuale funzione dei BST che cerca una chiave `k` nel sottoalbero radicato in `x` si indica con `Search(x,k)`.

`Searchunique`: dato un BST "`T`", verifichiamo se "`k`" è presente in un unico nodo, per cui se presente, viene restituito.

Quindi:

- ci prendiamo la radice dell'albero
- usiamo la funzione `Search` dei BST e cerchiamo a sx oppure a dx se ci sta il nodo "`x`" e se non ci sono ulteriori occorrenze a sx o a dx della chiave, blocchiamo la ricerca (`x==nil`), avendo guardato tutti i nodi e restituiamo "`x`", altrimenti "`nil`".

```
SearchUnique(T,k)
  x = Search(T.root,k)

  if (x == nil) or (Search(x.left, k) <> nil) or (Search(x.right, k) <> nil)
    return x
  else
    return nil
```

L'algoritmo è corretto. L'osservazione fondamentale è che dato un nodo `x` di un BST, se la chiave `k` compare nel sottoalbero destro e in quello sinistro allora è anche la chiave della radice (infatti, per le proprietà dei BST, si avrebbe $k \leq x.key \leq k$). Ne segue che tutti i nodi con chiave `k` in `T` saranno nel sottoalbero del nodo `x`, ritornato dalla prima chiamata della funzione `Search`. Questo immediatamente implica la correttezza dell'algoritmo.

La complessità è quella di tre ricerche ciascuna con costo $O(h)$, dove h è l'altezza dell'albero, e quindi $O(h)$.

In alternativa, si può osservare che, se la chiave `k` dovesse comparire nel sottoalbero sinistro, sarebbe il massimo di questo sottoalbero e, dualmente, se comparisse nel secondo sottoalbero, sarebbe il minimo. Questo porta alla soluzione che segue

```
SearchUnique(T,k)
  x = Search(T.root,k)

  if (x <> nil)
    y = Max(x.left)
    z = Min(x.right)

    if ((y <> nil) and (y.key == k)) or (z <> nil) and (z.key == k))
      return nil
    else
      return x
  else
    return nil
```

Alternativamente, sfruttando le proprietà dei BST, sappiamo che, (solo nel sottoalbero in cui stiamo cercando):

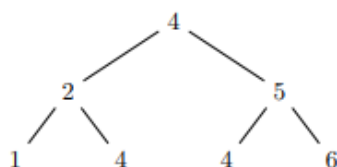
- se la chiave "`k`" compare a sx, allora questo è il massimo (cioè vuol dire, più andiamo verso il basso a sx più il valore diminuisce e sarebbe il massimo solo relativamente al suo sottoalbero)
- se la chiave "`k`" compare a dx, allora questo è il minimo (cioè vuol dire, più andiamo verso il basso a dx più il valore aumenta e sarebbe il minimo solo relativamente al suo sottoalbero)

Questo mi porta, presa la radice di `T` in "`x`", a vedere, quando questa non sia nulla, il massimo a sinistra e il minimo a destra e:

- se il massimo non è nullo e la chiave del massimo corrisponde al massimo del sottoalbero sx, abbiamo trovato il valore che cercavamo a sx
- se il minimo non è nullo e la chiave del minimo corrisponde al minimo del sottoalbero dx, abbiamo trovato il valore che cercavamo a dx
- ritorniamo "`x`" solamente se siamo sicuri che entrambi non siano nulli (in questo modo, o "`x`" compare a sx, oppure compare a dx ed è l'unica occorrenza).

La complessità asintotica non cambia, ma questa funzione è più efficiente della precedente dato che `Min` e `Max`, diversamente a `Search` non eseguono confronti sulle chiavi.

Per concludere, è da osservare che l'assunzione che le chiavi duplicate siano solo nel sottoalbero destro (o in quello sinistro) o che siano adiacenti non è legittima. Ad esempio, il seguente è un BST valido:



Esercizio 1 (9 punti) Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno un campo addizionale $x.min$, che rappresenta il minimo delle chiavi nel sottoalbero radicato in x . Realizzare la procedura $Insert(T,z)$ che inserisce un nodo z nell'albero e la rotazione a sinistra $Left(T,x)$ (assumendo che x e $x.right$ non siano nil). Valutarne la complessità.

Soluzione: La procedura di inserimento scende dalla radice fino ad una foglia dove il nodo viene inserito z . Dato che z sarà nel sottoalbero di ogni nodo attraversato durante la discesa, per ciascuno di questi nodi e soltanto per essi il campo min deve essere possibilmente aggiornato. Una piccola ottimizzazione: la verifica che $z.key < x.min$ ha senso farla solo se $z.key < x.key$ dato che $x.min \leq x.key$.

```

Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x

    if (z.key < x.key)
      if z.key < x.min      /* il sottoalbero radicato in z contiene z */
        x.min = z.key      /* quindi si aggiorna x.min */

    x = x.left
  else
    x = x.right

  z.p = y
  if (y <> nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  z.min = z.key /* il sottoalbero radicato in z contiene solo z */

```

La procedura dell'inserimento con una variante: salviamo anche il minimo del sottoalbero radicato in "x":
L'osservazione è che questo vada fatto "solo" quando sono a sx e, tra le altre cose, quando la chiave di "z" è minore di "x", perché il minimo è certamente $\leq$ della chiave di "z":

La complessità rimane la stessa di $Insert$ negli alberi binari di ricerca "standard", ovvero $O(h)$, dove h è l'altezza dell'albero.

Anche la rotazione a sinistra si ottiene inserendo, in coda al codice standard, l'aggiornamento del campo min . Si osservi che gli unici nodi per i quali si modificano i nodi contenuti nel corrispondente sottoalbero sono x e $y = x.right$. I nodi che prima erano nel sottoalbero radicato in x poi sono in quello radicato in y , quindi i valori dei campi min e max per y dopo la rotazione sono quelli che prima erano in x . Per calcolare il nuovo valore del campo min per x si usa la conoscenza di tali valori per i figli destro e sinistro.

https://upload.wikimedia.org/wikipedia/commons/f/f2/Binary_Tree_Rotation_%28animated%29.gif

```

Left(T,z)
  y = x.right
  x.right = y.left
  x.right.p = x
  Transplant(T,x,y)
  y.left = x
  x.p = y

  y.min = x.min

  x.min = min(x.key, x.left.min, x.right.min)

```

Descrivendo quello che succede:

- "y" è il nodo destro di "x"
- il nodo destro di "x" diventa il nostro sinistro di "y"
- il nodo destro destro del parent di "x" è "x"
- eseguo lo scambio tra "x" ed "y"
- il nodo sinistro di "y" è "x"
- il parent di "x" è "y".

Ora:

- il minimo di "y" è quello che c'era prima in "x" in quanto li abbiamo scambiati e ruotati
- il nuovo minimo di x è dato dalla ricerca del minimo su tutti i nodi una volta eseguita tutta la rotazione usando i nuovi figli dx e sx.

La complessità resta costante $O(1)$.

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *triangolare* se esiste $q \in [1, n]$ tale che $A[1..q]$ è ordinato in modo crescente e $A[q..n]$ è ordinato in modo decrescente. Realizzare una funzione $\text{maxTr}(A)$ che dato un array triangolare $A[1..n]$, senza elementi ripetuti, ne trova il massimo. Valutarne la complessità.

Soluzione: Si realizza una procedura divide et impera che opera sul sotto-array $A[p, r]$, che inizialmente sarà l'intero array. Se $p = r$, ovvero il sotto-array ha dimensione 1, semplicemente si ritorna l'unico elemento $A[p]$. Altrimenti, si considera il punto intermedio $q = \lfloor p + r \rfloor$. Se $A[q] < A[q + 1]$ significa che la parte $A[p..q]$ è crescente e la parte $A[q+1..r]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima e può essere cercato ricorsivamente. Dualmente se $A[q] > A[q + 1]$ significa che la parte $A[q+1..r]$ è decrescente e la parte $A[p..q]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima.

```

maxTr(A, p, r)
    if p=r
        return A[p]
    else
        q = (p+r)/2
        if A[q]<A[q+1]
            return maxTr(A, q+1, r)
        else
            return maxTr(A, p, q)

maxTr(A)
    return maxTr(A, 1, A.length)

```

Avendo dalla consegna un ragionamento esplicito su due metà distinte di array, allora:

- verifichiamo se abbiamo almeno due metà su cui lavorare (inizio e fine) e se avessimo un solo elemento, lo restituiamo
- prendiamo l'elemento medio e
- se l'elemento medio è minore dell'elemento medio +1, allora spostiamo in avanti l'indice medio (così, il massimo sarà scorrendo avanti sulla seconda partizione)
- se l'elemento medio è maggiore dell'elemento medio +1, allora spostiamo indietro l'indice medio (così, il massimo sarà su "p", quindi sulla prima partizione)

La complessità è espressa dalla ricorrenza

$$T(n) = T(n/2) + c$$

Si può risolvere utilizzando il master theorem, confrontando $n^{\log_b a} = n^{\log_2^2} = n$ con $f(n) = c$. Dato che $f(n) = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ per $0 < \epsilon < 1$ si conclude che siamo nel primo caso del teorema che ci dà la soluzione $T(n) = \Theta(\log n)$.

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
val(n)
  if n <= 2
    return 1
  else
    return val(n-1) + val(n-2) + val(n-2)
```

La ricorrenza è data da una sola chiamata con complessità $(n-1)$ e due chiamate, come si vede,, con complessità $(n-2)$. Dimostriamo che è sia Omega che O grande e risolviamo per sostituzione come sempre.

Determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Omega(2^n)$. La complessità è anche $O(2^n)$? Motivare le risposte.

Soluzione: Nel caso ricorsivo, si effettuano tre chiamate ricorsive, una con argomento $n - 1$ e due con argomento $n - 2$, e inoltre due somme (operazioni di tempo costante) si ottiene quindi

$$T(n) = T(n - 1) + 2T(n - 2) + c$$

Dimostriamo con il metodo di sostituzione che $T(n) = \Omega(2^n)$, ovvero dimostriamo che esistono $d > 0$ e n_0 tali che $T(n) \geq d \cdot 2^n$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= T(n - 1) + 2T(n - 2) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq d2^{n-1} + 2d2^{n-2} + c && \text{[ipotesi induttiva]} \\ &\geq d2^{n-1} + 2d2^{n-2} && \text{[poiché } c > 0\text{]} \\ &= d(2^{n-1} + 2 \cdot 2^{n-2}) \\ &= d2^n \end{aligned}$$

qualunque siano d e n_0 .

Vale anche $T(n) = O(2^n)$, ovvero esistono $d > 0$ e n_0 tali che $T(n) \leq d2^n$ per un'opportuna costante $d > 0$ e $n \geq n_0$. Ai fini della prova induttiva è opportuno dimostrare che $T(n) \leq d(2^n - 1)$. Infatti:

$$\begin{aligned} T(n) &= T(n - 1) + 2T(n - 2) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq d(2^{n-1} - 1) + 2d(2^{n-2} - 1) + c && \text{[ipotesi induttiva]} \\ &= d(2^{n-1} + 2 \cdot 2^{n-2}) - 3d + c \\ &= d2^n - 3d + c \\ &\leq d2^n \end{aligned}$$

dove l'ultima disuguaglianza vale se $-3d + c \leq 0$, e quindi per $d \geq c/3$.

Domanda A (6 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 4T(n/3) + 2n^2 + 1$$

Caso 3 del MT e si risolve con condizione di regolarità.

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 3$, $f(n) = 2n^2 + 1$. Si osserva che $\log_3 4 = \log_3 4 < 2$ quindi $f(n) = \Omega(n^{\log_3 4 + \epsilon})$ (per $0 < \epsilon \leq 2 - \log_3 4$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{3}) = 4(\frac{2n^2}{9} + 1) \leq 2cn^2 \leq c(2n^2 + 1)$, asintoticamente, quando $4/9 < c < 1$. Più precisamente, per avere

$$4(\frac{2n^2}{9} + 1) \leq 2cn^2$$

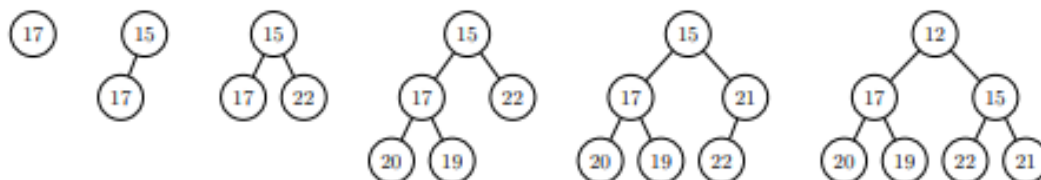
Occorre che $c \geq \frac{4}{9} + \frac{2}{n^2}$, che, per $n \geq 2$, si ottiene quando $c \geq \frac{4}{9} + \frac{2}{4} = \frac{15}{18} < 1$.

Domanda B (7 punti) Dare la definizione di min-heap. Data la sequenza di elementi 17, 15, 22, 20, 19, 21, 12, si specifichi il min-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 17. Si descriva sinteticamente come si procede per arrivare al risultato.

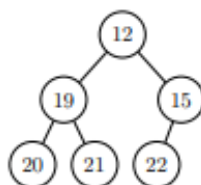
Soluzione: Un min-heap è un albero binario ordinato quasi-completo con la proprietà che per ogni nodo x , se x non è radice, il genitore di x ha chiave minore o uguale a quella di x , o equivalentemente ogni nodo x ha chiave minore o uguale a quella dei suoi successori.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MinHeapifyUp** per ripristinare la proprietà di min-heap.

Gli inserimenti nell'ordine producono gli heap indicati in figura



Infine, la rimozione di un nodo con chiave x si realizza sostituendolo con l'ultima foglia y e richiama **MinHeapifyUp**, se $y < x$ oppure **MinHeapify**, se $y > x$. Nello specifico per eliminare 17 lo si rimpiazza con l'ultima foglia 21 e si richiama **MinHeapify**, ottenendo il min-heap



che in forma di array è [12, 19, 15, 20, 21, 22].

Esercizio 1 (10 punti) Diciamo che un array senza ripetizioni $A[1..n]$ è *semi-ordinato* se esiste un indice k , con $1 \leq k < n$, tale che $A[k+1..n]$ sia ordinato, ovvero i sottoarray $A[k+1..n]$ e $A[1..k]$ sono ordinati e $A[n] < A[1]$. In questo caso l'indice k viene detto il centro dell'array. Ad esempio l'array che segue è semi-ordinato con centro $k = 4$.

1	2	3	4	5	6	7
4	9	12	18	-1	1	2

Semi-ordinato: dato un centro "k", due sottoporzioni di array sono ordinate in senso crescente e $A[1] > A[n]$, dato che sono gli estremi delle partizioni.

Scrivere una funzione `centre(A)` che dato un array A semi-ordinato ne restituisce il centro. Giustificare la correttezza dell'algoritmo e valutarne la complessità.

Soluzione: L'idea è quella di procedere con un algoritmo divide et impera. A tal fine osservazione fondamentale è la seguente: dato un sottoarray $A[p..r]$ semi-ordinato, se lo divido in due sottoarray $A[p..q]$ e $A[q..r]$, allora uno solo dei due è semi-ordinato (quello che contiene il centro), mentre l'altro è ordinato. Quando la dimensione del sottoarray $A[p..r]$ diventa 2, il centro è p .

Lo pseudo-codice è quindi il seguente:

```

centre(A)
    return centre-rec(A,1,A.length)

centre-rec(A,p,r)
    if r == p+1
        return p
    else
        q = (p+r)//2
        if (A[q]<A[p])
            return centre-rec(A,p,q)
        else
            return centre-rec(A,q,r)

```

Come sempre, ragioniamo tramite Mergesort, quindi prendiamo due metà di array su cui andare a iterare.

- se sono uguali, ritorno l'indice unico dell'array, indice iniziale.
- altrimenti, prendo l'indice medio
- se l'elemento con indice medio è minore dell'elemento con estremo inferiore della prima partizione, allora cerchiamo nella prima metà di array
- se l'elemento con indice medio è maggiore dell'elemento con estremo inferiore della prima partizione, allora cerchiamo nella seconda metà di array

In questo modo, cerchiamo come al solito su due metà di array distinte e prima o poi, essendo corrette ed ordinate le partizioni, troviamo un centro per cui si ha questa proprietà

Essendo che è il centro, certamente::

- se si trova in prima posizione dell'array, è minore dell'elemento in fondo all'array, perché induttivamente, iteriamo su due metà ordinate in senso crescente nell'array
- se si trova in ultima posizione dell'array, è maggiore dell'elemento in cima all'array, perché induttivamente, iteriamo su due metà ordinate in senso crescente nell'array

La prova di correttezza procede per induzione sul numero di elementi n dell'array $A[p..r]$

- Se $n = 2$, ovvero $r = p + 1$, allora $A[p + 1] < A[p]$ e chiaramente p è il centro.
- Se invece $n > 2$, l'array viene diviso in due parti, ovvero si definisce $q = \lfloor (p+r)/2 \rfloor$ e si considerano gli array $A[p..q]$ e $A[q..r]$. Detto k il centro, ci sono due possibilità.
 - ($q \leq k$) Questo accade se e solo se $A[p] > A[q]$, e correttamente in questo caso si ricorre sul sottoarray $A[q..r]$
 - ($q > k$) Questo accade se e solo se $A[p] < A[q]$, e correttamente in questo caso si ricorre sul sottoarray $A[p..q]$

Per quanto riguarda la complessità, la ricorrenza è del tipo $T(n) = T(n/2) + c$, dato che ad ogni chiamata si dimezza la dimensione dell'array e il costo della parte non ricorsiva è costante. Per il Master Theorem (con $a = 1$, $b = 2$, $f(n) = c = \Theta(n^{\log_b a}) = \Theta(n^0) = \Theta(1)$) si conclude $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$.

Domanda A (7 punti) Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{1}{4} T(n-1) + 3n^2$$

ha soluzione in $\Theta(n^2)$.

Dimostrare le due disuguaglianze ($\leq cn^2$ e $\geq dn^2$) tali che valga esattamente la condizione theta

Soluzione: Per provare che $T(n) = O(n^2)$ dobbiamo dimostrare che $T(n) \leq cn^2$, per un'opportuna costante $c > 0$. Procediamo per induzione:

$$\begin{aligned} T(n) &= \frac{1}{4} T(n-1) + 3n^2 \\ &\leq \frac{1}{4} c(n-1)^2 + 3n^2 \\ &\leq \frac{1}{4} cn^2 + 3n^2 \\ &\leq cn^2 \end{aligned}$$

dove, per la validità dell'ultima disuguaglianza $\frac{1}{4} cn^2 + 3n^2 \leq cn^2$, occorre che

$$\frac{3}{4} cn^2 \geq 3n^2$$

ovvero, $c \geq 4$, con n qualunque.

Per provare $T(n) = \Omega(n^2)$ dobbiamo dimostrare che $T(n) \geq dn^2$, per un'opportuna costante $d > 0$. La prova per induzione non usa neppure l'ipotesi induttiva. Infatti

$$T(n) = \frac{1}{4} T(n-1) + 3n^2 \geq dn^2$$

purché $d \leq 3$.

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *oro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	<i>o</i>	<i>r</i>	<i>o</i>
	0	0	0
<i>a</i>	0	0	0
<i>r</i>	0	0	1
<i>m</i>	0	0	1
<i>o</i>	0	1	2

Esercizio 1 (9 punti) Realizzare una funzione `strongBST(T)` che dato un albero binario `T` con chiavi numeriche non negative, verifica se, per ogni nodo, la chiave è maggiore uguale del doppio di ogni chiave nel sottoalbero sinistro e minore o uguale della metà di ogni chiave nel sottoalbero destro, e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo `x` ha i campi `x.left`, `x.right` e `x.key`). Valutarne la complessità.

Soluzione:

```
strongBST(T)
    s, m, M = strongBSTRec(T.root)
    return v
```

StrongBST: Dato un albero, verifica se la chiave è $\geq 2 \times (\text{chiavi sott. sx})$ ed è $\leq (\text{chiavi sott. dx})/2$ e ritorna un booleano.

```
# strongBSTRec(x):
# verifica se il sottoalbero radicato in x e' uno strongBST e ritorna tre valori:
# - un booleano (che indica l'esito della verifica)
# - il massimo delle chiavi nel sottoalbero sx
# - il minimo delle chiavi nel sottoalbero dx

strongBSTRec(x)

if x = nil
    return true, 0, +infty
else
    # ispeziono i sottoalberi sinistro e destro
    sl, ml, Ml = strongBSTRec(x.left)
    sr, mr, Mr = strongBSTRec(x.right)

    # se la chiave del nodo in esame e' maggiore o uguale del doppio
    # del massimo delle chiavi nel sottoalbero sinistro (quindi di
    # tutte le chiavi nel sottoalbero sinistro) e minore o uguale della
    # meta' del minimo delle chiavi nel sottoalbero destro (quindi di
    # tutte le chiavi nel sottoalbero destro) delle chiavi dei
    # discendenti, ritorna true
    s = (x.key >= 2*Ml) and (x.key <= 1/2 mr)

    # calcola il minimo e il massimo delle chiavi nel sottoalbero radicato in x
    m = min { ml, mr, x.key }
    M = max { Ml, Mr, x.key }

    return s, m, M
```

Caso base: albero vuoto, verifica conduce a vero e il minimo è 0 e il massimo è +inf

L'idea è questa:
prima ricorro a `sx` e poi a `dx` salvando l'analisi in opportune variabili.

Uso le variabili appena considerate per esaminare se la chiave "`x`" è $\geq 2 \times (\text{chiavi sott. sx})$ e se la chiave "`x`" è $\leq (\text{chiavi sott. dx})/2$

A questo punto, vado a calcolare il minimo e massimo radicato nei rispettivi sottoalberi.

Costo totale: una singola visita completa dell'albero, quindi lineare.

Si tratta di una visita e quindi la complessità è $\Theta(n)$.

6 Tabelle Hash

Domanda 31 Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 12, 3, 22, 14, 38.

Doppio hash: $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$

Soluzione: Si ottiene

0	-
1	14
2	-
3	3
4	12
5	-
6	22
7	38

Nell'ordine:

1) $h(12, 0) = (12 \bmod 8) + 0 * (1 + 12 \bmod 6) = 4$

Inseriamo quindi 12 in posizione 4

2) $h(3, 0) = (3 \bmod 8) + 0 * (1 + 3 \bmod 6) = 3$

Inseriamo quindi 3 in posizione 3

3) $h(22, 0) = (22 \bmod 8) + 0 * (1 + 22 \bmod 7) = 6$

Inseriamo quindi 22 in posizione 6

4) $h(14, 0) = (14 \bmod 8) + 0 * (1 + 14 \bmod 6) = 6$

Collisione --> incremento il tentativo "i" di 1

$h(14, 1) = (14 \bmod 8) + 1 * (1 + 14 \bmod 6) = 9 \bmod 8 = 1$

Inseriamo 14 in posizione 1

5) $h(38, 0) = (38 \bmod 8) + 0 * (1 + 38 \bmod 6) = 6$

Collisione --> incremento il tentativo "i" di 1

$h(38, 1) = (38 \bmod 8) + 1 * (1 + 38 \bmod 6) = 9 \bmod 8 = 1$

Altra collisione --> incremento il tentativo "i" di 2

$h(38, 2) = (38 \bmod 8) + 2 * (1 + 38 \bmod 6) = 12 \bmod 8 = 4$

Altra collisione --> incremento il tentativo "i" di 3

$h(38, 3) = (38 \bmod 8) + 3 * (1 + 38 \bmod 6) = 15 \bmod 8 = 7$

Domanda 32 Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + 2 * (k \bmod (m - 3))$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 34, 12, 18, 9, 42.

Soluzione: Si ottiene

Gli altri sono svolti nella stessa maniera
(quelli col doppio hash --> 32/33)

0	-
1	18
2	34
3	9
4	12
5	-
6	-
7	42

Domanda 33 Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 14, 22, 10, 16, 8.

Soluzione: Si ottiene

0	16
1	8
2	10
3	22
4	-
5	-
6	14
7	-

Domanda 34 Si consideri una tabella hash di dimensione $m = 8$, gestita mediante chaining (liste di trabocco) con funzione di hash $h(k) = k \bmod m$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 14, 10, 22, 18, 19.

Soluzione: Si ottiene

0		
1		
2		$18 \rightarrow 10$
3		19
4		
5		
6		$22 \rightarrow 14$
7		

Quindi:

$14 \bmod 8 = 6$

Inserisco 14 in posizione 6

$10 \bmod 8 = 2$

Inserisco 10 in posizione 2

$22 \bmod 8 = 6$

Essendo chaining, concatenato nella posizione 6 una lista che incorpora come primo elemento il 22

$18 \bmod 8 = 2$

Inseriamo 18 in posizione

Essendo chaining, concatenato nella posizione 2 una lista che incorpora come primo elemento il 18

$19 \bmod 8 = 3$

Inseriamo 19 in posizione 3

Domanda 35 Si consideri una tabella hash di dimensione $m = 9$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 12, 3, 22, 14, 38.

Soluzione: Si ottiene

Svolto come sopra
(es. 31 di riferimento)

0	-
1	-
2	38
3	12
4	22
5	14
6	-
7	3
8	-

7 Alberi Binari di Ricerca e Red Black Trees

Domanda 36 Realizzare una funzione $\text{pred}(x)$ che dato in input un nodo x , di un albero binario di ricerca T , restituisce il predecessore di x (oppure nil, se il predecessore non esiste). Come a lezione, supporre che ogni nodo abbia i campi $x.\text{left}$, $x.\text{right}$, $x.p$, $x.\text{key}$.

Soluzione:

```

pred(x)
  if x.left <> nil
    return max (x.left)
  else
    y = x.p
    while (y <> nil) and (x == y.left)
      x=y
      y=y.p
    return y

max (x)
  while x.right <> nil
    x = x.right
  return x

```

Quanto presente è la classica funzione "predecessore", quindi il nodo precedente incontrato in una visita InOrder dell'albero.
Quindi:
- se l'albero sx dovesse essere nullo, allora corrisponde al massimo di quel sottoalbero, quindi il nodo più a destra oppure il figlio dx stesso
- se così non fosse, allora, salvo il parent del nodo attuale in "y" e finché esiste il parent, continuo a cercare ricorsivamente a sinistra, cercando l'estremo inferiore (il nodo più grande tra tutti i più piccoli)-
- se non trovassi nulla a sx , allora comincio a cercare ricorsivamente a dx e una volta trovato, questo è il predecessore

Domanda 37 Scrivere una funzione $\text{IsABR}(A)$ che dato in input un array di interi $A[1..n]$, interpretato come albero binario (come nel caso degli heap, ogni $A[i]$ è un nodo con figlio sinistro e destro $A[2i]$ e $A[2i+1]$) verifica se A è un albero binario di ricerca. Valutarne la complessità.

Soluzione: Una versione iterativa, di tempo $O(n \log n)$ consiste nel verificare che ogni nodo sia minore o uguale dei nodi dei quali è nel sottoalbero sinistro e maggiore o uguale di quelli in cui è nel sottoalbero destro

```

IsABR(A)
  n = A.length
  i = n
  isABR = true
  while ((i > 1) and isABR)
    j = i
    while j > 1
      if even(j)
        # i nodi di indice pari sono figli sx
        isABR = isABR and A[i] <= A[j/2]
      else
        # quelli di indice dispari figli dx
        isABR = isABR and A[i] >= A[j/2]
      j = j/2
    i = i-1
  return isABR

```

Per verificare se l'array è organizzato ad albero, considero la lunghezza dell'array e da lì inizializzo l'indice che parte dal punto finale di questo albero e da lì risalgo iterativamente e:
- se risalendo, incontro un nodo con indice pari (caso $A[2i]$) è figlio sx (se rispetta anche la proprietà simile a max heap, $A[i] \leq A[i/2]$)
- se risalendo, incontro un nodo con indice pari (caso $A[2i+1]$) è figlio dx (se rispetta anche la proprietà simile a max heap, $A[i] \geq A[i/2]$)
risale l'albero
i nodi di indice pari sono figli sx
quelli di indice dispari figli dx
Decremento i corrispettivi indici e continuo in questo modo; questa scansione esamina due metà di array alla max-heap e costa $O(n \log n)$.

E varie versioni ricorsive, la prima per ogni nodo verifica che sia maggiore o uguale al massimo del sottoalbero destro e minore o uguale del minimo del sottoalbero destro. Dato che prima controlla che tali sottoalberi siano ABR può cercare minimo e massimo in tempo $\log n$ cosa che assicura che il costo complessivo è lineare. Infatti $T(n) = 2T(n/2) + 2 \log n$ che secondo il master theorem porta a $T(n) = \Theta(n)$.

```

IsABRO(A,n)
    return IsABRO_rec(A,n,1)

IsABRO_rec(A,n,i):
    if i > n
        return True
    else:
        isABRL = IsABRO_rec(A,n,2*i)
        isABRR = IsABRO_rec(A,n,2*i+1)
        M = max(A,n,2*i)
        m = min(A,n,2*i+1)

        return isABRL and isABRR
                and A[i] >= m
                and A[i] <= M

```

Altre varianti ricorsive:

1) la prima, che sfrutta una procedura ricorsiva ausiliaria, semplicemente chiama sé stessa sfrutta [2i] per il figlio sx e [2i + 1] per il figlio dx.
A quel punto verifica, essendo ABR, se questi nodi siano rispettivamente minimo e massimo (perché ho ricorsivamente esaminato tutto l'albero); nel caso lo fosse, ritorno i booleani e gli indici esaminati.

massimo del sottoalbero sx (assumendolo ABR, già controllato!)

minimo del sottoalbero dx (assumendolo ABR, già controllato!)

Un'altra versione "naïf" (semplice e primitiva):
essendo un ABR, esamino linearmente il mio array e verifico se, scendendo a sx, ogni elemento rispetta la proprietà di figlio sx (quindi, ha indice [2*i].
Se così fosse, sicuramente l'albero conserva anche l'altra proprietà di albero binario (massimi a dx, grazie alla correttezza delle proprietà) e così, ritorno l'indice esaminato.

```

Tmin(A,n,i):
    if i>n:
        return +INFTY
    else:
        while (2*i <=n):
            i = 2*i      # scende a sx

        return A[i]

```

Si può evitare il ciclo per cercare minimo e massimo tenendo conto conto che il minimo corrisponde a $A[m]$ con m massimo valore $\leq n$ ottenuto moltiplicando i per 2, ovvero $m = i * 2^k \leq n$, con h massimo quindi

$$h = \lfloor \log_2 n/i \rfloor$$

e similmente, il massimo corrisponde a $A[M]$ con M ottenuto da i moltiplicando per 2 e sommando 1, iterativamente, ma senza superare n . Osservando che l'operazione suddetta iterata k , ovvero $(i * 2 + 1) * 2 + 1 \dots$ fatto k volte, conduce a $i * 2^k + 1 + 2 + \dots + 2^{k-1}$, si ha $M = i * 2^k + 2^k - 1 = 2^k(i + 1) - 1 \leq n$ con k massimo, per cui

$$k = \lfloor \log_2(n + 1)/(i + 1) \rfloor$$

Le altre di costo lineare:

soluzione di tempo lineare

```

IsABR1(A,n)
    isABR,m,M = IsABR1_rec(A,n,1)
    # ritorna tre valori per il sottoalbero radicato in A[i]
    # - e' ABR? (true/false)
    # - minimo
    # - massimo

    return isABR

IsABR1_rec(A,n,i):
    if i > n

```

```

    return True, INFY, -INFY
else:
    isABRL, mL, ML = IsABR1_rec(A,n,2*i)
    isABRR, mR, MR = IsABR1_rec(A,n,2*i+1)

    return isABRL and isABRR
           and A[i] >= ML
           and A[i] <= mR,
           min(mL, mR, A[i]),
           max(ML, MR, A[i])

// altra soluzione di tempo lineare
// effettua una visita e verifica simmetrica e che produca l'array ordinato

IsABR2(A,n)
    isABR, last = visitAndCheck(A,1,-INFY)
                                // visita simmetrica con verifica che il prossimo
                                // valore incontrato sia inferiore al precedente
                                // ritorna due valori
                                //    ok
                                //    ultimo valore prodotto

    return isABR

visitAndCheck(A,i,last)
    if i > n
        return True, last
    else:
        ok_left, last = visitAndCheck(A,2*i,last)
        // conceptually, now visit A[i]
        ok = last <= A[i]
        ok_right, last = visitAndCheck(A,2*i+1,A[i])
        return ok_left and ok and ok_right, last

```

Codice di prima ottimizzato verificando:

- che l'indice di sx sia effettivamente $[2*i]$
- che l'indice di dx sia effettivamente $[2*i+1]$
- che gli indici rispettino le corrispondenti proprietà a sx (parent $\geq$ figlio sx e parent $\leq$ figlio dx) e che si trovino min e max dei corrispettivi alberi, elementi più a sx e più a dx dell'albero considerato.

Questo algoritmo:

- verifica di poter ancora esaminare gli indici
- visita a sinistra verificando che ogni nodo sia figlio sx (indice $[2*i]$)
- ogni nodo parent è $\geq$ dei nodi di sx e così deve essere anche per l'ultimo
- fatto questo, visita a destra verificando che ogni nodo sia figlio dx (indice $[2*i+1]$) e verificando con un booleano se il parent $A[i]$ è $\leq$ dei nodi di dx, mettendo true se così fosse, tale da essere albero binario

Fatto tutto ciò, restituisce le variabili booleane corrispondenti.

Domanda 38 Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno un campo $x.pred$ (predecessore) invece che il campo $x.p$ (parent). Realizzare la procedura di $Insert(T,z)$ che inserisce un nodo z nell'albero. Valutarne la complessità.

Soluzione: La soluzione con complessità $O(h)$ dove h è l'altezza dell'albero può essere:

```

Insert(T,z)
    x = T.root
    y = nil          // father
    w = nil          // successor
    while (x <> nil)
        y = x

        if z.key < x.key
            x = x.left
            w = y
        else
            x = x.right

```

L'insert per l'ABR ragiona sugli stessi principi già discussi:

- creare la radice dell'albero
- e impostare la copia dell'elemento attuale a nil
- finché l'albero esiste (creiamo l'albero rispetto a z)
- ci copiamo la radice
- confronto "z" ed "x"
- se "z" è minore di "x", ci creiamo il nodo sinistro e salviamo come successore il nodo padre "y"
- se "z" è maggiore di "x", ci creiamo il nodo destro e il parent del nodo "z" da inserire sarà "y"

```

z.p = y          // omit if 'parent' is not a field

if y = nil
    T.root = z
else if z.key < y.key
    y.left = z
    z.prec = y.prec
else
    y.right = z
    z.prec = y

if w <> nil
    w.prec = z

```

Una volta creato l'albero:

- se l'albero è nullo, la radice è "z"
- confronto "z" ed "x"
- se "z" è minore di "x", il nodo sinistro sarà "z" e il predecessore del nodo è quello della radice "y", avendo appena messo il nodo sx
- se "z" è maggiore di "x", il nodo destro sarà "z", considerando che se siamo a dx, siamo già stati nel nodo radice e il predecessore è quindi "y"
- se il successore fosse nullo, si considera che il predecessore sia il nodo appena inserito, tale che "w" sia un successore nullo (campo w.succ = nil)

Esercizio 14 Realizzare una procedura $BST(A)$ che dato un array $A[1..n]$ di interi, ordinato in modo crescente, costruisce un albero binario di ricerca di altezza minima che contiene gli elementi di A e ne restituisce la radice. Per allocare un nuovo nodo dell'albero si utilizzi una funzione $mknode(k)$ che dato un intero k ritorna un nuovo nodo con $x.key=k$ e figlio destro e sinistro $x.left = x.right = nil$. Valutarne la complessità.

Soluzione:

i. L'implementazione è la seguente:

```

BST(A)
    return BST-rec(A,1,n)

BST-rec(T,A,p,q)
    if p <= q
        m = floor((p+q)/2)
        x=mknode(A[m])
        x.l = BST-rec(A,p,m-1)
        x.r = BST-rec(A,m+1,q)
    else
        x = nil

    return x

```

Idea che sfrutta la base di merge sort, considerando però una singola visita dell'albero completa, per crearsi un BST.

Quindi, sfrutta i due estremi "1" ed "n" e considera che:

- finché l'indice della prima metà è minore della seconda, considero l'indice medio ottimale
- creo un nodo con indice medio ottimale
- considero il nodo di sx, decrementando sempre "alla mergesort" (quindi, l'estremo dx per restringere la partizione andando indietro verso sx)
- considero il nodo di dx, incrementando sempre "alla mergesort" (quindi, l'estremo medio per restringere la partizione andando avanti verso dx)

Se così non fosse, abbiamo creato tutto l'albero, oppure non ci sono abbastanza elementi e ritorno il nodo nullo.

Tutto ciò, come detto, ha complessità lineare.

ii. Si ottiene la ricorrenza $T(n) = 2T(n/2) + \Theta(1)$ e quindi il costo è $T(n) = O(n)$.

Esercizio 15 Si consideri una estensione degli alberi binari di ricerca nei quali ogni nodo x ha anche un campo booleano $x.even$ che vale *true* o *false* a seconda che la somma delle chiavi nel sottoalbero radicato in x sia pari o dispari. Realizzare la procedura $Insert(T,z)$ di modo che mantenga correttamente aggiornato anche il campo *even*. Valutarne la complessità.

Soluzione:

```

Insert(T,z)
    x = T.root
    y = nil
    z.even = (z.key mod 2 == 0)

```

Even = Verifica se la somma delle chiavi sia pari o dispari (dato che l'inserimento avviene rispetto a "z", allora il campo pari sarà inizializzato sulla base del valore di "z", quindi facendo il modulo e capendo se pari o dispari per le somme successive)

```

while (x <> nil)
    x.even = (x.even == z.even)    // z e' inserito nel sottoalbero radicato
    // in x, quindi si aggiunge alla somma
    // che sara' pari se la somma precedente
    // e z.key sono entrambi pari o entrambi
    // dispari e sara' dispari altrimenti
    In poche parole:
    dovendo inserire "z", la radice, a seconda
    che la sua chiave sia pari o dispari,
    si verifica immediatamente se la radice
    abbia somma di "z" e del nodo stesso radice
    pari o dispari
    y = x

    if z.key < x.key
        x = x.left
    else
        w = y
        x = x.right

z.p = y
if y = nil
    T.root = z
else if z.key < y.key
    y.left = z
else
    y.right = z

```

La complessità è $\Theta(h)$.

Esercizio 16 Scrivere una funzione `range(T,k1,k2)` che dato un albero binario di ricerca T e due interi k_1 e k_2 tali che $k_1 \leq k_2$, stampa, in ordine crescente, tutte le chiavi k contenute nell'albero tali che $k_1 \leq k \leq k_2$. Valutarne la complessità.

Soluzione:

```

range(T, k1, k2)
    range_rec(T.root, k1, k2)

range_rec(x,k1,k2)
    if x <> nil
        if x.key >= k1
            range_rec(x.left, k1, k2)

            if x.key >= k1 and x.key <= k2
                print x.key

            if x.key <= k2
                range_rec(x.right, k1, k2)

```

Passando gli estremi inf e sup ad una funzione ricorsiva di supporto, abbiamo che:

- se il nodo radice non è nullo
- se la chiave di "x" è $\geq k_1$, allora abbiamo che tutti i nodi di "x" saranno minori della chiave e ricorriamo a sx
- se la chiave di "x" è $\leq k_2$, allora abbiamo che tutti i nodi di "x" saranno maggiori della chiave e ricorriamo a dx

Esercizio 17 Scrivere una funzione `RBTree(T)` che dato in input un albero binario di ricerca T , i cui nodi x , oltre ai campi $x.key$, $x.left$ e $x.right$, hanno un campo $x.col$ che può essere B (per "black") oppure R (per "red"), verifica se questo è un Red-Black tree. In caso negativo, restituisce -1 , altrimenti restituisce l'altezza nera della radice. Valutarne la complessità.

Soluzione:

```

RBTree(T) // verifica se T e' un RB-tree
    if T.root == T.nil
        return 0
    elif T.root.color == R
        return -1
    else
        return RB-rec(T.root)

RB-rec(x) // calcola l'altezza nera di x e la restituisce se
    - le altezze nere dei figli sono uguali
    - se il nodo e' R i figli sono B
    - le foglie dei sottoalberi sono B
    altrimenti restituisce -1

if x == T.nil      // foglia, T.nil e' automaticamente nero
    return 0

elif (x.color == R) and ((x.l.color == R) or (x.r.color == R))
    return -1      // similmente, verifichiamo che i figli abbiano colori alterni
                    // (se figlio sinistro, avrà colore rosso e almeno uno dei figli è rosso, per non avere
                    // due nodi con lo stesso colore vicini)

else
    left = RB-rec(x.left)    // Similmente, ricorriamo a sinistra e a destra
    right = RB-rec(x.right)  // e verifichiamo, dato che siamo nelle foglie (nodi neri)
                             // se abbiano la stessa altezza nera e se siano foglie; se così non fosse si ritorna

    // se uno dei due sottoalberi non soddisfa le richieste
    // oppure entrambi i sottoalberi le soddisfano ma
    // sono non vuoti e con altezze nere diverse

    if (right == -1) or (left == -1) or (left <> right)
        count = -1
    else
        count = left
        if x.col == B
            count = count + 1

        // Se hanno la stessa altezza nera, salviamo quella di sx (dato che hanno la
        // stessa altezza, mi basta salvare una delle due, quella di sx dato che almeno una
        // volta ricorsivamente viene calcolata) e, se il colore di "x" fosse nero, "x" è foglia
        // e va sommato questo fatto nel calcolo dell'altezza (mancava un livello,
        // ora siamo nella foglia e ritorno l'altezza calcolata).

    return count

```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n - k)$ per un'opportuna costante c e quindi, la complessità è $O(n)$.

Per quanto riguarda la complessità, la ricorrenza è del tipo $T(n) = T(n/2) + c$, dato che ad ogni chiamata si dimezza la dimensione dell'array e il costo della parte non ricorsiva è costante. Per il Master Theorem (con $a = 1$, $b = 2$, $f(n) = c = \Theta(n^{\log_b a}) = \Theta(n^0) = \Theta(1)$) si conclude $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$.

Esercizio 2 (9 punti) Sia n un intero positivo. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 2 & \text{se } i = j, \\ 3 & \text{(caso base) se } j = i + 1, \\ M(i+1, j-1) \cdot M(i+1, j) + M(i, j-1) & \text{se } j > i + 1. \end{cases}$$

(vedo (1,n) e so che la scansione è per diagonale)

- (a) Si scriva una coppia di algoritmi INIT_M(n) e REC_M(i, j) per il calcolo memoizzato di $M(1, n)$.
 (b) Si calcoli il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Soluzione:

(a) INIT_M(n)
 if $n=1$ then return 2
 if $n=2$ then return 3
 for $i=1$ to $n-1$ do
 $M[i, i] = 2$
 $M[i, i+1] = 3$
 $M[n, n] = 2$
 for $i=1$ to $n-2$ do
 for $j=i+2$ to n do
 $M[i, j] = 0$
 return REC_M(1, n)

Se gli indici sono uno solo o due soli, allora, seguendo i casi base si ritorna 2 e 3.

Ricordandosi che:

- "i" va da 1 ad ($n-1$), quindi diventa perché abbiamo già inizializzato, ($n-2$) la scansione
 - "j" va da ($n-1$) a 0 compresi, quindi dato che abbiamo inizializzato, parte da ($n-2$)

Una volta inizializzato secondo i casi base, tutto il resto della matrice viene riempita di zeri. Poi, si considera, essendo una scansione per diagonale, noi partiamo dal basso e riempiamo solo la parte sopra delle diagonali principali, ovviamente riempita anche questa di zeri.

REC_M(i, j)
 if $M[i, j] = 0$ then
 $M[i, j] = \text{REC_M}(i+1, j-1) * \text{REC_M}(i+1, j) + \text{REC_M}(i, j-1)$
 return $M[i, j]$

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 = \sum_{i=1}^{n-2} (n - i - 1) = \sum_{k=1}^{n-2} k = (n-2)(n-1)/2$$

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra tre numeri interi, nello specifico ($n-2$), ($n-1$), n .

Quanti uno?

Beh, da $j=i+2$ a n ci sono esattamente $n-(i+2)+1=n-i-1$ termini (sostituisco j in i , perché la serie è basata su i e il $+1$ si ha per il fatto che $i=1$). Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$. Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di ($n-2$)($n-1$) e quindi il risultato è proprio ($n-2$)($n-1$)/2

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbb{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

Si capisce che si scansione in ordine decrescente di colonna perché quando "i" vale 0, allora "j" vale qualcosa

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j-1) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

- (a) Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = +infinito
for i=1 to n-1 do
    C[i,n-1] <- a_i
    m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
    C[0,j] <- b_j
    m <- MIN(m,C[0,j])
for i=1 to n-2 do
    for j=n-2 downto i do
        C[i,j] <- C[i-1,j-1] * C[i,j+1]
        m <- MIN(m,C[i,j])
return m

```

(ci prendiamo la lunghezza dell'array e inizializziamo il minimo ad infinito (così, qualsiasi prima quantità minima sarà più piccola))

Siccome devo salvare il minimo, ogni volta si fa il confronto tra il minimo attuale e l'indice di C[i, j] appena salvato.

Ricordandosi che:

- "i" va da i ad (n-1), quindi diventa perché abbiamo già inizializzato, (n-2) la scansione
- "j" va da (n-1) a 0 compresi, quindi dato che abbiamo inizializzato, parte da (n-2)

Siccome nuovamente abbiamo un ciclo in cui "i" va ad (n-2) e in quello di inizializzazione andava ad (n-1), la sommatoria è data da (n-2)(n-1)/2

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra tre numeri interi, nello specifico (n-2), (n-1), n.

Quanti uno?

Beh, da $j=i+2$ a n ci sono esattamente $n-(i+2)+1=n-i-1$ termini (sostituisco j in i , perché la serie è basata su i e il $+1$ si ha per il fatto che $i=1$). Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$.

Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di (n-2)(n-1) e quindi il risultato è proprio (n-2)(n-1)/2

Uguale a quello precedente, ma abbiamo a che fare con MAX piuttosto che MIN

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n-1\}$. $M(n, 1)$

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```
COMPUTE(a,b)
n <- length(a)
M = -infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  M <- MAX(M,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  M <- MAX(M,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j] * C[i,j+1]
    M <- MAX(M,C[i,j])
return M
```

(ci prendiamo la lunghezza dell'array e inizializziamo il minimo a meno infinito (così, qualsiasi prima quantità massima sarà più grande)

Ricordandosi che:
- "i" va da 1 ad (n-1), quindi diventa perché abbiamo già inizializzato, (n-2) la scansione
- "j" va da (n-1) a 0 compresi, quindi dato che abbiamo inizializzato, parte da (n-2)

Si come devo salvare il massimo, ogni volta si fa il confronto tra il massimo attuale e l'indice di C[i, j] appena salvato.

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra due numeri interi, nello specifico (n-2), (n-2) (apici della serie).

Quanti uno?

Beh, da $j=n-2$ a i ci sono esattamente $n-i-1 = n-1-1$ termini (sostituisco j in i , perché la serie è basata su i e sottraggo 1 in quanto $i=1$ ed i sarebbe il valore di j ($j=i$ è il pedice della seconda serie)).

Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$ quando si abbia come qui la serie con termine generale k (serie di Gauss).

Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di $(n-2)(n-1)$ e quindi il risultato è proprio $(n-2)(n-1)/2$

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

(vedo $(1, n)$ e so che la scansione è per diagonale)

Soluzione:

1. Pseudocodice:

```
INIT_L(X)
  n <- length(X)
  if n = 1 then return 1
  if n = 2 then return 2
  for i=1 to n-1 do
    L[i,i] <- 1
    L[i,i+1] <- 2
  L[n,n] <- 1
  for i=1 to n-2 do
    for j=i+2 to n do
      L[i,j] <- 0
    return REC_L(X,1,n)
```

Prendo la lunghezza della stringa iniziale

Similmente, anche $n=1$ e $n=2$ vengono dai due casi base (array di 1/2 elementi max), quindi o ha un elemento oppure ne ha due soli

(ultimo elemento diagonale inizializzato)

Una volta inizializzato secondo i casi base, tutto il resto della matrice viene riempita di zeri. Poi, si considera, essendo una scansione per diagonale, noi partiamo dal basso e riempiamo solo la parte sopra delle diagonali principali, ovviamente riempita anche questa di zeri.

```
REC_L(X,i,j)
  if L[i,j] = 0 then
    if x_i = x_j then L[i,j] <- 2 + REC_L(X,i+1,j-1)
    else for k=i to j-1 do
      L[i,j] <- L[i,j] + REC_L(X,i,k) + REC_L(X,k+1,j)
  return L[i,j]
```

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

$(1, n), (2, n - 1), \dots, (k, n - k + 1)$, ognuno associato a un costo unitario. Ci sono al più $\lfloor n/2 \rfloor$ di queste chiamate, e quindi $T_{\text{best}}(n) = O(n)$.

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ z(i-1, j) \cdot z(i, j+1) \cdot z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up $Z(A)$ che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$. (vedo $(n, 1)$ e la scansione è per colonna)
2. Si valuti il numero esatto $T_Z(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

Reverse column major:

a11	a12	a13
a21	a22	a23
a31	a32	a33

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione "reverse column-major", in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
Z(A)
n = length(A)
for i=1 to n do
    z[1,i] = a_i
    z[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        z[i,j] = z[i-1,j] * z[i,j+1] * z[i-1,j+1]
return z[n,1]
```

Piuttosto che usare per l'inizializzazione due indici, se ne usa solo uno.
Quando si ha "j", si sa che "i" vale 1, da cui i=j e quindi [1, i]
Quando invece si ha "n+1-i" come indice, si vede che "j" è =n e quindi
avremo che per [i, n] avremo [n+1-i].

Vedendo che "j" parte da n e invece "i" parte da 1,
per effettuare una scansione giusta per colonne (avendo che
la prima colonna/riga è stata inizializzata dal caso base), allora
"j" parte da (n-1) piuttosto che da (n) e "i" parte da 2 piuttosto che da (1)

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione "reverse diagonal", che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $z[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due moltiplicazioni tra interi, e quindi

$$\begin{aligned} T_Z(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

La sommatoria interna è costituita da soli 2, in quanto richiede due moltiplicazioni tra due numeri interi, nello specifico $(n-1)$ ed n (apici della serie). Beh, da $i=2$ ad n . Quindi, dovendo sostituire i in j (essendo la serie basata sull'indice più esterno), si ha che $n-1$ semplicemente (in quanto, l'indice j è pari ad 1 e il 2 verrebbe portato poi fuori per linearità).
Ora, siccome abbiamo una serie che ha come apice $(n-1)$, non possiamo esprimere la soluzione in questo modo, dato che $(n-1)$ compare sia a pedice che dentro la serie. Dobbiamo quindi esprimere la serie in termini di j . Ciò comporta che io vada a "portare dentro" $\leftarrow(n-1\rightarrow$ nella serie, in quanto moltiplicheremmo implicitamente $2(n-1)$ per altre $(n-1)$ volte e quindi è come se andassi a fare $2(n-1)(n-2) = 2(n-1)^2$

Esercizio 2 (9 punti) Sia $n > 0$ un intero. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 1 & \text{se } i = j, \\ 2 & \text{se } j = i + 1, \\ \underline{M(i+1, j-1) \cdot M(i+1, j) \cdot M(i, j-1)} & \text{se } j > i + 1. \end{cases}$$

1. Scrivere una coppia di algoritmi INIT_M(n) e REC_M(i, j) per il calcolo memoizzato di $M(1, n)$.
2. Calcolare il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Soluzione:

1. Pseudocodice:

```
INIT_M(n)
if n=1 then return 1
if n=2 then return 2
for i=1 to n-1 do
    M[i,i] = 1
    M[i,i+1] = 2
M[n,n] = 1      (ultimo elemento diagonale inizializzato)
for i=1 to n-2 do
    for j=i+2 to n do
        M[i,j] = 0
return REC_M(1,n)
```

Siccome viene già passata la lunghezza, non serve salvarla.
Similmente, anche $n=1$ e $n=2$ vengono dai due casi base (array di 1/2 elementi max), quindi o ha un elemento oppure ne ha due soli

Ricordandosi che:
- "i" va da 1 ad $(n-1)$, quindi diventa perché abbiamo già inizializzato, $(n-2)$ la scansione,
- "j" va da $(n-1)$ a 0 compresi, quindi dato che abbiamo inizializzato, parte da $(n-2)$

Una volta inizializzato secondo i casi base, tutto il resto della matrice viene riempita di zeri. Poi, si considera, essendo una scansione per diagonale, noi partiamo dal basso e riempiamo solo la parte sopra delle diagonali principali, ovviamente riempita anche questa di zeri.

```
REC_M(i,j)
if M[i,j] = 0 then
    M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) * REC_M(i,j-1)
return M[i,j]
```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 = 2 \sum_{i=1}^{n-2} n - i - 1 = 2 \sum_{k=1}^{n-2} k = (n-2)(n-1)$$

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente

ricorrenza $c(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$c(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ \underline{c(i-1, j) \cdot c(i, j+1) \cdot c(i-1, j+1)} & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up COMPUTE_C(A) che, data in input la stringa A restituisca in uscita il valore $c(n, 1)$. (vedo $(1, n)$ e so che la scansione è per diagonale)
2. Si valuti il numero esatto $T_{CC}(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

La sommatoria interna è costituita da soli 2, in quanto richiede due moltiplicazioni tra tre numeri interi, nello specifico $(n-2)$, $(n-1)$, n . Quanti due? Beh, da $j=i+2$ a n ci sono esattamente $n-(i+2)+1=n-i-1$ termini (sostituisco j in i , perché la serie è basata su i).

Quindi sarebbe sommatoria di $2(n-i-1)$, poi si può portare fuori il 2 per linearità.

Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$ quando si abbia come qui la serie con termine generale k (serie di Gauss).

Infatti, la sommatoria da 1 a $n-2$ è $(n-2)(n-2+1)/2$, semplificando diventa quello che vedi

Il 2 come argomento della serie deriva dal fatto che ogni termine $M[i, j]$ richiede due moltiplicazioni tra 3 numeri interi.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
COMPUTE_C(A)
n = length(A)
for i=1 to n do
    c[1,i] = a_i
    c[i,n] = a_{n+1-i}
    for j=n-1 downto 1 do
        for i=2 to n do
            c[i,j] = c[i-1,j] * c[i,j+1] * c[i-1,j+1]
    return c[n,1]
```

Dato che non viene passata la lunghezza dell'array, conviene salvarla nella variabile "n" per riuscire ad iterare.

Ricordandosi che:
- "j" va da n ad 1, quindi diventa perché abbiamo già inizializzato, (n-1) la scansione
- "i" va da 1 a n compresi, quindi dato che abbiamo inizializzato, parte da 2

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonali parallele alla diagonale principale partendo da quella contenente solo $c[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due operazioni tra interi, e quindi

$$\begin{aligned} T_{CC}(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

La sommatoria interna è costituita da soli 2, in quanto richiede due moltiplicazioni tra due numeri interi, nello specifico (n-1) ed n (apici della serie). Beh, da $i=2$ ad n. Quindi, dovendo sostituire i in j (essendo la serie basata sull'indice più esterno), si ha che $n-1$ semplicemente (in quanto, l'indice j è pari ad 1 e il 2 verrebbe portato poi fuori per linearità). Ora, siccome abbiamo una serie che ha come apice (n-1), non possiamo esprimere la soluzione in questo modo, dato che (n-1) compare sia a pedice che dentro la serie. Dobbiamo quindi esprimere la serie in termini di j. Ciò comporta che io vada a “portare dentro” $(n-1)$ nella serie, in quanto moltiplicheremmo implicitamente $2(n-1)$ per altre (n-1) volte e quindi è come se andassi a fare $2(n-1)(n-2) = 2(n-1)^2$.

```

else
    L[j] = A[j]

max = A[1]
for j=2 to n
    if L[j]>max
        max = L[j]

return max

```

Se vogliamo anche il sottoarray, occorre ricordare il massimo e dove viene raggiunto, lo facciamo mediante un array $S[1..n]$ tale che $S[j]$ contenga l'indice dal quale inizia il sottoarray massima che termina con $A[j]$.

```

maxsum (A, n)
    L[1] = A[1]
    S[1] = 1

    for j=2 to n
        if (L[j-1] > 0)
            L[j] = L[j-1] + A[j]
            S[j] = S[j-1]
        else
            L[j] = A[j]
            S[j] = j

    max = 1
    for j=2 to n
        if L[j]>max
            max = j

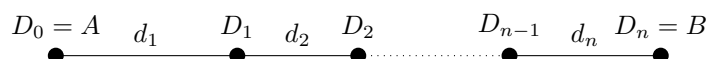
    # returns the first and last index of the substring
    return S[max], max

```

La complessità è $\Theta(n)$.

9 Greedy

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a d km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq d$) come indicato in figura



L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B, minimizzando il numero di soste ai distributori per il rifornimento.

- i. Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.

- ii. Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy `stop(d,n)` che dato in input l'array delle distanze `d[1..n]` restituisce una soluzione ottima.
- iii. Valutare la complessità dell'algoritmo.

Soluzione: La scelta ottima del problema consiste nel trovare un insieme di soste tali che la loro lunghezza sia minima.

Soluzione. In generale, la specifica del problema consiste in una sequenza di soste possibili $D_0(=A), D_1, \dots, D_n(=B)$. Per una coppia di soste D_i, D_j con $i \leq j$ poniamo $d_{i,j} = \sum_{h=i+1}^j d_h$. Quindi una soluzione del problema è una sottosequenza di soste che porta dal punto iniziale al punto finale, senza percorrere tratti di lunghezza maggiore di d ovvero:

$$S = D_{i_0} \dots D_{i_k}$$

(con $i_0 < i_1 < \dots < i_k$) tali che $D_{i_0} = A$ e $D_{i_k} = B$, per ogni $j \in \{0, \dots, k-1\}$ vale $d_{i_j, i_{j+1}} \leq d$. Il costo $c(S)$ è il numero $k-1$ di soste.

Sottostruttura ottima. Sia $S = D_{i_0} \dots D_{i_k}$ una soluzione ottima per il problema $D_0, D_1, \dots, D_n$. Se consideriamo il sottoproblema di andare da D_{i_1} a D_n , ovvero $D_{i_1}, D_{i_1+1} \dots D_n$, allora $S_1 = D_{i_1}, \dots, D_{i_k}$ è una soluzione ottima. Infatti, è chiaramente una soluzione. Inoltre, se ci fosse una soluzione migliore S'_1 , con un numero inferiore di soste per il sottoproblema, ovvero $c(S'_1) < c(S_1)$, aggiungendo la sosta D_{i_0} otterremmo una soluzione S' migliore di S per il problema originale dato che $c(S') = c(S'_1) + 1 < c(S_1) + 1 = c(S)$.
Significa che, avendo scelto bene prima, aggiungendo altre soste, la scelta rimane minima.

Scelta greedy. Per il problema $D_0, D_1, \dots, D_n$, si fissa necessariamente $i_0 = 0$, e la scelta greedy consiste nel raggiungere la sosta più lontana a distanza minore o uguale di d , ovvero definire $i_1 = \max\{j \mid d_{0,j} \leq d\}$.

Data una soluzione ottima per il problema $S = D_{j_0} \dots D_{j_k}$, certamente $j_0 = 0 = i_0$ e $j_1 \leq i_1$. Quindi è immediato verificare che anche $S' = D_{j_0} D_{i_1} D_{j_2} \dots D_{j_k}$ è una soluzione per il problema $D_0, D_1, \dots, D_n$, ed è ottima, dato che $c(S') = c(S)$. (Si noti che, più precisamente, in prima istanza si potrebbe pensare che D_{i_1} potesse essere anche oltre D_{j_2} , ma questo darebbe una soluzione migliore di quella ottima, portando ad un assurdo).

Algoritmo. Ne segue l'algoritmo che riceve in la sequenza di distanze nella forma di un array `d[1..n]` e restituisce un array `S[1..n-1]` con le soste scelte (la prima e l'ultima sono scelte sempre, non serve indicarlo).

```
stop(d, n)
  dist = d[1]           // distanza percorsa
  for i=2 to n
    if dist + d[i] > d
      S[i-1]=1
      dist=d[i]
    else
      S[i-1] = 0
  return S
```



Per scelta greedy, prendiamo la sosta con distanza $\leq$ a quella attuale, tale da capire caso per caso quale conviene di più.

L'algoritmo ricalca proprio la selezione delle attività e:
 - prende come distanza la prima, per inizializzazione
 - se la somma tra la prima distanza e quella attuale è $>$ della sequenza delle distanze, allora abbiamo la distanza buona e salviamo la sosta con la distanza ottima (sapendo che ci siamo fermati "prima", quindi $S[i-1]=1$)
 - se invece non ci siamo fermati prima, avremo $S[i-1] = 0$

La complessità è $\Theta(n)$.

Esercizio 34 L'ufficio postale offre un servizio di ritiro pacchi in sede su prenotazione. Il destinatario, avvisato della presenza del pacco, deve comunicare l'orario preciso al quale si recherà allo

sportello. Sapendo che gli impiegati dedicano a questa mansione turni di un'ora, con inizio in un momento qualsiasi, si chiede di scrivere un algoritmo che individui l'insieme minimo di turni di un'ora sufficienti a soddisfare tutte le richieste. Più in dettaglio, data una sequenza $\vec{r} = r_1, \dots, r_n$ di richieste, dove r_i è l'orario della i -ma prenotazione, si vuole determinare una sequenza di turni $\vec{t} = t_1, \dots, t_k$, con t_j orario di inizio del j -mo turno, che abbia dimensione minima e tale che i turni coprano tutte le richieste.

- i. Formalizzare la nozione di soluzione per il problema e il relativo costo. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- ii. Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy `time(R, n)` che dato in input l'array delle richieste `r[1..n]` restituisce una soluzione ottima.
- iii. Valutare la complessità dell'algoritmo.

L'idea è di crearsi un array/sequenza di turni tale che siano con un costo "k" minore possibile.

Soluzione: Una soluzione è una sequenza $\vec{t} = t_1, \dots, t_k$ tale che per ogni $i = 1, \dots, n$ l'istante $r_i \in I(t_j)$ per qualche $j = 1, \dots, k$, dove $I(t_j)$ indica l'intervallo di un'ora con inizio in t_j . Il costo è k .

La sottostruttura ottima vale avendo le richieste e, se abbiamo fatto la scelta minore in partenza tra turni e richieste, di sicuro questa tuttora si mantiene.

Sottostruttura ottima Vale la sottostruttura ottima. Infatti se $\vec{t} = t_1, \dots, t_k$ è ottima per le richieste $\vec{r} = r_1, \dots, r_n$ allora $t_2, \dots, t_k$ è ottima per il sottoproblema $\vec{r}'$ che comprende le richieste r_i tali che $r_i \notin I(t_1)$. Infatti, se ci fosse un insieme di turni di dimensione minore di $k-1$ per servire le richieste in $\vec{r}'$, aggiungendo t_1 otterremmo una soluzione del problema originale $\vec{r}$ migliore di $\vec{t}$.

Scelta greedy Assumiamo che la lista di richieste $\vec{r} = r_1, \dots, r_n$ sia ordinata in modo crescente. La scelta greedy consiste nel considerare come primo turno $t_1 = r_1$.

Esiste sempre una soluzione ottima che la contiene. Infatti se $\vec{t}' = t'_1, \dots, t'_k$ è una qualsiasi soluzione ottima e supponiamo che i turni siano ordinati anch'essi in senso crescente, allora certamente $t'_1 \leq r_1$, dato che la prima richiesta deve essere servita. Quindi se sostituiamo t'_1 con $t_1 = r_1$, otteniamo una nuova soluzione (tutte le richieste servite da t'_1 sono anche servite da t_1 !), anch'essa ottima.

Ne segue l'algoritmo che riceve in input l'array delle richieste `r[1..n]` (che si assume non vuoto), e fornisce in uscita `t[1, k]`.

`time(r, n)`

`t[1] = r[1]`

`turni=1`

for `i = 2 to n`

if `t[turni] < r[i]`

`turni++`

`t[turni] = r[i]`

return `t`

Salviamo nel vettore dei turni, la prima posizione delle richieste, in maniera tale da avere almeno una richiesta assegnata, ovviamente nel caso in cui non ce ne siano altre.

La scelta greedy consiste nel scegliere la prima richiesta ed assegnare le altre, conoscendo già il tempo delle richieste in modo crescente (considerando ovviamente tutti i turni).

Quindi, se abbiamo il turno con una richiesta che ci mette poco, la assegniamo e ritorniamo l'array di turni ottimi.

La complessità è $\Theta(n)$.

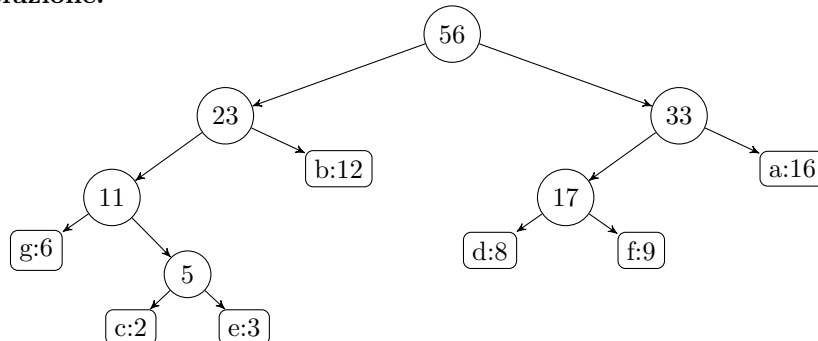
Domanda 39 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
16	12	2	8	3	9	6

Huffman prevede di costruire un albero partendo dalle foglie del livello più basso che hanno i valori numerici più bassi e, per somma, si ottengono le radici composte a loro volta per somma da altri nodi di valore numerico crescente.

Spiegare il processo di costruzione del codice.

Soluzione:

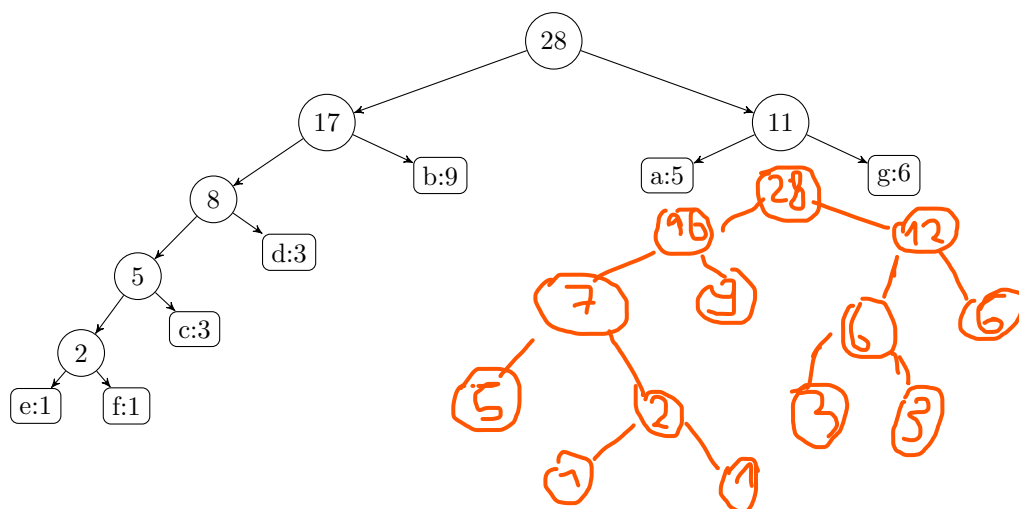


Domanda 40 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
5	9	3	3	1	1	6

Spiegare il processo di costruzione del codice.

Soluzione:

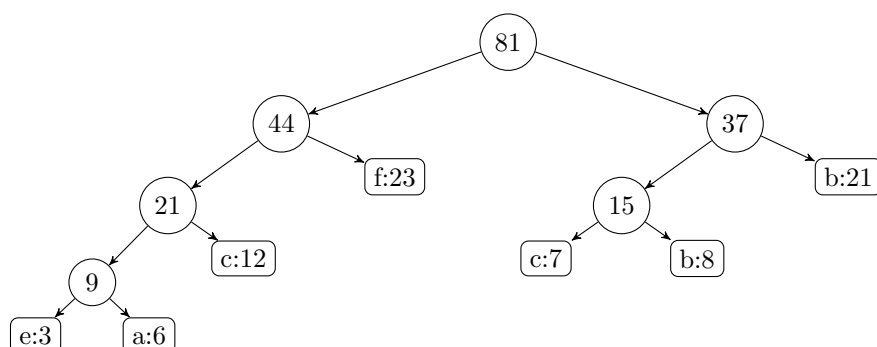


Domanda 41 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
6	21	12	8	3	23	8

Spiegare il processo di costruzione del codice.

Soluzione:

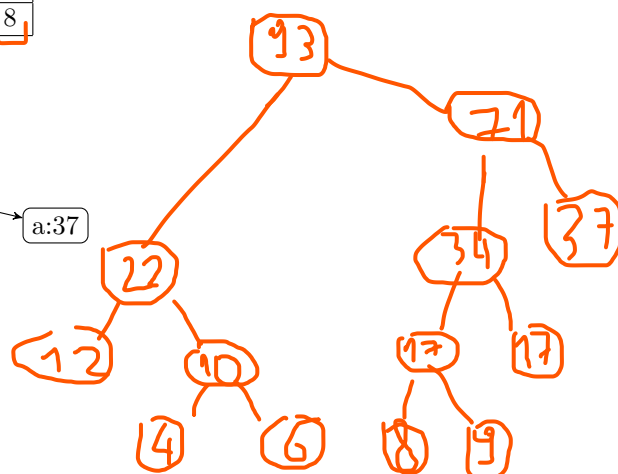
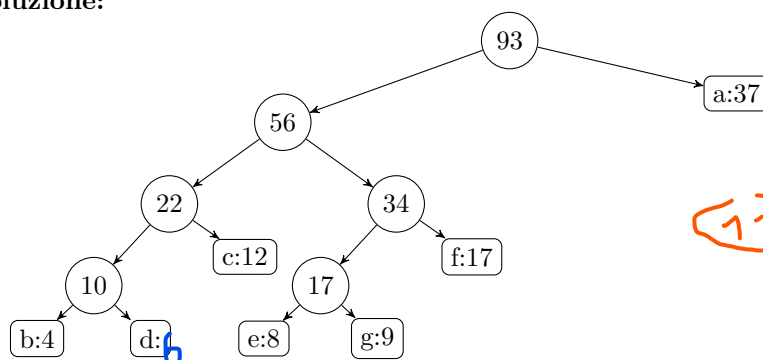


Domanda 42 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
37	4	12	6	9	17	8

Spiegare il processo di costruzione del codice.

Soluzione:

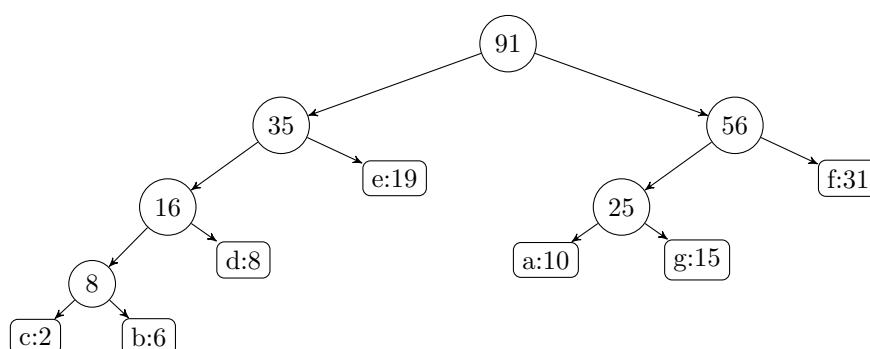


Domanda 43 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
10	6	2	8	19	31	15

Spiegare il processo di costruzione del codice.

Soluzione:

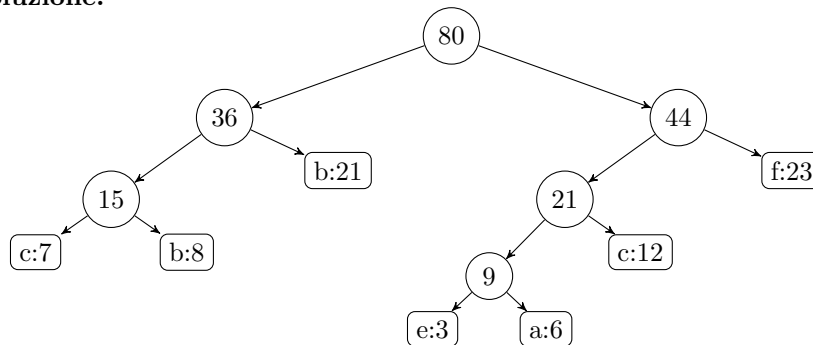


Domanda 44 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
3	8	7	12	6	23	21

Spiegare il processo di costruzione del codice.

Soluzione:

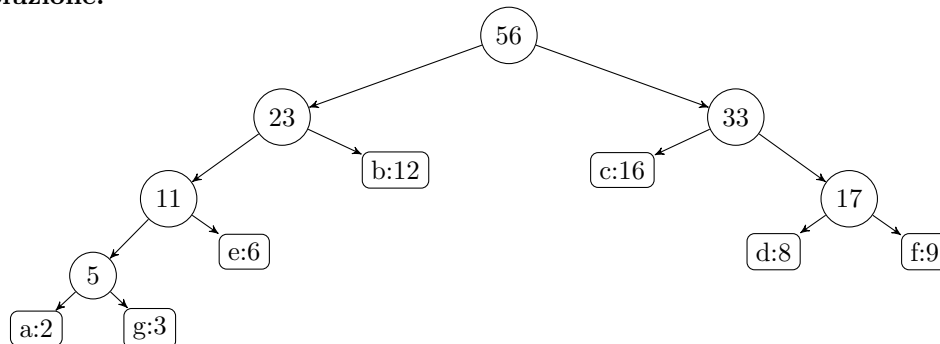


Domanda 45 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
2	12	16	8	6	9	3

Spiegare il processo di costruzione del codice.

Soluzione:



Domanda B (6 punti) Si consideri un insieme di 7 attività $a_i, 1 \leq i \leq 7$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (1, 4, 2, 3, 7, 8, 11)$$

$$\mathbf{f} = (3, 6, 9, 10, 11, 12, 13).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_2, a_5, a_7\}$.

Esercizio 2 (9 punti) Lungo una strada ci sono, in vari punti, n parcheggi liberi e n auto. Un posteggiatore ha il compito di parcheggiare tutte le auto, e lo vuole fare minimizzando lo spostamento totale da fare. Formalmente, dati n valori reali $p_1, p_2, \dots, p_n$ e altri n valori reali $a_1, a_2, \dots, a_n$, che rappresentano

le posizioni lungo la strada rispettivamente di parcheggi e auto, si richiede di assegnare ad ogni auto a_i un parcheggio $p_{h(i)}$ minimizzando la quantità

$$\sum_{i=1}^n |a_i - p_{h(i)}|.$$

L'idea di questo algoritmo, se fosse in codice, sarebbe simile a Metric Matching, quindi cerco di minimizzare la differenza partendo dalla fine (quindi, vedendo quante auto e quanti parcheggi ci sono) e verificando quale, a coppie, ha la minima differenza.

1. Si consideri il seguente algoritmo greedy. Si individui la coppia (auto, parcheggio) con la minima differenza. Si assegni quell'auto a quel parcheggio. Si ripeta con le auto e i parcheggi restanti fino a quando tutte le auto sono parcheggiate. Dimostrare che questo algoritmo non è corretto, esibendo un controesempio.
2. Si consideri il seguente algoritmo greedy. Si assuma che i valori $p_1, p_2, \dots, p_n$ e $a_1, a_2, \dots, a_n$ siano ordinati in modo non decrescente. Si produca l'assegnazione $(a_1, p_1), (a_2, p_2), \dots, (a_n, p_n)$. Dimostrare la correttezza di questo algoritmo per il caso $n = 2$.

Questo significa che l'assegnazione parte dagli ultimi parcheggi e dalle ultime auto (quindi, massima somma al contrario significa minima differenza). Essendo elementi a coppia, si possono "mischiare" le assegnazioni come si vede sotto.

Soluzione:

1. Si consideri il seguente input:

$$p_1 = 5, p_2 = 10 \quad \text{e} \quad a_1 = 9, a_2 = 14.$$

L'algoritmo produce l'assegnazione $(a_1, p_2), (a_2, p_1)$, che ha costo $1 + 9 = 10$, mentre l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $4 + 4 = 8$.

2. Ci sono vari casi possibili:

Dal ragionamento detto, matematicamente, si vede che basta prendere un qualsiasi ordinamento tra le due auto e i due parcheggi di due generiche differenze e si esprime la somma in termini matematici (l'idea concreta è quella spiegata da me).

(a) Caso $a_1 \leq p_1 \leq p_2 \leq a_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + a_2 - p_2 = (a_2 - a_1) - (p_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (a_2 - a_1) + (p_2 - p_1)$; siccome $p_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(b) Caso $a_1 \leq p_1 \leq a_2 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) - (a_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (p_2 - a_1) + (a_2 - p_1)$; siccome $a_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(c) Caso $a_1 \leq a_2 \leq p_1 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) + (p_1 - a_2)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + p_1 - a_2 = (p_2 - a_1) + (p_1 - a_2)$, uguale a quello precedente

Tutti gli altri casi sono simmetrici e si dimostrano nella stessa maniera.

Domanda B (6 punti) Si consideri un insieme di 8 attività $a_i, 1 \leq i \leq 8$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (1, 1, 2, 4, 2, 5, 6, 9)$$

$$\mathbf{f} = (3, 4, 5, 6, 7, 9, 10, 12).$$

[Si veda la risoluzione di pag. 52 per capire come ragionarlo](#)

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_4, a_7\}$.

Esercizio 2 (9 punti) Si consideri un file definito sull'alfabeto $\Sigma = \{a, b, c\}$, con frequenze $f(a), f(b), f(c)$. Per ognuna delle seguenti codifiche si determini, se esiste, un opportuno assegnamento di valori alle 3 frequenze $f(a), f(b), f(c)$ per cui l'algoritmo di Huffman restituisce tale codifica, oppure si argomenti che tale codifica non è mai ottenibile.

1. $e(a) = 0, e(b) = 10, e(c) = 11$

[Si veda la risoluzione di pag. 94 completa di questo esercizio](#)

2. $e(a) = 1, e(b) = 0, e(c) = 11$

3. $e(a) = 10, e(b) = 01, e(c) = 00$

Soluzione:

1. Questa codifica viene restituita dall'algoritmo di Huffman quando $f(b), f(c) < f(a)$: in questo caso i nodi associati a b e c vengono uniti creando un nuovo nodo interno, che poi viene unito al nodo associato ad a . Quindi, per esempio, $f(a) = 40, f(b) = 25, f(c) = 35$.
2. La codeword $e(a) = 1$ è un prefisso della codeword $e(c) = 11$, cioè questa codifica non è libera da prefissi, e quindi non è una codifica di Huffman.
3. L'albero associato a questa codifica non è pieno perché c'è un nodo interno con un solo figlio (quello nel cammino che porta alla foglia associata al carattere a). Quindi questa codifica non è ottima e quindi non è una codifica di Huffman.

Esercizio 2 (10 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k

sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un $(2,1)$ -boxing che minimizza il numero di sottoinsiemi della partizione.

1. Scrivere il codice di un algoritmo greedy che restituisce un $(2,1)$ -boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

(2-1) boxing: ogni sottoinsieme contiene al più 2 valori la cui somma è al più 1. Si chiede di minimizzare il numero di sottoinsiemi della partizione. Per fare ciò, la somma è al più 1 considerando per certo gli estremi (inferiore e superiore). Grazie a questi, sappiamo per certo che, prendendo il valore più grande (sup) e il valore più piccolo (inf), la scelta funziona.

Soluzione:

1. L'idea è provare ad accoppiare il numero più piccolo (a_1) con quello più grande (a_n). Se la loro somma è al massimo 1, allora $S_1 = \{a_1, a_n\}$, altrimenti $S_1 = \{a_n\}$. Poi si procede analogamente sul sottoproblema $S \setminus S_1$.

(2,1)-BOXING(S)

```

n <- |S|
P <- empty_set
first <- 1
last <- n
while (first <= last)
  if (first < last) and a_first + a_last <= 1 then
    P <- P U {{a_first, a_last}}
    first <- first + 1
  else
    P <- P U {{a_last}}
    last <- last - 1
return P
```

Inizializziamo l'insieme, la partizione e gli estremi.

Esempio:
1 2 3 4 5 6 7
P = (7, 6, 5, 4, 3, 2, 1)

Si considera un ciclo per cui "first" è <= "last" (perché scansioniamo gli estremi, come detto)

Se l'estremo inf. è < dell'estremo sup.
non abbiamo ancora salvato nulla in P (non abbiamo estr. inf)
allora salvo in P sia l'estremo inf che l'estremo sup
e incremento first. Salvandolo una volta sola, so che la somma è sempre > 1.

Altrimenti,
salvo solo l'estremo superiore migliore, la cui somma
è sempre >= 1

Questo algoritmo scansiona ogni elemento una sola volta, quindi la sua complessità è lineare.

2. La scelta greedy è $\{a_1, a_n\}$ se $n > 1$ e $a_1 + a_n \leq 1$, altrimenti $\{a_n\}$. Ora dimostriamo che esiste sempre una soluzione ottima che contiene la scelta greedy. I casi $n = 1$ e $a_1 + a_n > 1$ sono banali, visto che in questi casi ogni soluzione ammissibile deve contenere il sottoinsieme $\{a_n\}$. Quindi assumiamo che la scelta greedy sia $\{a_1, a_n\}$. Consideriamo una qualsiasi soluzione ottima dove a_1 e a_n non sono accoppiati nello stesso sottoinsieme. Quindi, esistono due sottoinsiemi S_1 e S_2 , con $a_1 \in S_1$ e $a_n \in S_2$. Sostituuiamo questi due sottoinsiemi con $S'_1 = \{a_1, a_n\}$ (cioè, la scelta greedy) e $S'_2 = S_1 \cup S_2 \setminus \{a_1, a_n\}$. $|S'_2| \leq 2$ e, se $|S'_2| = 2$, allora $S'_2 = \{a_s, a_t\}$ con $a_s \in S_1$ e $a_t \in S_2$. Siccome a_t era precedentemente accoppiato con a_n , a maggior ragione può essere accoppiato con $a_s < a_n$, quindi la nuova soluzione così creata è ammissibile e ancora ottima.

Spiegata in termini semplici: la scelta greedy consiste nel scegliere quello che sta agli estremi. Esiste sempre una soluzione ottima in quanto il sottoinsieme che consideriamo è sempre formato da almeno due elementi (tolto il caso base) per cui tra questi ci siano i nostri estremi inf. e sup. Andando a fare le differenze ottime (scegliendo di volta in volta gli estremi migliori), di sicuro avremo ancora una differenza ottima, perché scegliamo gli estremi migliori (più piccolo per inf. e più grande per sup) in ogni momento.

Esercizio 2 (10 punti) Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza ℓ_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione. Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{j_i}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$. L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_{j=1}^n C_j(\sigma)$.

- Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
- Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Abbastanza laconico, ma l'idea è che, ordinando come detto, poi se la somma deve essere minima, facciamo un algoritmo che ricalca proprio la selezione delle attività, quindi prendendo il programma che completa prima. La complessità è $O(n \log n)$ perché evidentemente il prof usa MergeSort

Soluzione:

- Ordina i programmi per lunghezza crescente. Complessità: $O(n \log n)$.
- La scelta greedy consiste nello scegliere, come prossimo programma da eseguire, quello di lunghezza minima. Sia σ^* una soluzione ottima. Se il programma di lunghezza minima è il primo in σ^* , abbiamo finito. Consideriamo quindi il caso in cui il programma di lunghezza minima sia in posizione $k > 1$ in σ^* . Costruiamo una nuova soluzione σ' scambiando, in σ^* , il k -esimo programma con il primo. Possiamo osservare che:
 - l'insieme dei primi k programmi $j_1, j_2, \dots, j_k$ è lo stesso in σ^* e σ' , quindi il k -esimo programma ha lo stesso tempo di completamento in σ^* e σ' ; lo stesso vale per tutti i programmi successivi al k -esimo, visto che lo scambio non influisce su di loro;
 - per quanto riguarda tutti gli altri programmi, cioè quelli fino alla posizione $k-1$, questi hanno un tempo di completamento inferiore o uguale in σ' , perché lo scambio può solo avere ridotto la lunghezza del primo programma.

Quindi

$$\sum_{j=1}^n C_j(\sigma') \leq \sum_{j=1}^n C_j(\sigma^*);$$

siccome σ^* è una soluzione ottima, allora deve valere che

$$\sum_{j=1}^n C_j(\sigma') = \sum_{j=1}^n C_j(\sigma^*),$$

cioè anche σ' è una soluzione ottima.

Quindi:
- caso base, abbiamo già il programma ottimo.
- caso induttivo, avendo che ordiniamo i programmi, ogni nostro programma con tempo di completamento minimo ricalca la somma di programmi con completamento minimo, anche matematicamente.

Ciò detto, avremo per forza una serie ottima.